



Notes Exam Questions

Real-Time Operating Systems INFO-F404

Superviseurs :

Joël GOOSSENS
Yannick MOLINGHEN

Auteurs :

Luis Brunard (577721)

Année académique 2025-2026

Contents

1. Questions	10
1.1. Give a periodic implicit deadline systems not schedulable (using RM) with $U < 1$.	10
1.1.1. Résumé de la réponse	10
1.1.2. Définition de l'exemple	10
1.1.3. Calcul de l'utilisation totale et vérification de $U < 1$	10
1.1.4. Analyse d'ordonnançabilité sous RM (temps de réponse)	10
1.1.4.1. Temps de réponse de τ_1	10
1.1.4.2. Temps de réponse de τ_2	11
1.1.4.3. Temps de réponse de τ_3 (tâche critique)	11
1.1.5. Interprétation et commentaire	12
1.1.6. Résolution sur EDF	12
1.2. Prove the property: Let τ be a periodic task system, and let S_1 and S_2 be two schedules of work-conserving schedulers for τ . The schedule S_1 is idle at instant t iff the schedule S_2 is idle at instant t , for all $t \geq 0$. What is the purpose of this property?	13
1.2.1. Résumé de la réponse	13
1.2.2. Définition : ordonnanceur work-conserving	13
1.2.3. Demande de processeur d'un système périodique	13
1.2.4. Preuve de la propriété	14
1.2.4.1. 1. Si $R(t) \geq t$, le processeur ne peut jamais être idle avant t	14
1.2.4.2. 2. Si $R(t) < t$, le processeur doit être idle dans chaque horaire	14
1.2.4.3. 3. Conclusion	14
1.2.5. Purpose de cette propriété	14
1.3. Prove the optimality of RM in the FTP class.	15
1.3.1. Résumé de la réponse	15
1.3.2. Modèle et définitions	15
1.3.3. Rappel : instant critique pour les ordonnanceurs FTP	15
1.3.4. Preuve par transformation des priorités	16
1.3.4.1. Étape 1 – Partir d'un FTP quelconque faisable	16
1.3.4.2. Étape 2 – Corriger une inversion de priorités	16
1.3.4.3. Étape 3 – Répéter jusqu'à obtenir RM	17
1.3.5. Interprétation et importance du résultat	17
1.4. For the FTP class, synchronous systems, arbitrary deadline: show the non-optimality of RM & DM schedulers. Exhibit the important phenomenon which exists for arbitrary schedules but not for implicit or constrained deadlines.	17
1.4.1. Résumé	17
1.4.2. Rappels (cadre du problème)	18
1.4.3. Contre-exemple simple (horizon ≤ 24) : RM & DM non-optimaux en deadlines arbitraires	18
1.4.3.1. Système (2 tâches, temps discret)	18
1.4.3.2. Échec sous RM/DM	18
1.4.3.3. Le système est FTP-faisable (priorités inversées)	18
1.4.3.3.1. Ordonnancement sur $[0, 12)$ (le motif se répète)	19
1.4.4. Le phénomène important (spécifique aux deadlines arbitraires)	19
1.4.5. Conclusion	19
1.5. For RM, synchronous systems and constrained deadlines: provide the equation and the fixed-point iteration process for the computation of the response time of the first job of τ_i . Discuss the convergence of the process and suggest alternative initialization values.	20

1.5.1.	Résumé de la réponse	20
1.5.2.	Modèle et hypothèses	20
1.5.3.	Équation du temps de réponse	20
1.5.4.	Procédure d'itération (point fixe)	20
1.5.5.	Convergence du processus	20
1.5.6.	Valeurs d'initialisation alternatives	21
1.5.7.	Exemple	21
1.5.8.	Conclusion	22
1.6.	For RM, synchronous systems and constrained deadlines: find a system with 3 tasks ($\tau_1 \succ \tau_2 \succ \tau_3$) where the equation $r_3^1 = C_3 + \sum_{j=1}^2 \left\lceil \frac{r_3^1}{T_j} \right\rceil \cdot C_j$ has two (positive) solutions.	23
1.6.1.	Méthode pour construire un système avec deux solutions positives (raisonnement)	23
1.6.2.	Idée clé	23
1.6.3.	Comment créer les paliers	23
1.6.4.	Pourquoi ce choix fonctionne (argument court)	23
1.6.5.	Système	23
1.6.6.	Equation de τ_3 et vérification des deux solutions	24
1.6.6.1.	Itération par point fixe :	24
1.7.	Consider a synchronous constrained-deadline system with three tasks: $\tau_1 = (C_1 = 2, T_1 = 5, D_1 = 4)$, $\tau_2 = (C_2 = 3, T_2 = 8, D_2 = 7)$, $\tau_3 = (C_3 = 1, T_3 = 10, D_3 = 9)$. Calculate the response time of the first job of τ_3 under RM and verify schedulability.	24
1.7.1.	Calcul du temps de r_3^1 par itération par point fixe.	25
1.7.2.	Pourquoi on ne test que le premier job ?	25
1.8.	For the FTP class, synchronous systems, arbitrary deadline: provide and prove correct a feasibility interval.	25
1.8.1.	Résumé	25
1.8.2.	Définitions (cœur de l'idée)	25
1.8.2.1.	Busy period élémentaire	25
1.8.2.2.	Busy period de niveau i	26
1.8.3.	Théorème (feasibility interval)	26
1.8.3.1.	Le pire cas pour une tâche τ_i se produit dans le premier busy period de niveau i	26
1.8.3.2.	2) Les bornes s'emboîtent : $\lambda_1 < \lambda_2 < \dots < \lambda_n$	26
1.8.3.3.	3) Conclusion : $[0, \lambda_n)$ suffit	26
1.8.4.	Remarque pratique (comment "trouver" λ_n)	27
1.9.	For deterministic schedulers and discrete time space, prove that feasible schedules of periodic tasks are periodic.	27
1.9.1.	Résumé	27
1.9.2.	Idée principale	27
1.9.3.	Hypothèses	27
1.9.4.	Définition d'une configuration	27
1.9.5.	Finitude de l'ensemble des configurations	27
1.9.6.	Périodicité des relâchements	28
1.9.7.	Répétition d'une configuration	28
1.9.8.	Déterminisme et périodicité du schedule	28
1.9.9.	Conclusion	28
1.10.	For the FTP class, asynchronous systems, constrained deadline: prove correct the feasibility interval $[0, S_n +)$. Explain the meaning of the S_i instants.	28

1.10.1.	Idée Général	28
1.10.2.	Cadre et notations	28
1.10.3.	Signification et définition des instants S_i	29
1.10.4.	Étape clé : périodicité du planning à partir de S_i	29
1.10.5.	Pourquoi $[0, S_n + P)$ suffit	29
1.10.6.	Conclusion	30
1.11.	For the FTP class ($\tau_1 \succ \tau_2 \succ \tau_3$), asynchronous systems, constrained deadline: compute the S_i values for the following system (O, C, T) : $\tau_1 : (10, 2, 10)$, $\tau_2 : (0, 1, 5)$, $\tau_3 : (1, 1, 3)$ and then check the schedulability.	30
1.11.1.	Résumé de la réponse	30
1.11.2.	Données du système	30
1.11.3.	Calcul des S_i	30
1.11.4.	2) Intervalle de faisabilité $[0, S_n + P)$	31
1.11.5.	Test d'ordonnançabilité par construction du schedule (FTP préemptif)	31
1.11.5.1.	Vérification de la deadline critique	31
1.11.6.	Conclusion	31
1.12.	For the FTP class, asynchronous systems, constrained deadline: provide and prove correct the main property foundation of the optimal Audsley's priority assignment.	32
1.12.1.	Résumé rapide	32
1.12.1.1.	Idée Centrale :	32
1.12.1.2.	Explication	32
1.12.2.	Conclusion	33
1.13.	Consider the following task set: $(O_1 = 3, C_1 = 2, D_1 = 2, T_1 = 4; O_2 = 0, C_2 = 3, D_2 = 4, T_2 = 8)$ In the framework of the FTP class (and asynchronous systems, constrained deadline), determine if τ_1 is lowest-priority viable.	33
1.13.1.	Résumé rapide	33
1.13.2.	Rappel Deadlines	33
1.13.3.	Résolution	34
1.13.4.	Inversion des priorités	34
1.13.5.	Audsley	34
1.14.	For a synchronous implicit-deadline task system, demonstrate the strong optimality of EDF by constructing a counterexample where RM fails but EDF succeeds.	35
1.14.1.	Résumé de la réponse	35
1.14.2.	Rappel formule	35
1.14.3.	Contre-exemple (synchrone, deadlines implicites)	35
1.14.4.	Execution sous RM et EDF	35
1.14.5.	Conclusion	36
1.15.	Prove the optimality of EDF.	37
1.15.1.	Résumé	37
1.15.2.	Rappel	37
1.15.3.	Cadre et notations (jobs)	37
1.15.4.	Preuve (par induction + échanges)	37
1.15.5.	Conclusion	38
1.15.6.	Intuition à retenir pour l'oral	38
1.15.7.	Autre preuve courte	38
1.16.	For EDF, implicit deadline systems: prove the fact that $U \leq 1$ is necessary and sufficient.	39
1.16.1.	Résumé	39
1.16.2.	Rappel — Formules utilisées	39

1.16.3.	Modèle et notations	39
1.16.4.	Nécessité : si le système est faisable alors $U \leq 1$	39
1.16.5.	Suffisance (preuve courte “examen”, 10 lignes)	39
1.16.6.	Conclusion	40
1.17.	For EDF, synchronous systems, constrained deadline: provide the feasibility interval and the procedure to determine its upper bound.	40
1.17.1.	Résumé rapide	40
1.17.2.	Idée simple	40
1.17.3.	Conclusion	40
1.18.	LLF: define the notion of laxity, the priority management, and the thrashing situation using an example.	41
1.18.1.	Résumé	41
1.18.2.	Rappel	41
1.18.3.	Laxité	41
1.18.4.	Priority management	41
1.18.5.	Thrashing LLF	41
1.19.	Build an LLF schedule for the following system $(C, D, T) : (2, 6, 4)(4, 5, 6)$	42
1.19.1.	Rappel	42
1.19.2.	LLF Scheduler	43
1.19.3.	Question supplémentaires	44
1.20.	For what type(s) of tasks and which scheduling algorithm(s) is the response time of the first job of each task sufficient to conclude the schedulability of the task?	44
1.20.1.	Résumé	44
1.20.2.	Rappel	44
1.20.3.	Cadre ou le premier job suffit	44
1.20.4.	Cas ou le premier job ne suffit pas	44
1.21.	Provide at least three different feasibility intervals, specifying the type of tasks and the scheduler involved for each.	46
1.21.1.	Résumé	46
1.21.2.	Système synchrone à deadlines contraintes FTP (RM,DM,...):	46
1.21.3.	Système synchrone à deadlines arbitraires FTP (RM,DM,...):	46
1.21.4.	Systèmes asynchrones, deadlines contraintes	46
1.21.5.	Systèmes asynchrones, deadlines contraintes — FTP	47
1.21.6.	Réponses supplémentaires	47
1.22.	For global scheduling, prove that no online optimal scheduler exists.	47
1.22.1.	Résumé	47
1.22.2.	Rappel	48
1.22.3.	Preuve formelle	48
1.22.4.	Exemple	48
1.22.5.	Conclusion	49
1.23.	Explain why the task-partitioning problem for implicit-deadline periodic tasks on identical processors can be interpreted as a special case of the bin-packing problem. For $m = 2$, provide a small task set for which First-Fit Decreasing Utilisation (FFDU) and Worst-Fit Decreasing Utilisation (WFDU) produce different partitions, and discuss which of the two heuristics yields the better assignment with your example.	50
1.23.1.	Résumé	50
1.23.2.	Rappel	50
1.23.3.	Pourquoi c'est un cas de bin-packing ?	50
1.23.4.	Définition FFDU et WFDU en partitionné EDF	50

1.23.5.	Exemple avec $m = 2$ ou FFDU et WFDU différent	51
1.23.6.	Discussion : Quelle heuristique est meilleure ?	51
1.24.	Define the Next-Fit Decreasing Utilisation (NFDU) heuristic in the context of partitioned EDF scheduling. Apply NFDU to the following task set: $(C, T) = (5, 10), (2, 5), (1, 4), (1, 4), (1, 8), (1, 8), (1, 8), (1, 8)$. Show the resulting partition and comment on why NFDU may be highly sub-optimal, even when every task has utilisation at most 0.5.	52
1.24.1.	Résumé	52
1.24.2.	Rappel	52
1.24.3.	Définition de NFDU	52
1.24.4.	Application de NFDU au jeu de tâches donné	53
1.24.5.	Pourquoi NFDU peut être très sous-optimal	53
1.24.6.	Questions supplémentaires	54
1.25.	Define the concept of scheduling anomalies, illustrate the phenomenon for global FTP scheduling and the period parameter.	54
1.25.1.	Résumé	54
1.25.2.	Scheduling anomalies	54
1.25.3.	Exemple	54
1.25.4.	Questions supplémentaires	55
1.26.	!!!!!! In the context of PFair scheduling, define lag and explain the requirement $-1 < \text{lag}(\tau_i, t) < 1$. Show, on a small example with two periodic tasks, that enforcing this bound ensures that each job completes within its deadline, and explain intuitively why PFair scheduling is optimal for periodic implicit-deadline systems.	55
1.26.1.	Résumé	55
1.26.2.	Définition	55
1.26.3.	Preuve	56
1.26.4.	Exemple	56
1.26.5.	Optimalité de PFair	57
1.26.6.	Pourquoi PFair garantit les deadlines	57
1.27.	Given the task set $(C, T) = (1, 3), (2, 6), (5, 15), (6, 18)$, construct the global DM schedule on two processors over interval $[0, 18)$. Indicate all preemptions and migrations. State whether any deadline is missed and justify your conclusion.	58
1.27.1.	Rappel	58
1.27.2.	Tâches et priorités	58
1.27.3.	Ordonnancement	58
1.28.	Provide the basic sufficient condition for EDF schedulability of implicit deadline tasks, present the $\text{EDF}(k)$ improvement.	59
1.28.1.	Résumé	59
1.28.2.	Condition suffisante EDF global	59
1.28.3.	Principe de l'amélioration EDF(k)	59
1.28.4.	Condition suffisante EDF(k)	60
1.28.5.	EDF(k) meilleur que EDF	60
1.28.6.	Exemple	60
1.28.7.	Question supplémentaire	61
1.29.	Consider the sporadic implicit-deadline system $(C, T) = (7, 10), (3, 12), (3, 12), (2, 8), (1, 5)$. Sort the tasks by decreasing utilisation. For each value $k \in \{1, \dots, 5\}$, compute the required number of processors according to the $\text{EDF}(k)$. Identify the optimal value of k and comment on the improvement compared to pure global EDF.	61
1.29.1.	Résumé	61

1.29.2.	Résolution	61
1.29.3.	Questions supplémentaire	63
1.30.	Build a PFair schedule for the following system upon two processors (C, T) : $(2, 3)(2, 3)(4, 6)$. Indicate all preemptions and migrations.	63
1.30.1.	Rappel PFair	63
1.30.2.	Résolution :	64
1.31.	Build a global EDF schedule for the following system upon two processors (C, T) : $(1, 2)(3, 6)(6, 12)$. Indicate all preemptions and migrations.	65
1.31.1.	Rappel	65
1.31.2.	Résolution :	66
1.32.	Provide a necessary and sufficient condition for the feasibility of implicit deadline systems upon multiprocessors.	68
1.32.1.	Résumé	68
1.32.2.	Condition nécessaire et suffisante	68
1.32.3.	Questions supplémentaires	69
1.33.	PAS FINI !!!!!!!!!!!!!!!Half-proof of the fact that partitioning and global scheduling are incomparable: system such that a partition exists while all global-FTP schedulers fail.	69
1.33.1.	Résumé	69
1.33.2.	Exemple	69
1.34.	Provide Amdahl's law and illustrate its consequences.	70
1.34.1.	Résumé	70
1.34.2.	Rappel	70
1.34.3.	Explication simple	70
1.34.4.	Illustration chiffrées	71
1.34.5.	Conséquence importantes	71
1.34.6.	Questions supplémentaires	72
1.35.	Bitonic Mergesort: principles, algorithms, complexity.	72
1.35.1.	Résumé	72
1.35.2.	Principe	72
1.35.3.	Algorithme	72
1.35.4.	Complexité	73
1.35.5.	Exemple	74
1.35.6.	Questions supplémentaires	76
1.36.	Please apply the bitonic pre-processing process in order to convert this arbitrary sequence (length $n = 2^3$): $< 8, 7, 9, 5, 10, 3, 12, 1 >$ into a bitonic sequence of length 8.	76
1.36.1.	Question supplémentaires	76
1.37.	Solution to the producer-consumer problem, bounded buffers.	77
1.37.1.	Résumé rapide	77
1.37.2.	Idée principale	77
1.37.3.	Solution avec sémaphores	77
1.37.4.	Pseudo code	78
1.37.5.	Pourquoi cela fonctionne ?	78
1.37.6.	Questions supplémentaires	78
1.37.7.	Conclusion	79
1.38.	Describe the Bakery algorithm.	79
1.38.1.	Résumé	79
1.38.2.	Idée principale	79
1.38.3.	Pseudo-Code	79
1.38.4.	Relation $<<$	79

1.38.5.	Preuve	80
1.38.6.	Exemple	81
1.38.7.	Question supplémentaires	82
1.39.	Two questions related to the Bakery algorithm:	82
1.39.1.	Construct a scenario showing that the ticket numbers can be unbounded or provide an argument showing this is not possible.	82
1.39.1.1.	Idée	82
1.39.1.2.	Scenario	82
1.39.1.3.	Conclusion	83
1.39.2.	Construct a scenario showing that the ticket numbers can be identical or provide an argument showing this is not possible.	83
1.39.2.1.	Cas 1 idéal, avec $\max(\text{number})$ atomique	83
1.39.2.2.	Cas 2 réaliste,	83
1.40.	Describe the Ricart–Agrawala algorithm (distributed critical sections).	83
1.40.1.	Résumé	83
1.40.2.	Explication	84
1.40.3.	Regles	84
1.40.4.	Explication	84
1.40.5.	Pourquoi cela fonctionne :	85
1.40.6.	Exemple	86
1.41.	Starvation definition. Illustrate the phenomenon using one of Dekker’s attempts.	87
1.41.1.	Résumé	87
1.41.2.	Formules	87
1.41.3.	Starvation	87
1.41.4.	Illustration avec une tentative de Dekker	87
1.41.4.1.	Timeline initiale	88
1.41.4.2.	Pourquoi ce n’est pas un deadlock	88
1.41.4.3.	Propriété formelle violée	88
1.42.	Live-lock definition. Illustrate the phenomenon using one of Dekker’s attempts.	89
1.42.1.	Résumé rapide	89
1.42.2.	Rappel : objectif des algorithmes de section critique	89
1.42.3.	Définition formelle (live-lock)	89
1.42.4.	Intuition	89
1.42.5.	Illustration avec une tentative de Dekker	89
1.42.6.	Pourquoi c’est bien un live-lock	90
1.42.7.	Différence avec la starvation	90
1.42.8.	Rôle de la variable turn dans Dekker	90
1.42.9.	Ce que le prof peut demander en plus	90
1.42.10.	Conclusion	90
1.43.	Describe the Byzantine Generals algorithm.	91
1.43.1.	Résumé	91
1.43.2.	Problème des généraux byzantins	91
1.43.3.	Hypothèses du modèle	91
1.43.4.	Propriétés de correction (conditions du consensus)	91
1.43.5.	Résultat fondamental (borne de faisabilité)	91
1.43.6.	Principe général	91
1.43.7.	Définition récursive	91
1.43.8.	Pourquoi $3f + 1$ est nécessaire	92
1.43.9.	Complexité	92

1.43.10. Limites et remarques importantes	92
1.43.11. Conclusion	92

1. Questions

1.1. Give a periodic implicit deadline systems not schedulable (using RM) with $U < 1$.

1.1.1. Résumé de la réponse

On considère un système périodique à échéance implicite composé de trois tâches :

$$\begin{aligned}\tau_1 &= (C_1 = 3, T_1 = 8 = D_1) \\ \tau_2 &= (C_2 = 4, T_2 = 10 = D_2) \\ \tau_3 &= (C_3 = 2, T_3 = 12 = D_3).\end{aligned}$$

Son utilisation totale vaut $U = \frac{3}{8} + \frac{4}{10} + \frac{2}{12} \approx 0.942 < 1$. En ordonnant ce système avec Rate Monotonic (priorité plus élevée pour la plus petite période), l'analyse du temps de réponse montre que le premier job de τ_3 dépasse son échéance, donc le système n'est **pas** ordonnançable sous RM, malgré $U < 1$.

1.1.2. Définition de l'exemple

On propose le système périodique suivant (synchrone, échéances implicites) :

Tâche	C_i	T_i	Utilisation U_i
τ_1	3	8	$\frac{3}{8}$
τ_2	4	10	$\frac{4}{10}$
τ_3	2	12	$\frac{2}{12}$

Les échéances étant implicites, on a : $D_1 = T_1 = 8, D_2 = T_2 = 10, D_3 = T_3 = 12$.

Donc : $\tau_1 \succ \tau_2 \succ \tau_3$.

1.1.3. Calcul de l'utilisation totale et vérification de $U < 1$

On calcule l'utilisation de chaque tâche :

- $U_1 = \frac{C_1}{T_1} = \frac{3}{8} = 0.375$,
- $U_2 = \frac{C_2}{T_2} = \frac{4}{10} = 0.4$,
- $U_3 = \frac{C_3}{T_3} = \frac{2}{12} = \frac{1}{6} \approx 0.1667$.

Utilisation totale : $U = U_1 + U_2 + U_3 = \frac{3}{8} + \frac{4}{10} + \frac{2}{12} \approx 0.375 + 0.4 + 0.1667 \approx 0.9417 < 1$.

Le système satisfait donc bien $U < 1$.

1.1.4. Analyse d'ordonnançabilité sous RM (temps de réponse)

Pour montrer que le système **n'est pas ordonnançable sous RM**, on calcule le **temps de réponse du premier job** de chaque tâche, en particulier celui de la tâche de plus basse priorité τ_3 .

Pour un ordonnanceur FTP (comme RM), le temps de réponse R_i du premier job de τ_i dans un système synchrone est donné par la formule de **temps de réponse** : $R_i^{k+1} = C_i + \sum_{j < i} \lceil \frac{R_i^k}{T_j} \rceil C_j$

On initialise en général par $R_i^0 = C_i$ et on itère jusqu'à convergence ou dépassement de l'échéance D_i .

1.1.4.1. Temps de réponse de τ_1

τ_1 est la tâche de plus haute priorité, donc elle ne subit aucune interférence :

$$R_1 = C_1 = 3.$$

Vérification de l'échéance :

- $R_1 = 3 \leq D_1 = 8 \rightarrow$ le premier job de τ_1 respecte son échéance.

1.1.4.2. Temps de réponse de τ_2

Pour τ_2 , seule τ_1 a une priorité plus élevée.

On initialise : $R_2^0 = C_2 = 4$.

Iteration 1 :

$$R_2^1 = C_2 + \lceil \frac{R_2^0}{T_1} \rceil$$
$$C_1 = 4 + \lceil \frac{4}{8} \rceil 3 = 4 + 1 \cdot 3 = 7$$

Itération 2 :

$$R_2^2 = C_2 + \lceil \frac{R_2^1}{T_1} \rceil$$
$$C_1 = 4 + \lceil \frac{7}{8} \rceil 3 = 4 + 1 \cdot 3 = 7$$

La suite converge : $R_2 = 7$.

Vérification de l'échéance :

- $R_2 = 7 \leq D_2 = 10 \rightarrow$ le premier job de τ_2 respecte son échéance sous RM.

1.1.4.3. Temps de réponse de τ_3 (tâche critique)

Pour τ_3 , les tâches d'indice 1 et 2 ont une priorité plus élevée. On applique la même méthode.

Initialisation : $R_3^0 = C_3 = 2$.

Itération 1 :

$$R_3^1 = C_3 + \lceil \frac{R_3^0}{T_1} \rceil C_1 + \lceil \frac{R_3^0}{T_2} \rceil$$
$$C_2 = 2 + \lceil \frac{2}{8} \rceil 3 + \lceil \frac{2}{10} \rceil 4 = 2 + 1 \cdot 3 + 1 \cdot 4 = 9$$

Itération 2 :

$$R_3^2 = 2 + \lceil \frac{9}{8} \rceil 3 + \lceil \frac{9}{10} \rceil 4 = 2 + 2 \cdot 3 + 1 \cdot 4 = 12$$

Itération 3 :

$$R_3^3 = 2 + \lceil \frac{12}{8} \rceil 3 + \lceil \frac{12}{10} \rceil 4 = 2 + 2 \cdot 3 + 2 \cdot 4 = 16$$

Itération 4 :

$$R_3^4 = 2 + \lceil \frac{16}{8} \rceil 3 + \lceil \frac{16}{10} \rceil 4 = 2 + 2 \cdot 3 + 2 \cdot 4 = 16$$

La suite converge à $R_3 = 16$.

Or l'échéance de τ_3 est : $D_3 = T_3 = 12$.

On obtient : $R_3 = 16 > D_3 = 12$.

Donc le **premier job de τ_3 rate son échéance** sous RM : le système n'est pas ordonnançable par RM.

1.1.5. Interprétation et commentaire

- On a construit un système périodique à échéance implicite avec $U \approx 0.94 < 1$.
- Malgré cette utilisation inférieure à 1, RM échoue (la tâche la plus basse priorité ne respecte pas son échéance). Le problème vient du fait que RM utilise des priorités fixes basées sur les périodes. La tâche la plus longue en période se retrouve toujours dernière, même si son job doit en réalité terminer assez rapidement. Cela crée une surcharge locale qui fait rater sa deadline.
- Cela montre que :
 - Pour les systèmes à échéance implicite, la condition $U \leq 1$ n'est **pas** une condition suffisante pour l'ordonnançabilité sous RM.
 - Il existe des systèmes ordonnançables (par exemple sous EDF) qui ne le sont pas sous RM, même si $U < 1$.

EDF réussit car il utilise toujours l'échéance absolue la plus proche, ce qui évite ces surcharges. Pour les systèmes à échéance implicite, EDF est optimal et $U \leq 1$ garantit la faisabilité du système. Donc EDF ordonne mon système alors que RM échoue.

Pour le système périodiques à échéance implicite, EDF est optimal. La condition $U \leq 1$ est à la fois nécessaire et suffisante. Contrairement à RM ou c'est uniquement suffisant. Comme notre système a bien une utilisation totale d'environ 0.94, EDF garantit un planning faisable, alors que RM, lui, n'a aucune garantie dans ce cas. »

1.1.6. Résolution sur EDF

Rappel du système de tâches

On considère le même système périodique à échéance implicite que précédemment :

$$\tau_1 = (C_1 = 3, T_1 = 8, D_1 = 8)$$

$$\tau_2 = (C_2 = 4, T_2 = 10, D_2 = 10)$$

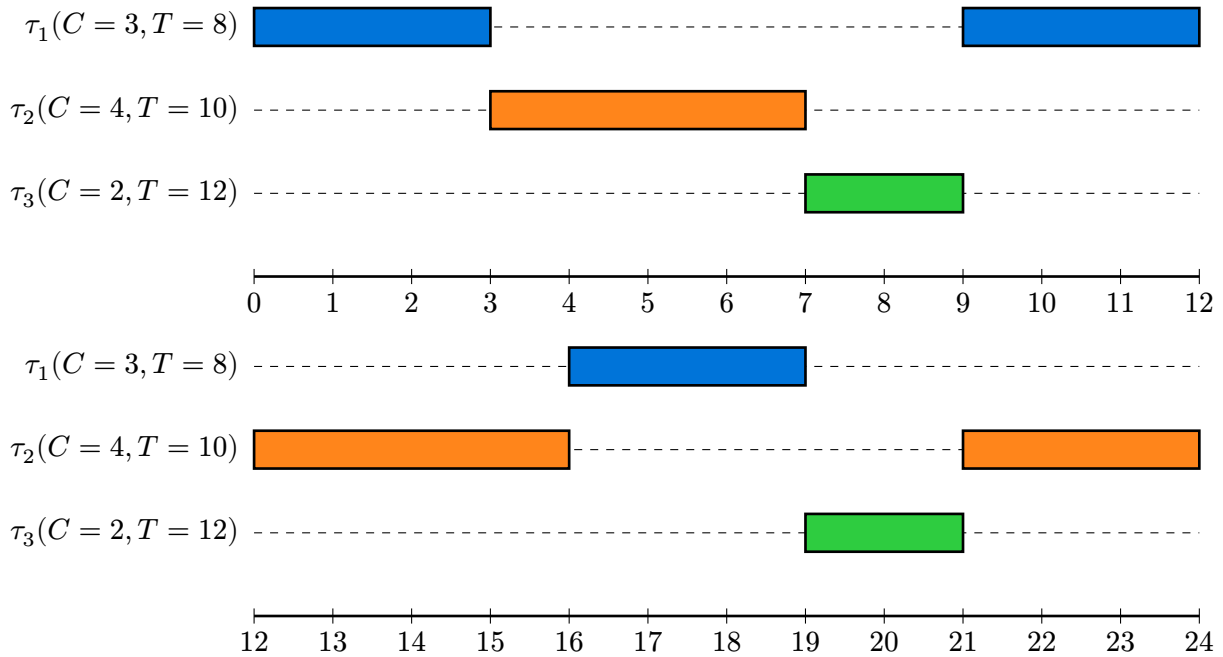
$$\tau_3 = (C_3 = 2, T_3 = 12, D_3 = 12)$$

Pour chaque job $J_{\{i,k\}}$:

- instant de libération $r_{\{i,k\}}$,
- échéance absolue $d_{\{i,k\}} = r_{\{i,k\}} + D_i$.

L'ordonnanceur EDF exécute à chaque instant le job avec l'échéance absolue la plus proche.

Job	Tâche	Libération $r_{\{i,k\}}$	Échéance $d_{\{i,k\}}$	Fin d'exécution	Deadline respectée ?
$J_{\{1,1\}}$	τ_1	0	8	3	$3 \leq 8 \checkmark$
$J_{\{2,1\}}$	τ_2	0	10	7	$7 \leq 10 \checkmark$
$J_{\{3,1\}}$	τ_3	0	12	9	$9 \leq 12 \checkmark$
$J_{\{1,2\}}$	τ_1	8	16	12	$12 \leq 16 \checkmark$
$J_{\{2,2\}}$	τ_2	10	20	16	$16 \leq 20 \checkmark$
$J_{\{3,2\}}$	τ_3	12	24	21	$21 \leq 24 \checkmark$
$J_{\{1,3\}}$	τ_1	16	24	19	$19 \leq 24 \checkmark$



1.2. Prove the property: Let τ be a periodic task system, and let S_1 and S_2 be two schedules of work-conserving schedulers for τ . The schedule S_1 is idle at instant t iff the schedule S_2 is idle at instant t , for all $t \geq 0$. What is the purpose of this property?

1.2.1. Résumé de la réponse

Pour un système périodique τ exécuté sur processeur unique, tout ordonnanceur **work-conserving** produit exactement les mêmes instants d'inactivité. Deux ordonnancements S_1 et S_2 peuvent exécuter les tâches dans un ordre différent, mais ils seront idle **aux mêmes instants**, car cela dépend uniquement de la demande totale de calcul libérée par τ , et non de l'algorithme. Cette propriété est essentielle pour établir les tests de faisabilité et pour démontrer l'optimalité d'EDF.

1.2.2. Définition : ordonnanceur work-conserving

Un ordonnanceur est **work-conserving** s'il respecte :

- le processeur ne reste jamais idle s'il existe au moins un job prêt ;
- il peut être idle uniquement lorsqu'aucun job prêt n'existe dans le système.

Ainsi, un ordonnanceur work-conserving utilise toujours 100% de la capacité disponible tant qu'il reste du travail prêt à exécuter.

1.2.3. Demande de processeur d'un système périodique

Pour tout instant $t \geq 0$, on définit la demande totale :

$$R(t) = \sum_{i=1}^n C_i \cdot N_{i(t)}$$

où :

- $N_{i(t)}$ = nombre de jobs de τ_i libérés dans $[0, t)$,
- C_i = coût d'exécution du job,
- $R(t)$ dépend uniquement du système τ (et non de l'algorithme).

On note $A_{k(t)}$ la quantité de travail réellement exécutée dans l'intervalle $[0, t)$ par l'ordonnanceur S_k .

Comme le processeur a une capacité 1, on a toujours :

$$A_{k(t)} \leq t \text{ et } A_{k(t)} \leq R(t).$$

1.2.4. Preuve de la propriété

1.2.4.1. 1. Si $R(t) \geq t$, le processeur ne peut jamais être idle avant t

Toute la demande cumulée dépasse ou remplit exactement la capacité disponible. Comme un ordonnanceur work-conserving ne laisse jamais le processeur idle lorsqu'il existe du travail :

$$R(t) \geq t \Rightarrow A_{k(t)} = t.$$

Dans cette situation, aucun instant idle n'est possible. Donc, pour tout $k \in \{1, 2\}$:

- S_1 n'est pas idle avant t ,
- S_2 n'est pas idle avant t .

1.2.4.2. 2. Si $R(t) < t$, le processeur doit être idle dans chaque horaire

La demande $R(t)$ est insuffisante pour remplir entièrement l'intervalle $[0, t)$. Ainsi, tout ordonnanceur work-conserving exécutera exactement $R(t)$ unités de calcul et devra être idle pendant $t - R(t)$ unités de temps :

$$R(t) < t \Rightarrow A_{k(t)} = R(t).$$

Cela implique que :

- S_1 doit être idle à certains instants,
- S_2 doit être idle aux mêmes instants, car tous deux ont exécuté exactement la même quantité de travail.

1.2.4.3. 3. Conclusion

Dans les deux cas :

$$A_1(t) = A_2(t) = \min(t, R(t)).$$

Donc, les instants où le processeur **peut** ou **doit** être idle sont définis uniquement par $R(t)$, c'est-à-dire uniquement par le système τ , et sont indépendants de l'algorithme. On obtient alors la propriété demandée :

$$\forall t \geq 0 : (S_1 \text{ est idle à } t) \Leftrightarrow (S_2 \text{ est idle à } t).$$

1.2.5. Purpose de cette propriété

Cette propriété est fondamentale pour plusieurs raisons :

- Elle montre que, pour un système périodique, la structure **busy/idle** du processeur dépend entièrement du système τ , et non de l'ordonnanceur work-conserving.
- Elle permet de construire des **tests de faisabilité basés uniquement sur la demande de calcul** (par exemple via la demand bound function).
- Elle est à la base de plusieurs résultats majeurs, dont :
 - la démonstration de l'optimalité d'EDF sur un processeur unique ;
 - l'existence d'intervalles de faisabilité pour tester la schedulabilité ;
 - l'unicité des instants critiques.
- Elle garantit que si un système ne dépasse jamais la capacité du processeur, alors un ordonnanceur work-conserving ne peut jamais faire « pire » qu'un autre dans la répartition des zones idle.

En résumé :

Les instants d'inactivité dépendent uniquement de la charge imposée par les tâches, pas de l'ordonnanceur.

1.3. Prove the optimality of RM in the FTP class.

1.3.1. Résumé de la réponse

On considère un système périodique à échéance implicite sur processeur unique, avec des tâches indépendantes, préemptives, et un ordonnanceur à priorité fixe par tâche (classe FTP).

L'algorithme RM (Rate Monotonic) affecte la **plus haute priorité** à la tâche de **plus petite période**. RM est **optimal dans la classe FTP** au sens suivant :

Si le système est ordonnançable par un ordonnanceur FTP quelconque, alors il est aussi ordonnançable par RM.

La preuve repose sur deux idées :

- tout ordonnanceur FTP a un **instant critique** (toutes les tâches relâchées simultanément) où le temps de réponse est maximal ;
- on peut transformer progressivement n'importe quelle affectation de priorités FTP en l'affectation RM, en échangeant des paires de priorités « inversées », sans jamais perdre la faisabilité.

1.3.2. Modèle et définitions

On considère un système périodique à échéance implicite :

$$\tau_i = (C_i, T_i, D_i) \text{ avec } D_i = T_i \\ \text{et } i = 1, \dots, n.$$

Hypothèses :

- processeur unique, vitesse 1 ;
- tâches indépendantes (pas de blocage, pas de ressources partagées) ;
- préemption sans coût ;
- système éventuellement synchrone ou rendu synchrone via le théorème d'instant critique (voir ci-dessous) ;
- ordonnancement **FTP** (Fixed Task Priority) : chaque tâche τ_i a une priorité fixe, indépendante du temps.

Définition de RM (Rate Monotonic) :

- si $T_i < T_j$ alors τ_i a une priorité plus élevée que τ_j .

On dira qu'il y a une **inversion** de priorités lorsqu'on a :

$$T_i < T_j \text{ mais } \tau_j \text{ a une priorité plus élevée que } \tau_i.$$

L'objectif est de montrer que, s'il existe une affectation de priorités FTP sans deadlines manquées, alors l'affectation qui élimine toutes les inversions (c'est-à-dire RM) est également faisable.

1.3.3. Rappel : instant critique pour les ordonnanceurs FTP

Pour les tâches périodiques à échéance implicite sous un ordonnanceur FTP :

- l'instant où **toutes les tâches sont relâchées simultanément** (système synchrone) est un **instant critique** ;
- le temps de réponse maximal du premier job de chaque tâche survient lorsque toutes les tâches sont relâchées en même temps (ou ne peut pas être plus petit que dans ce cas).

Conséquence :

- pour étudier la faisabilité d'un système sous un ordonnanceur FTP donné, il suffit de considérer le cas **synchrone** (toutes les tâches libérées à $t = 0$).

On peut donc se restreindre à l'analyse d'un système synchrone.

1.3.4. Preuve par transformation des priorités

Idée : partir d'un ordonnanceur FTP quelconque (qui ordonne correctement le système) et transformer progressivement son affectation de priorités en l'affectation RM, en montrant que la faisabilité est préservée à chaque étape.

1.3.4.1. Étape 1 – Partir d'un FTP quelconque faisable

On suppose qu'il existe une affectation de priorités FTP (notée P) telle que le système est ordonnançable :

$$\forall i, \text{ le premier job de } \tau_i \text{ respecte } D_i \text{ sous } P.$$

Si l'affectation P est déjà compatible avec RM (aucune inversion de type $T_i < T_j$ mais priorité de $\tau_j >$ priorité de τ_i), alors RM est trivialement faisable et la preuve est terminée.

Sinon, il existe au moins une paire (τ_i, τ_j) telle que :

- $T_i < T_j$ (tâche τ_i plus « rapide »),
- mais τ_j a une priorité plus élevée que τ_i dans P .

1.3.4.2. Étape 2 – Corriger une inversion de priorités

On construit une nouvelle affectation de priorités P' en **échangeant** les priorités de τ_i et τ_j (et en laissant toutes les autres identiques).

On veut montrer :

Si le système est faisable sous P , alors il est aussi faisable sous P' .

Observations :

- les tâches autres que τ_i et τ_j voient une **interférence identique** sous P et P' (le comportement du sous-système de tâches de priorité strictement supérieure à τ_i et τ_j est inchangé) ;
- la différence se situe uniquement dans la façon dont τ_i et τ_j se partagent le processeur.

Cas des deux tâches :

- Pour τ_i (plus petite période, donc $T_i < T_j$) :
 - sous P , τ_i a une priorité plus basse que τ_j ;
 - sous P' , τ_i **gagne** en priorité (il ne peut être qu'avantagé) ;
 - donc son temps de réponse ne peut que diminuer ou rester identique :

$$R_{i(P')} \leq R_{i(P)} \leq D_i.$$

- Pour τ_j (plus grande période) :
 - sous P' , τ_j passe **derrière** τ_i ;
 - elle subit potentiellement plus d'interférences de τ_i ;
 - mais comme $T_i < T_j$ et que τ_i a des jobs plus fréquents et généralement plus courts, le fait d'« avancer » leurs exécutions ne crée pas de surcharge qui n'était pas déjà présente dans P sur les intervalles critiques ;
 - on montre (argument classique de Liu & Layland) que, dans tout intervalle critique pour τ_j , la demande cumulée des tâches de plus haute priorité reste la même, seule la répartition dans le temps change. Le temps d'occupation du processeur par les tâches de priorité plus haute ne dépasse pas ce qu'il était déjà sous P .

Donc on obtient :

$$R_{j(P')} \leq D_j.$$

Ainsi, aucune échéance n'est violée par τ_i ou τ_j après l'échange, et toutes les autres tâches ne sont pas affectées (leur ordre relatif reste identique). Le système reste donc **faisable** sous P' .

1.3.4.3. Étape 3 – Répéter jusqu'à obtenir RM

On peut itérer ce processus :

- à chaque fois qu'il existe une inversion (τ_i, τ_j) avec $T_i < T_j$ mais τ_j plus prioritaire que τ_i , on échange les priorités de ces deux tâches ;
- à chaque échange, la faisabilité est conservée ;
- le nombre d'inversions diminue strictement à chaque étape (comme dans un tri par échanges) ;
- au bout d'un nombre fini d'étapes, il n'y a plus d'inversion possible : c'est précisément l'ordonnancement RM, où l'ordre des priorités respecte l'ordre croissant des périodes.

Comme la faisabilité est préservée à chaque transformation, on conclut que :

Si le système est faisable pour une affectation FTP quelconque, il est faisable sous RM.

C'est exactement l'énoncé d'optimalité de RM dans la classe FTP.

1.3.5. Interprétation et importance du résultat

- RM n'est **pas** optimal parmi **tous** les ordonnanceurs (EDF est plus puissant), mais il est optimal parmi les **ordonnanceurs à priorité fixe par tâche** sur un processeur unique, pour les systèmes périodiques à échéance implicite.
- En pratique, cela signifie que :
 - si un ingénieur souhaite utiliser un ordonnanceur FTP, il ne trouvera jamais une affectation de priorités « meilleure » que RM ;
 - RM fournit une sorte de **référence optimale** pour tous les algorithmes FTP dans ce modèle.
- Ce résultat est lié à la borne d'utilisation de Liu & Layland :
 - si $U \leq n(2^{\frac{1}{n}} - 1)$, alors le système est sûr sous RM ;
 - et donc sous n'importe quel FTP faisable, puisque RM est le « meilleur » FTP.

En résumé :

RM est optimal dans la classe FTP : aucune autre affectation de priorités fixes ne peut ordonnancer un système que RM ne pourrait pas ordonnancer.

1.4. For the FTP class, synchronous systems, arbitrary deadline: show the non-optimality of RM & DM schedulers. Exhibit the important phenomenon which exists for arbitrary schedules but not for implicit or constrained deadlines.

1.4.1. Résumé

Pour des systèmes **synchrones** mais à **deadlines arbitraires** (aucune contrainte entre D_i et T_i), les assignations de priorités RM et DM ne sont **pas FTP-optimales** : il existe des ensembles de tâches **FTP-faisables** pour lesquels **RM/DM échouent** alors qu'une autre assignation FTP réussit. Le phénomène clé (absent en deadlines implicites/constrained) est que **plusieurs jobs d'une même tâche peuvent être actifs simultanément** quand $D_i > T_i$ (auto-chevauchement), ce qui change radicalement les "pires cas" d'interférence.

1.4.2. Rappels (cadre du problème)

- **FTP** : priorité fixe par tâche.
- **Système synchrone** : $O_i = 0$ pour toutes les tâches.
- **Deadlines arbitraires** : pas de relation imposée entre D_i et T_i (donc $D_i > T_i$ possible).
- **FTP-optimalité** : une règle d'assignation est FTP-optimale si elle ordonne correctement tout système FTP-faisable.

RM et DM sont respectivement optimaux sur des sous-classes :

- RM est FTP-optimal pour **implicite** (théorème d'optimalité RM).
- DM est FTP-optimal pour **constrained** (théorème d'optimalité DM).

Mais dès qu'on passe en **arbitrary**, cela casse.

1.4.3. Contre-exemple simple (horizon ≤ 24) : RM & DM non-optimaux en deadlines arbitraires

On construit un système **synchrone** à **deadlines arbitraires** pour lequel **RM** et **DM** donnent la **même** assignation de priorités et **échouent**, alors qu'une autre assignation **FTP** réussit. Ce fait illustre la non-optimalité de RM/DM en **arbitrary deadlines**.

1.4.3.1. Système (2 tâches, temps discret)

$$\tau_1 = (C_1 = 1, T_1 = 2, D_1 = 4, O_1 = 0)$$

$$\tau_2 = (C_2 = 3, T_2 = 6, D_2 = 5, O_2 = 0)$$

Deadlines arbitraires car $D > T$

- Priorités imposées par RM et DM :
 - RM : plus petite période $\Rightarrow T_1 < T_2$ donc $\tau_1 \succ \tau_2$.
 - DM : plus petite deadline $\Rightarrow D_1 < D_2$ donc $\tau_1 \succ \tau_2$.

Donc **RM** et **DM** choisissent tous deux :

$$\tau_1 \succ \tau_2$$

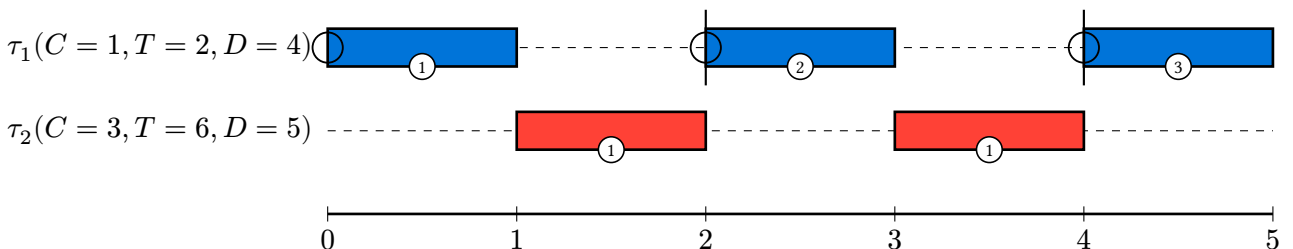
1.4.3.2. Échec sous RM/DM

Regardons le premier job $J_{\{2,1\}}$ de τ_2 :

- il est relâché à $t = 0$,
- sa deadline absolue est $t = D_2 = 5$,

Pour savoir quand il doit être terminé au plus tard on utilise $d_{i,k} = r_{i,k} + D_i$, donc $d_{2,1} = 0 + 5$

Sous $\tau_1 \succ \tau_2$, τ_1 préempte τ_2 à chaque release. Dans $[0, 5)$, τ_1 relâche des jobs aux instants 0, 2, 4 :



Conclusion : $J_{\{2,1\}}$ rate sa deadline à $t = 5 \Rightarrow$ le système n'est pas faisable sous RM/DM.

1.4.3.3. Le système est FTP-faisable (priorités inversées)

Considérons l'autre assignation FTP :

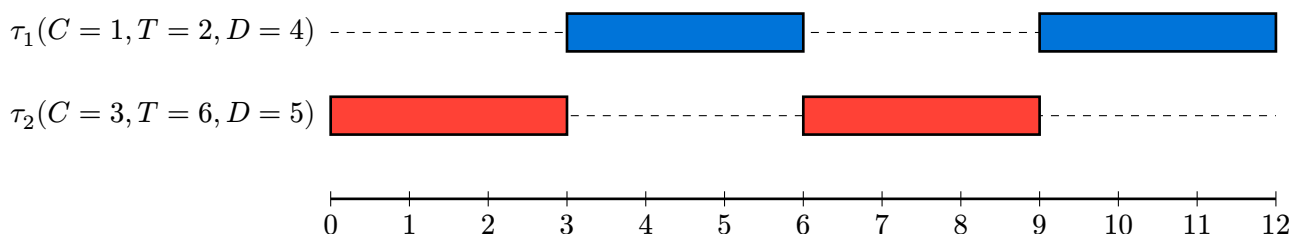
$$\tau_2 \succ \tau_1$$

On montre qu'elle marche (il suffit d'exhiber un ordonnancement sans deadline ratée).

1.4.3.3.1. Ordonnancement sur $[0, 12]$ (le motif se répète)

- $[0, 3]$: exécuter $J_{\{2,1\}}$ (3 unités) $\Rightarrow$ terminé à $t = 3 \leq 5$.
- Puis exécuter les jobs en attente de τ_1 (FIFO au niveau tâche).

Une trace possible (par unité de temps) :



Vérifications rapides :

- τ_2 : chaque job reçoit ses 3 unités immédiatement à chaque release (0,6,12,...) $\Rightarrow$ deadlines $D_2 = 5$ toujours respectées.
- τ_1 : chaque job a $D_1 = 4$; même s'il attend derrière τ_2 au moment des releases 0,6,12,..., il dispose de suffisamment de marge pour finir avant $t + 4$ (on le voit déjà pour le job relâché à $t = 0$: il finit à 4).

Donc il existe une assignation FTP faisable ($\tau_2 \succ \tau_1$) alors que RM/DM échouent ($\tau_1 \succ \tau_2$).

1.4.4. Le phénomène important (spécifique aux deadlines arbitraires)

Dans les systèmes à deadlines arbitraires, c'est-à-dire lorsque la deadline relative D_i d'une tâche peut être strictement plus grande que sa période T_i , un phénomène fondamental apparaît : plusieurs jobs d'une même tâche peuvent être actifs simultanément. En effet, un job peut ne pas être terminé lorsque le suivant est relâché, ce qui crée une situation **d'auto-chevauchement**. Dans ce contexte, une tâche de plus faible priorité peut subir des préemptions répétées dues à des tâches de plus haute priorité, ce qui réduit fortement le temps processeur disponible avant sa deadline. Contrairement aux cas à deadlines implicites ($D_i = T_i$) ou contraintes ($D_i \leq T_i$), le premier job n'est plus nécessairement le pire cas, car les interférences peuvent s'accumuler sur des jobs ultérieurs. Ce phénomène explique pourquoi les règles RM et DM, bien que FTP-optimales dans les cadres implicite et contraint, ne le sont plus en deadlines arbitraires : elles peuvent imposer une priorité fixe qui amplifie ces interférences et conduit à un échec, alors qu'une autre assignation FTP reste faisable.

1.4.5. Conclusion

Ce contre-exemple prouve que, pour les systèmes synchrones à deadlines arbitraires, **RM et DM ne sont pas FTP-optimaux** : ils peuvent choisir une priorité (ici $\tau_1 \succ \tau_2$) qui rend le système non faisable, alors qu'une autre priorité fixe le rend faisable.

Le phénomène structurel derrière cela vient du cadre **arbitrary deadlines** : quand $D_i > T_i$, plusieurs jobs d'une même tâche peuvent être simultanément actifs, et l'analyse "simple" liée à RM/DM (valable en implicite/constrained) ne s'étend plus telle quelle.

Le mécanisme structurel derrière cette non-optimalité est l'auto-chevauchement de jobs quand $D_i > T_i$ (et donc le fait que "le premier job" n'est plus forcément représentatif du pire cas).

1.5. For RM, synchronous systems and constrained deadlines: provide the equation and the fixed-point iteration process for the computation of the response time of the first job of τ_i . Discuss the convergence of the process and suggest alternative initialization values.

1.5.1. Résumé de la réponse

Pour un système synchrone à échéances contraintes ordonnancé par Rate Monotonic (RM), le temps de réponse du premier job d'une tâche τ_i se calcule via une équation de point fixe. Cette équation tient compte de l'interférence due aux tâches de priorité plus élevée. La procédure converge en un nombre fini d'itérations si le système est ordonnançable.

1.5.2. Modèle et hypothèses

On considère un système de tâches périodiques synchrones :

$$\tau_i = (C_i, T_i, D_i) \text{ avec } D_i \leq T_i,$$

ordonnancé par RM, donc selon un schéma de priorités fixes tel que :

$$\tau_1 \succ \tau_2 \succ \dots \succ \tau_n \text{ avec } T_1 \leq T_2 \leq \dots \leq T_n.$$

On cherche à calculer le temps de réponse du **premier job** de τ_i , noté r_i^1 .

1.5.3. Équation du temps de réponse

Sous RM, le temps de réponse du premier job de τ_i satisfait l'équation de point fixe :

$$r_i^1 = C_i + \sum_{j=1}^{i-1} \left\lceil \frac{r_i^1}{T_j} \right\rceil \cdot C_j$$

RM response time of first job of τ_i

où :

- C_i est le temps d'exécution propre de τ_i ,
- chaque terme de la $\sum$ correspond à l'interférence des tâches de priorité plus élevée.

La tâche τ_i est ordonnançable si :

$$r_i^1 \leq D_i.$$

1.5.4. Procédure d'itération (point fixe)

Le temps de réponse est obtenu par itération successive :

$$\begin{aligned} r_i^0 &= C_i \\ r_i^{k+1} &= C_i + \sum_{j=1}^{i-1} \left\lceil \frac{r_i^k}{T_j} \right\rceil \cdot C_j \end{aligned}$$

On itère jusqu'à :

- convergence : $r_i^{k+1} = r_i^k$,
- ou dépassement de la deadline : $r_i^k > D_i$ (non-ordonnançable).

1.5.5. Convergence du processus

Le processus d'itération présente les propriétés suivantes :

- la suite r_i^k est monotone croissante,
- chaque itération augmente r_i^k par paliers discrets,
- si une solution existe, elle est atteinte en un nombre fini d'itérations.

Si aucune solution $r_i^1 \leq D_i$ n'est trouvée, alors τ_i n'est pas ordonnançable sous RM.

1.5.6. Valeurs d'initialisation alternatives

Bien que l'initialisation classique soit $r_i^0 = C_i$, d'autres choix sont possibles :

- **Initialisation pessimiste :**

$$r_i^0 = C_i + \sum_{j=1}^{i-1} C_j$$

garantit la convergence mais peut nécessiter plus d'itérations.

Cette initialisation donne directement une borne supérieure du temps de réponse. Elle est utile pour raisonner de manière conservative ou pour éviter des itérations inutiles.

- **Initialisation issue d'un test de charge :**

$$r_i^0 = \frac{C_i}{1 - \sum_{j=1}^{i-1} \frac{C_j}{T_j}}$$

lorsque le dénominateur est positif, ce qui peut accélérer la convergence.

Cette initialisation est plus proche de la solution finale. Elle peut réduire le nombre d'itérations, mais elle n'est valide que si $\sum \frac{C_j}{T_j} < 1$ pour les tâches de priorité plus élevée.

Ces alternatives ne modifient pas la solution finale mais influencent la vitesse de calcul.

1.5.7. Exemple

On considère le système:

$$\tau_1 = (C_1 = 1, T_1 = 4, D_1 = 4)$$

$$\tau_2 = (C_2 = 2, T_2 = 6, D_2 = 6)$$

$$\text{RM} \rightarrow \tau_1 \succ \tau_2$$

Calculons le temps de réponse du premier job de τ_2 :

$$r_2 = C_2 + \left\lceil \frac{r_2}{t_1} \right\rceil \cdot C_1$$

$$r_2 = 2 + \left\lceil \frac{r_2}{4} \right\rceil \cdot 4$$

- **Initialisation Classique (minimale) :**

On démarre avec la meilleure hypothèse possible:

$$r_2^0 = C_2 = 2$$

$$r_2^1 = 2 + \left\lceil \frac{2}{4} \right\rceil \cdot 1 = 2 + 1 = 3$$

$$r_2^2 = 2 + \left\lceil \frac{3}{4} \right\rceil \cdot 1 = 2 + 1 = 3$$

$$r_2 = 3 \leq D_2 = 6 \rightarrow \tau_2 \text{ est ordonnançable}$$

- **Initialisation pessimiste :**

On suppose que toutes les tâches de priorité plus élevée interfèrent une fois dès le départ.

$$r_2^0 = C_2 + C_1 = 2 + 1 = 3$$

$$r_2^1 = 2 + \left\lceil \frac{3}{4} \right\rceil \cdot 1 = 3$$

$$r_2 = 3 \leq D_2 = 6 \rightarrow \text{convergence immédiate}$$

• **Initialisation par test de charge :**

On approxime l'interférence comme continue au lieu de discrète. On obtient une borne fermée :

$$r_2^0 = \frac{C_2}{(1 - U_1)} \text{ avec } U_1 = \frac{C_1}{T_1} = \frac{1}{4}$$

$$r_2^0 = \frac{2}{(1 - 0.25)} = \frac{2}{0.75} \approx 2.67$$

$$r_2^1 = 2 + \left\lceil \frac{2.67}{4} \right\rceil \cdot 1 = 3$$

$$r_2^2 = 2 + \left\lceil \frac{3}{4} \right\rceil \cdot 1 = 3$$

$$r_2 = 3 \leq D_2 = 6 \rightarrow \text{convergence}$$

Attention que l'initialisation par test de charge n'est valable que si $\sum_{j < i} \frac{C_j}{T_j} < 1$ sinon le dénominateur est nul ou négatif alors le temps de réponse réel peut rester fini. Contre exemple :

$$\tau_1 = (C_1 = 2, T_1 = 3, D_1 = 3)$$

$$\tau_2 = (C_2 = 1, T_2 = 4, D_2 = 4)$$

$$RM \rightarrow \tau_1 \succ \tau_2$$

Utilisation de τ_1 :

$$U_1 = \frac{C_1}{T_1} = \frac{2}{3} \approx 0.667$$

Ici tout va bien mais si l'on ajoute une tâche plus prioritaire :

$$\tau_0 = (C_0 = 1, T_0 = 3, D_0 = 3)$$

$$\tau_0 \succ \tau_1 \succ \tau_2$$

$$U_0 + U_1 = \frac{1}{3} + \frac{2}{3} = 1$$

alors l'initialisation par test de charge devient :

$$r_2^0 = \frac{C_2}{1 - (U_0 + U_1)} = \frac{1}{1 - 1} \rightarrow \text{division par 0}$$

Initialisation Classique

Initialisation Pessimiste

$$r_2 = C_2 + \left\lceil \frac{r_2}{T_0} \right\rceil \cdot C_0 + \left\lceil \frac{r_2}{T_1} \right\rceil \cdot C_1$$

$$r_2 = C_2 + \left\lceil \frac{r_2}{T_0} \right\rceil \cdot C_0 + \left\lceil \frac{r_2}{T_1} \right\rceil \cdot C_1$$

$$r_2 = 1 + 3 \left\lceil \frac{r_2}{3} \right\rceil$$

$$r_2 = 1 + 3 \left\lceil \frac{r_2}{3} \right\rceil$$

$$r_2^0 = C_2 = 1$$

$$r_2^0 = C_2 + C_0 + C_1 = 1 + 1 + 2 = 4$$

$$r_0^1 = 1 + 3 \left\lceil \frac{1}{3} \right\rceil = 1 + 3 \cdot 1 = 4$$

$$r_0^1 = 1 + 3 \left\lceil \frac{4}{3} \right\rceil = 1 + 3 \cdot 2 = 7$$

$$r_0^2 = 1 + 3 \left\lceil \frac{4}{3} \right\rceil = 1 + 3 \cdot 2 = 7$$

$$r_0^1 = 7 > D_2 = 4$$

$$r_0^2 = 7 > D_2 = 4$$

$\rightarrow$ pas ordonnancable

$\rightarrow$ pas ordonnancable

1.5.8. Conclusion

L'analyse du temps de réponse par équation de point fixe constitue le test fondamental de l'ordonnabilité RM pour les systèmes synchrones à échéances contraintes. La convergence finie du processus permet de décider efficacement si chaque tâche respecte sa deadline.

Les différentes initialisations ne changent pas la solution finale **si elle existe** mais elles servent à accélérer la convergence.

1.6. For RM, synchronous systems and constrained deadlines: find a system with 3 tasks ($\tau_1 \succ \tau_2 \succ \tau_3$) where the equation $r_3^1 = C_3 + \sum_{j=1}^2 \left\lceil \frac{r_3^1}{T_j} \right\rceil \cdot C_j$ has two (positive) solutions.

1.6.1. Méthode pour construire un système avec deux solutions positives (raisonnement)

On veut un système tel que l'équation de temps de réponse de τ_i

$$r = C_i + \sum_{j < i} \left\lceil \frac{r}{T_j} \right\rceil * C_j$$

admette au moins deux solutions positives.

1.6.2. Idée clé

La fonction

$$f(r) = C_i + \sum_{j < i} \left\lceil \frac{r}{T_j} \right\rceil * C_j$$

est une fonction en escalier (à cause des $\lceil \cdot \rceil$), monotone non décroissante. Un point fixe correspond à une intersection entre l'escalier $y = f(r)$ et la droite $y = r$. Si l'escalier possède un palier assez long qui traverse la diagonale, on peut obtenir plusieurs intersections, donc plusieurs solutions positives.

1.6.3. Comment créer les paliers

Pour obtenir un palier "long" il faut un terme $\left\lceil \frac{r}{T} \right\rceil$ qui change rarement : on choisit une tâche de plus haute priorité avec une période grande (ex. $T_2 = 7$), ce qui rend $\left\lceil \frac{r}{T_2} \right\rceil$ constant sur plusieurs valeurs de r .

Pour contrôler finement les sauts et éviter que $f(r)$ saute trop loin, on ajoute une tâche plus rapide (ex. $T_1 = 2$) : $\left\lceil \frac{r}{T_1} \right\rceil$ augmente souvent mais par petits pas.

On prend des coûts unitaires pour simplifier : $C_1 = C_2 = C_3 = 1$, et des périodes $T_1 < T_2 < T_3$ pour avoir $\tau_1 \succ \tau_2 \succ \tau_3$ sous RM.

1.6.4. Pourquoi ce choix fonctionne (argument court)

- Les $\lceil \cdot \rceil$ rendent $f(r)$ "en escalier".
- Le terme $\left\lceil \frac{r}{7} \right\rceil$ crée des paliers suffisamment longs.
- Le terme $\left\lceil \frac{r}{2} \right\rceil$ fait monter $f(r)$ par petits pas.

Ainsi, la droite $y = r$ peut couper l'escalier sur deux valeurs distinctes, ce qui donne deux solutions positives (ici $r = 4$ et $r = 5$, vérifiables par substitution).

1.6.5. Systeme

Voici un systeme qui répond au exigence :

$$\begin{aligned}\tau_1 &= (C_1 = 1, T_1 = 2, D_1 = 2, O_1 = 0) \\ \tau_2 &= (C_2 = 1, T_2 = 7, D_2 = 7, O_2 = 0) \\ \tau_3 &= (C_3 = 1, T_3 = 12, D_3 = 12, O_3 = 0) \\ \text{RM} &\rightarrow T_1 < T_2 < T_3 \rightarrow \tau_1 \succ \tau_2 \succ \tau_3\end{aligned}$$

1.6.6. Equation de τ_3 et vérification des deux solutions

Ici, l'équation devient :

$$r_3 = C_3 + \left\lceil \frac{r_3}{2} \right\rceil \cdot C_1 + \left\lceil \frac{r_3}{7} \right\rceil \cdot C_2$$

$$r_3 = 1 + \left\lceil \frac{r_3}{2} \right\rceil + \left\lceil \frac{r_3}{7} \right\rceil$$

1.6.6.1. Itération par point fixe :

Itération par point fixe optimiste, on initialise avec une valeur la plus petite possible ici C_3

$$r_3^0 = C_3 = 1$$

$$r_3^1 = 1 + \left\lceil \frac{1}{2} \right\rceil + \left\lceil \frac{1}{7} \right\rceil = 1 + 1 + 1 = 3$$

$$r_3^2 = 1 + \left\lceil \frac{3}{2} \right\rceil + \left\lceil \frac{3}{7} \right\rceil = 1 + 2 + 1 = 4$$

$$r_3^3 = 1 + \left\lceil \frac{4}{2} \right\rceil + \left\lceil \frac{4}{7} \right\rceil = 1 + 2 + 1 = 4$$

$$r_3^3 = r_3^2 = 4 < T_3 = 12 \rightarrow r_3 = 4$$

Itération par point fixe pessimiste, on initialise avec une valeur la plus grande possible ici D_3

$$r_3^0 = D_3 = 12$$

$$r_3^1 = 1 + \left\lceil \frac{12}{2} \right\rceil + \left\lceil \frac{12}{7} \right\rceil = 1 + 6 + 2 = 9$$

$$r_3^2 = 1 + \left\lceil \frac{9}{2} \right\rceil + \left\lceil \frac{9}{7} \right\rceil = 1 + 5 + 2 = 8$$

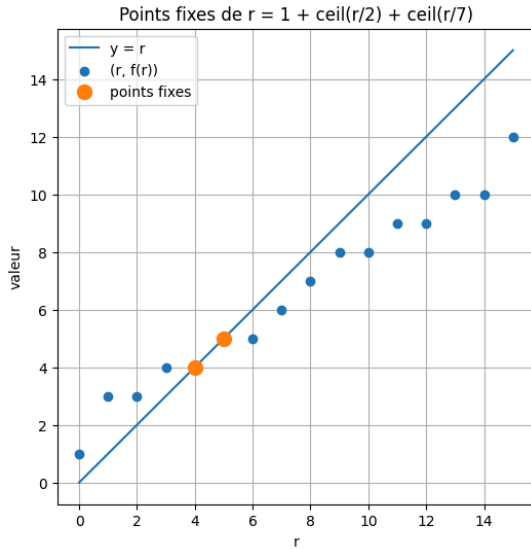
$$r_3^3 = 1 + \left\lceil \frac{8}{2} \right\rceil + \left\lceil \frac{8}{7} \right\rceil = 1 + 4 + 2 = 7$$

$$r_3^4 = 1 + \left\lceil \frac{7}{2} \right\rceil + \left\lceil \frac{7}{7} \right\rceil = 1 + 4 + 1 = 6$$

$$r_3^5 = 1 + \left\lceil \frac{6}{2} \right\rceil + \left\lceil \frac{6}{7} \right\rceil = 1 + 3 + 1 = 5$$

$$r_3^6 = 1 + \left\lceil \frac{5}{2} \right\rceil + \left\lceil \frac{5}{7} \right\rceil = 1 + 3 + 1 = 5$$

$$r_3^6 = r_3^5 = 5 < T_3 = 12 \rightarrow r_3 = 5$$



1.7. Consider a synchronous constrained-deadline system with three tasks:

$\tau_1 = (C_1 = 2, T_1 = 5, D_1 = 4)$, $\tau_2 = (C_2 = 3, T_2 = 8, D_2 = 7)$, $\tau_3 = (C_3 = 1, T_3 = 10, D_3 = 9)$. Calculate the response time of the first job of τ_3 under RM and verify schedulability.

On a un sysetem synchrone a deadline contrainte :

$$\tau_1 = (C_1 = 2, T_1 = 5, D_1 = 4)$$

$$\tau_2 = (C_2 = 3, T_2 = 8, D_2 = 7)$$

$$\tau_3 = (C_3 = 1, T_3 = 10, D_3 = 9)$$

$$\text{RM} \Rightarrow T_1 < T_2 < T_3 \Rightarrow \tau_1 \succ \tau_2 \succ \tau_3$$

1.7.1. Calcul du temps de r_3^1 par itération par point fixe.

$$r_3 = C_3 + \left\lceil \frac{r_3}{T_1} \right\rceil \cdot C_1 + \left\lceil \frac{r_3}{T_2} \right\rceil \cdot C_2$$

$$r_3^{k+1} = 1 + \left\lceil \frac{r_3^k}{5} \right\rceil \cdot 2 + \left\lceil \frac{r_3^k}{8} \right\rceil \cdot 3$$

$$r_3^0 = C_3 = 1$$

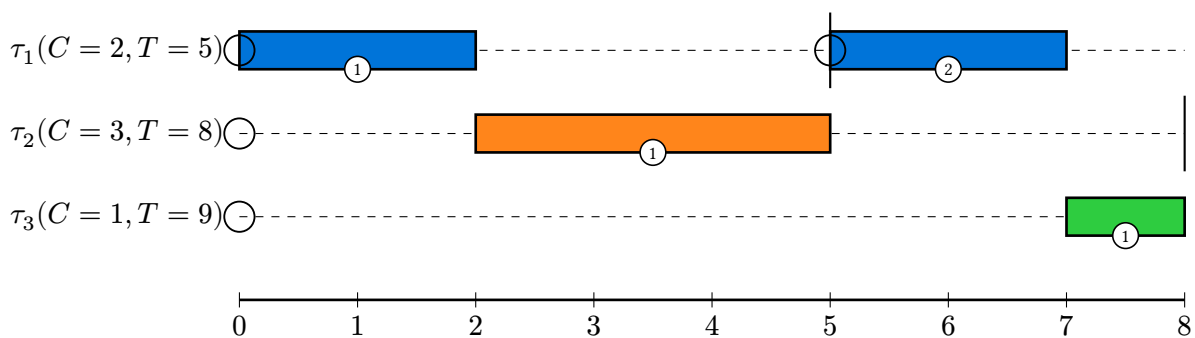
$$r_3^1 = 1 + \left\lceil \frac{1}{5} \right\rceil \cdot 2 + \left\lceil \frac{1}{8} \right\rceil \cdot 3 = 1 + 2 + 3 = 6$$

$$r_3^2 = 1 + \left\lceil \frac{6}{5} \right\rceil \cdot 2 + \left\lceil \frac{6}{8} \right\rceil \cdot 3 = 1 + 4 + 3 = 8$$

$$r_3^3 = 1 + \left\lceil \frac{8}{5} \right\rceil \cdot 2 + \left\lceil \frac{8}{6} \right\rceil \cdot 3 = 1 + 4 + 3 = 8$$

$$r_3^2 = r_3^3 = 8 \wedge r_3^3 = 8 \leq D_3 = 9$$

Le job 1 de r_3 est ordonnancable car il ne dépasse pas la deadline $D_3 = 9$. Sous RM, une tâche est ordonnancable si son temps de réponse pire cas, calculé par itération en tenant compte des tâches plus prioritaires, est inférieur ou égal à sa deadline.



1.7.2. Pourquoi on ne teste que le premier job ?

On teste uniquement le premier job car, dans un système synchrone à priorités fixes, il subit le maximum d'interférences des tâches plus prioritaires. Les jobs suivants arrivent dans un système déjà partiellement exécuté et ont donc un temps de réponse inférieur ou égal. $r_i^1 = \max_k r_i^k$

1.8. For the FTP class, synchronous systems, arbitrary deadline: provide and prove correct a feasibility interval.

1.8.1. Résumé

Pour un système **synchrone** à **échéances arbitraires** sous un ordonnanceur **FTP**, il suffit de vérifier qu'aucune deadline n'est manquée dans le **premier busy period**. Ce premier busy period est l'intervalle $[0, \lambda_n)$ où λ_n est le **premier instant idle strictement positif** du planning synchrone (toutes les tâches relâchées à $t = 0$).

1.8.2. Définitions (cœur de l'idée)

Soit un planning (work-conserving) FTP pour un ensemble de tâches périodiques synchrones.

1.8.2.1. Busy period élémentaire

Un **busy period élémentaire** est un intervalle $[a, b)$ tel que :

- a et b sont des **idle points** (processeur idle),
- et il n'y a **aucun** idle point dans (a, b) .

1.8.2.2. Busy period de niveau i

Un **busy period de niveau i** est un busy period élémentaire **en ne considérant que** $\{\tau_1, \cdot, \tau_i\}$ (les i tâches les plus prioritaires), et où τ_i exécute au moins un job.

On note λ_i le **premier idle point après 0** dans le planning synchrone du sous-ensemble $\{\tau_1, \cdot, \tau_i\}$. Alors $[0, \lambda_i)$ est le **premier** busy period de niveau i .

1.8.3. Théorème (feasibility interval)

Pour un système périodique **synchrone à deadlines arbitraires** (ordonnanceur **FTP**),

$$[0, \lambda_n)$$

est un **feasibility interval** : si aucune deadline n'est manquée parmi les jobs relâchés dans cet intervalle, alors aucune deadline ne sera manquée pour toute la durée de vie du système.

On veut montrer : “si ça passe jusqu'à λ_n , alors ça passera toujours”.

1.8.3.1. Le pire cas pour une tâche τ_i se produit dans le premier busy period de niveau i

Considère **n'importe quel** busy period de niveau i , disons $[a, b)$. On regarde les premiers relâchements après a : pour chaque τ_j (avec $j < i$), le premier job après a arrive à $a + \delta_j$ avec $\delta_j > 0$.

Idée clé : si on **réduit** un δ_j , on “ramène du travail plus tôt”, donc on **augmente** (ou ne diminue pas) la quantité de travail prioritaire dans la fenêtre, donc on **ne peut pas aider** à réduire les temps de réponse.

- Si $\delta_i > 0$: pendant $[a, a + \delta_i)$ seuls des jobs **plus prioritaires** que τ_i s'exécutent, donc avancer τ_i (diminuer δ_i) ne fait que **pousser** τ_i plus tôt $\rightarrow$ son pire temps de réponse est atteint quand $\delta_i = 0$.
- Si $\delta_j > 0$ pour un $j < i$: avancer τ_j augmente (ou conserve) l'interférence sur τ_i , donc **allonge** (ou conserve) le busy period et/ou les temps de réponse.

Donc le pire cas correspond à

$$\delta_1 = \delta_2 = \dots = \delta_i = 0$$

c'est-à-dire le cas **synchrone** démarrant à l'origine : les plus grands temps de réponse de τ_i apparaissent dans $[0, \lambda_i)$.

1.8.3.2. 2) Les bornes s'emboîtent : $\lambda_1 < \lambda_2 < \dots < \lambda_n$

Quand on ajoute des tâches (on passe de $\{\tau_1, \cdot, \tau_i\}$ à $\{\tau_1, \cdot, \tau_{i+1}\}$), on ajoute de la demande CPU : le premier instant idle ne peut qu'être repoussé vers la droite. Donc les premiers busy periods sont emboîtés et croissants.

1.8.3.3. 3) Conclusion : $[0, \lambda_n)$ suffit

Prends n'importe quel job d'une tâche τ_i .

- Son pire temps de réponse se réalise dans $[0, \lambda_i)$ (point 1),
- et $[0, \lambda_i) \subset [0, \lambda_n)$ (point 2).

Donc **si aucune deadline n'est manquée dans** $[0, \lambda_n)$, alors aucune tâche τ_i ne peut manquer une deadline (puisque tous les pires cas de toutes les tâches sont déjà “couverts”). Ainsi $[0, \lambda_n)$ est bien un feasibility interval.

1.8.4. Remarque pratique (comment “trouver” λ_n)

λ_n est le **premier idle point** après 0 dans le planning synchrone. Intuitivement, c’est la fin du premier busy period global : à cet instant, tout le travail relâché depuis 0 a été absorbé et le processeur redevient idle. (En exercice, on le détermine souvent en construisant le planning ou via une équation de charge de travail.)

1.9. For deterministic schedulers and discrete time space, prove that feasible schedules of periodic tasks are periodic.

1.9.1. Résumé

Dans un modèle à temps discret, un ordonnanceur déterministe prend ses décisions uniquement à partir de l’état courant du système. Pour un ensemble de tâches périodiques, le nombre d’états possibles est fini et le schéma de relâchement se répète à chaque hyperpériode. Il en résulte que tout schedule faisable devient nécessairement périodique.

1.9.2. Idée principale

- Les tâches sont périodiques : le motif des relâchements se répète tous les $P = \text{lcm}(T_i)$.
- Le temps est discret : on observe le système à des instants entiers.
- Un schedule faisable implique que la quantité de travail en attente reste bornée.
- Le nombre d’états possibles est donc fini.
- Un état finit par se répéter, et le déterminisme force la répétition du comportement futur.

1.9.3. Hypothèses

On considère un ensemble de tâches périodiques $\tau_i = (C_i, T_i, D_i)$ exécutées sur un processeur unique en temps discret.

L’ordonnanceur est déterministe : à un instant donné, la décision d’ordonnancement est une fonction déterministe de l’état courant.

On suppose l’existence d’un schedule faisable, c’est-à-dire qu’aucune échéance n’est manquée.

On note $P = \text{lcm}(T_1, T_2, \dots, T_n)$ l’hyperpériode du système.

1.9.4. Définition d’une configuration

À un instant entier t , on définit la configuration du système comme suit.

Pour chaque tâche τ_i :

- $a_{i(t)}$: temps restant avant le prochain relâchement, avec $a_{i(t)} \in \{0, 1, \dots, T_i - 1\}$,
- $e_{i(t)}$: temps d’exécution restant du job actif, avec $e_{i(t)} \in \{0, 1, \dots, C_i\}$.

La configuration globale est alors

$$\mathcal{C}(t) = ((a_1(t), e_1(t)), \dots, (a_n(t), e_n(t))).$$

1.9.5. Finitude de l’ensemble des configurations

Chaque variable $a_{i(t)}$ prend un nombre fini de valeurs (T_i possibilités), et chaque $e_{i(t)}$ prend également un nombre fini de valeurs ($C_i + 1$ possibilités).

Le nombre total de configurations possibles est donc borné par

$$\prod_{i=1}^n T_i(C_i + 1),$$

ce qui est fini.

1.9.6. Périodicité des relâchements

Par définition de l'hyperpériode P , pour toute tâche τ_i et tout instant t ,

$$a_{i(t+P)} = a_{i(t)}.$$

Ainsi, le schéma de relâchement des tâches est identique sur chaque intervalle de longueur P .

1.9.7. Répétition d'une configuration

Considérons la suite des configurations observées aux instants multiples de l'hyperpériode :

$$\mathcal{C}(0), \mathcal{C}(P), \mathcal{C}(2P), \dots$$

Cette suite prend ses valeurs dans un ensemble fini. Par le principe des tiroirs, il existe deux entiers $k < \ell$ tels que

$$\mathcal{C}(kP) = \mathcal{C}(\ell P).$$

1.9.8. Déterminisme et périodicité du schedule

Aux instants $t_0 = kP$ et $t_1 = \ell P$, le système est dans exactement le même état :

- mêmes temps avant relâchement,
- mêmes temps d'exécution restants.

Comme l'ordonnanceur est déterministe, il prendra les mêmes décisions à partir de t_0 et de t_1 . Par induction, l'exécution future est identique à partir de ces deux instants.

Le schedule est donc périodique à partir de t_0 , avec une période

$$T = t_1 - t_0 = (\ell - k)P.$$

1.9.9. Conclusion

Tout schedule faisable généré par un ordonnanceur déterministe en temps discret pour un ensemble de tâches périodiques est ultimement périodique. La périodicité découle directement de la finitude de l'espace d'états et du déterminisme de l'ordonnanceur.

1.10. For the FTP class, asynchronous systems, constrained deadline: prove correct the feasibility interval $[0, S_n +)$. Explain the meaning of the S_i instants.

1.10.1. Idée Général

Pour un système périodique asynchrone à deadlines contraintes (donc $D_i \leq T_i$) et un ordonnanceur FTP (priorités fixes), il suffit de vérifier qu'aucune échéance n'est manquée sur l'intervalle $[0, S_n + P)$, ou $P = \text{lcm}(T_1, \dots, T_n)$. L'idée clé est que, si le système est faisable, le planning devient périodique à partir d'un certain instant S_n , donc après S_n tout se répète tous les P .

1.10.2. Cadre et notations

On considère n tâches périodiques asynchrone $\tau_i = (O_i, C_i, D_i, T_i)$, avec des FTP. On suppose un ordre de priorité $\tau_1 \succ \tau_2 \succ \dots \succ \tau_n$. On Note :

$$P_i = \text{lcm}(T_1, \dots, T_i), \text{ et en particulier } P = P_n$$

On introduit les instants S_i qui marquent le début de la zone où le planning de $\tau_1, \dots, \tau_i$ est périodique.

1.10.3. Signification et définition des instants S_i

- **Intuition:**

- S_1 : premier instant où τ_1 peut “installer” son motif périodique. (Son premier release)
- S_2 : premier release de τ_2 à partir du moment où le motif des tâches plus prioritaires τ_1 est déjà en régime périodique.
- ...
- S_n pareil mais pour toutes les tâches ; à partir de S_n , tout le système est synchronisé avec un motif qui se répète.

- **Définition :**

- Base : $S_1 = O_1$
- Récurrence pour $i > 1$, S_i est le premier release de τ_i tel que $S_i \geq S_{i-1}$

En clair: S_i est le premier moment après S_{i-1} où τ_i apparaît donc à partir de là on peut comparer les deux fenêtres.

1.10.4. Étape clé : périodicité du planning à partir de S_i

- **Proposition $\mathcal{P}(i)$**

Si l'ensemble $\tau_1, \dots, \tau_i$ est faisable sous FTP, alors son planning est périodique à partir de S_i avec période $P_i = \text{lcm}(T_1, \dots, T_i)$.

- **Preuve (par induction)**

Base $i = 1$. Avec une seule tâche périodique : à partir de son premier release $S_1 = O_1$, le motif se répète tous les T_1 . Donc périodique de période $P_1 = T_1$.

Hérédité. Supposons $\mathcal{P}(i-1)$ vraie. Considérons $\tau_1, \dots, \tau_i$. Comme τ_i est la moins prioritaire, elle n'influence pas l'ordonnancement des tâches plus prioritaires $\tau_1, \dots, \tau_{i-1}$: leur motif reste le même.

- Par hypothèse d'induction, le planning de $\tau_1, \dots, \tau_{i-1}$ est périodique dès S_{i-1} , période P_{i-1} .
- La tâche τ_i est périodique à partir de son premier release S_i , période T_i .

Donc, à partir de S_i (choisi $\geq S_{i-1}$), on a superposition de deux comportements périodiques, et la période commune vaut : $\text{lcm}(P_{i-1}, T_i) = \text{lcm}(T_1, \dots, T_i) = P_i$.

Donc $\mathcal{P}(i)$ est vraie.

Conclusion : pour $i = n$, le planning global est périodique à partir de S_n avec période P .

1.10.5. Pourquoi $[0, S_n + P)$ suffit

1. **Avant S_n** : c'est le transitoire dû aux offsets ; il faut le vérifier.
2. **À partir de S_n** : le planning est périodique de période P . Donc ce qui se passe sur $[S_n, S_n + P)$ se répète sur $[S_n + kP, S_n + (k+1)P)$ pour tout $k \geq 0$.
3. Les releases et deadlines (avec $D_i \leq T_i$) se répètent aussi modulo P : un job relâché à t a un “jumeau” relâché à $t + P$ avec la même structure relative dans le cycle.

Donc :

- si aucune deadline n'est manquée sur $[0, S_n + P)$,
- alors aucune deadline ne sera manquée après, puisque tout après S_n est une copie d'un cycle déjà validé.

Ainsi, $[0, S_n + P)$ est un **intervalle de faisabilité** pour les systèmes asynchrones à deadlines contraintes sous FTP.

1.10.6. Conclusion

- $S_1 = O_1$ et $S_i =$ premier release de τ_i à $t \geq S_{i-1}$.
- Si faisable, le planning de $\tau_1, \dots, \tau_i$ est périodique à partir de S_i avec période $P_i = \text{lcm}(T_1, \dots, T_i)$ (induction + τ_i n'affecte pas les plus prioritaires).
- Donc le planning global est périodique à partir de S_n avec période P , et il suffit de vérifier une période complète après S_n plus le transitoire : intervalle $[0, S_n + P)$.

1.11. For the FTP class ($\tau_1 \succ \tau_2 \succ \tau_3$), asynchronous systems, constrained deadline: compute the S_i values for the following system (O, C, T) : $\tau_1 : (10, 2, 10), \tau_2 : (0, 1, 5), \tau_3 : (1, 1, 3)$ and then check the schedulability.

1.11.1. Résumé de la réponse

On calcule les instants S_i avec la définition de la Théorème 45, puis on teste l'ordonnabilité uniquement sur l'intervalle de faisabilité $[0, S_n + P)$ (Corollaire 46). Ici, on obtient $S_3 = 10$ et $P = 30$, donc il suffit de vérifier $[0, 40)$. Le système **n'est pas ordonnable** avec $\tau_1 \succ \tau_2 \succ \tau_3$ car un job de τ_3 (relâché à $t = 10$) finit après son échéance (à $t = 14$ au lieu de $t = 13$).

1.11.2. Données du système

Priorités FTP (données) : $\tau_1 \succ \tau_2 \succ \tau_3$.

Tâches (on suppose **deadline implicite** ici, donc $D_i = T_i$, ce qui est un cas particulier de "deadline contrainte") :

$$\tau_1 : (O_1 = 10, C_1 = 2, T_1 = 10, D_1 = 10)$$

$$\tau_2 : (O_2 = 0, C_2 = 1, T_2 = 5, D_2 = 5)$$

$$\tau_3 : (O_3 = 1, C_3 = 1, T_3 = 3, D_3 = 3)$$

1.11.3. Calcul des S_i

D'après le Théorème 45 :

Soit $x^+ = \max(x, 0)$ et pour $i > 1$:

$$S_1 := O_1$$

$$S_i := O_i + \left\lceil \frac{(S_{i-1} - O_i)^+}{T_i} \right\rceil * T_i$$

$S_1 :$

$$S_1 = O_1 = 10$$

$S_2 :$

$$O_2 = 0, T_2 = 5,$$

$$(S_1 - O_2)^+ = (10 - 0)^+ = 10$$

$$\left\lceil \frac{10}{5} \right\rceil = \lceil 2 \rceil = 2$$

$$S_2 = 0 + 2 * 5 = 10$$

$S_3 :$

$$O_3 = 1, T_3 = 3,$$

$$(S_2 - O_3)^+ = (10 - 1)^+ = 9$$

$$\left\lceil \frac{9}{3} \right\rceil = \lceil 3 \rceil = 3$$

$$S_3 = 1 + 3 * 3 = 10$$

- Résultat

$$S_n = S_3 = 10$$

1.11.4. 2) Intervalle de faisabilité $[0, S_n + P)$

On calcule l'hyperpériode :

$$P = \text{lcm}(T_1, T_2, T_3) = \text{lcm}(10, 5, 3)$$

$$\text{lcm}(10, 5) = 10$$

$$\text{lcm}(10, 3) = 30$$

$$P = 30$$

.

Par le Corollaire 46 : $[0, S_n + P)$ est un intervalle de faisabilité. Donc ici :

$$[0, S_3 + P) = [0, 10 + 30) = [0, 40)$$

Il suffit donc de vérifier qu'aucune deadline n'est manquée par les jobs relâchés dans $[0, 40)$.

1.11.5. Test d'ordonnançabilité par construction du schedule (FTP préemptif)

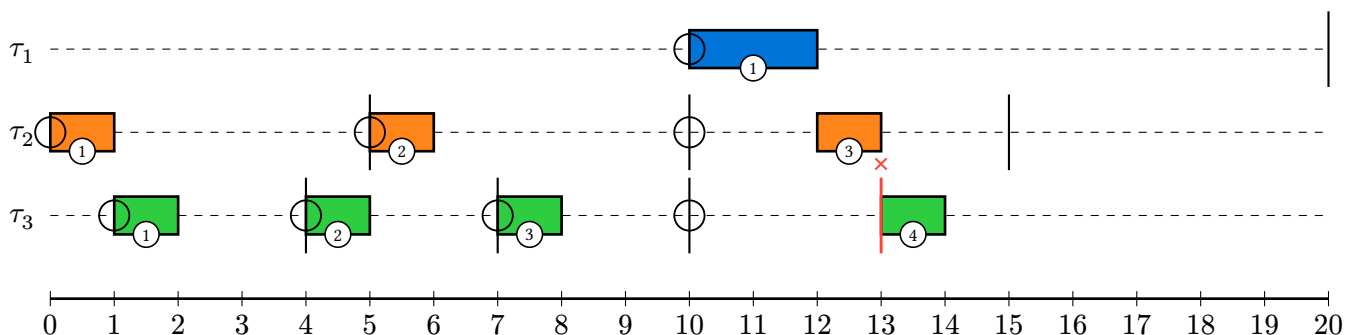
• Rappels :

- Releases de $\tau_i : r_{\{i,k\}} = O_i + kT_i$.
- Deadlines (implicites) : $d_{\{i,k\}} = r_{\{i,k\}} + T_i$.
- Exécution : à tout instant on exécute le job prêt de plus haute priorité.

$\tau_1 : (O_1 = 10, C_1 = 2, T_1 = 10, D_1 = 10)$ relâché à : 0, 5, 10, 15, ...

$\tau_2 : (O_2 = 0, C_2 = 1, T_2 = 5, D_2 = 5)$ relâché à : 1, 4, 7, 10, 13, ...

$\tau_3 : (O_3 = 1, C_3 = 1, T_3 = 3, D_3 = 3)$ relâché à : 10, 20, 30, ...



1.11.5.1. Vérification de la deadline critique

Le job de τ_3 relâché à $t = 10$ a pour deadline : $d = 10 + 3 = 13$.

Or il finit à $t = 14$.

Donc : **deadline manquée** → système **non ordonnançable** (et on peut s'arrêter, même sans dérouler jusqu'à 40).

1.11.6. Conclusion

- $S_1 = 10, S_2 = 10, S_3 = 10$
- $P = 30$, donc intervalle de test : $[0, 40)$
- En construisant le schedule FTP avec $\tau_1 \succ \tau_2 \succ \tau_3$, le job de τ_3 relâché à $t = 10$ finit à $t = 14$ alors que sa deadline est $t = 13$.

=> **Le système n'est pas schedulable** sous cet ordre de priorités.

1.12. For the FTP class, asynchronous systems, constrained deadline: provide and prove correct the main property foundation of the optimal Audsley's priority assignment.

1.12.1. Résumé rapide

L'algorithme d'Audsley est optimal car, s'il existe au moins une affectation de priorités fixes qui respecte toutes les deadlines, alors il est toujours possible de choisir une tâche qui peut être mise en dernière priorité sans casser la faisabilité. En répétant ce raisonnement, Audsley reconstruit une affectation faisable complète.

1.12.1.1. Idée Centrale :

Si un système de tâches est faisable avec des priorités fixes, alors la tâche la moins prioritaire de cette solution est "inoffensive", la mettre en dernière priorité ne gêne personne, et elle arrive quand même à respecter ses deadlines. C'est exactement cette tâche que l'algorithme d'Audsley cherche à chaque étape.

1.12.1.2. Explication

On raisonne sur un système FTP préemptif, mono-processeur, work-conserving.

Que fait une tâche de plus basse priorité ?

Une tâche de plus basse priorité :

- ne s'exécute que quand toutes les autres sont bloquées ou terminées,
- utilise uniquement les temps morts laissés par les tâches plus prioritaires.

Donc son comportement dépend seulement de :

- combien de temps les autres tâches occupent le processeur,
- et pas de l'ordre exact entre ces tâches plus prioritaires.

Conséquence importante

Si on fixe une tâche τ_k comme la moins prioritaire :

- peu importe l'ordre des autres tâches au-dessus,
- τ_k voit toujours exactement les mêmes créneaux libres pour s'exécuter.

Donc :

- soit τ_k arrive à finir avant ses deadlines,
- soit elle n'y arrive pas, et ce résultat ne dépend pas du classement interne des tâches plus prioritaires.

Pourquoi il existe toujours une "bonne" tâche à mettre en dernier

Supposons qu'il existe une affectation de priorités faisable. Dans cette affectation :

- regardons la tâche qui a la priorité la plus basse,
- elle respecte forcément ses deadlines (puisque la solution est faisable).

Donc cette tâche est compatible avec la dernière priorité : elle peut être placée en dernier sans rendre le système infaisable.

C'est cette tâche que l'algorithme d'Audsley est garanti de pouvoir trouver.

Pourquoi la fixer en dernier ne casse rien

Quand on place une tâche en dernière priorité :

- elle ne peut jamais ralentir les tâches au-dessus,
- les tâches plus prioritaires gardent exactement le même comportement.

Donc :

- si les autres tâches étaient faisables avant,
- elles le restent après.

1.12.2. Conclusion

L'algorithme procède ainsi :

1. Chercher une tâche qui fonctionne même en dernière priorité.
2. La fixer définitivement comme tâche la moins prioritaire.
3. Répéter le raisonnement sur les tâches restantes.

À chaque étape :

- une telle tâche existe si une solution existe,
- la fixer en dernier ne détruit pas la faisabilité.

L'algorithme d'Audsley est optimal car, dans tout système FTP faisable, il existe toujours une tâche qui peut être placée en dernière priorité sans violer les deadlines ; en la sélectionnant itérativement, l'algorithme reconstruit nécessairement une affectation de priorités faisable complète.

1.13. Consider the following task set: ($O_1 = 3, C_1 = 2, D_1 = 2, T_1 = 4; O_2 = 0, C_2 = 3, D_2 = 4, T_2 = 8$) In the framework of the FTP class (and asynchronous systems, constrained deadline), determine if τ_1 is lowest-priority viable.

1.13.1. Résumé rapide

On teste si τ_1 peut être mise en plus basse priorité (donc τ_2 au-dessus) tout en respectant toutes ses deadlines. Ici, un job de τ_1 relâché à $t = 7$ se fait préempter par τ_2 et rate sa deadline à $t = 9$. Donc τ_1 n'est pas lowest-priority viable.

1.13.2. Rappel Deadlines

- **Relative** : D_i
- **Absolute** : $d_{i,k} = r_{i,k} + D_i$
 - Ou $r_{i,k}$ est le temps auquel est release le job
 - La deadline absolue est la deadline réelle dans le temps pour un job donné.
- **Implicite** : $D_i = T_i$
 - Deadline implicite $\Leftrightarrow$ la deadline relative est égale à la période.
- **Contrainte** : $D_i \leq T_i$
 - Deadline contrainte $\Leftrightarrow$ la deadline est plus petite que la période.
- **Fenetre de faisabilité**: $[r_{i,k}, d_{i,k}]$
- **Condition de respect de deadline** : $f_{i,k} \leq d_{i,k}$
- **Audsley, test lowest-priority feasible** :

$$\forall k, \sum_{\tau_j \neq \tau_i} \left\lceil \frac{D_i + O_i - O_j}{T_j} \right\rceil C_j + C_i \leq D_i$$

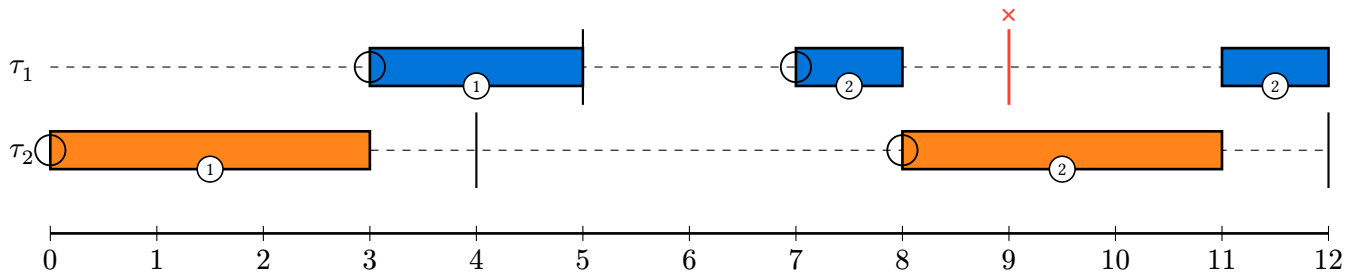
Cette inégalité sert à tester si τ_i peut être en plus basse priorité.

1.13.3. Résolution

$$\tau_1 = (O_1 = 3, C_1 = 2, D_1 = 2, T_1 = 4)$$

$$\tau_2 = (O_2 = 0, C_2 = 3, D_2 = 4, T_2 = 8)$$

$$\tau_2 \succ \tau_1$$



On regarde le job “dangereux” de τ_1 à $t = 7$:

- A $t = 7$, le job $J_{1,2}$ de τ_1 est relâché avec une deadline absolue de $7 + 2 = 9$
- Entre 7 et 8, τ_1 ne peut exécuter **au plus 1** unité car p^réempté par τ_2 à $t = 8$
- τ_1 ne peut reprendre son job qu'à $t = 11$ alors que la deadline était $t = 9$

Quand la Deadline est différente de la période, alors

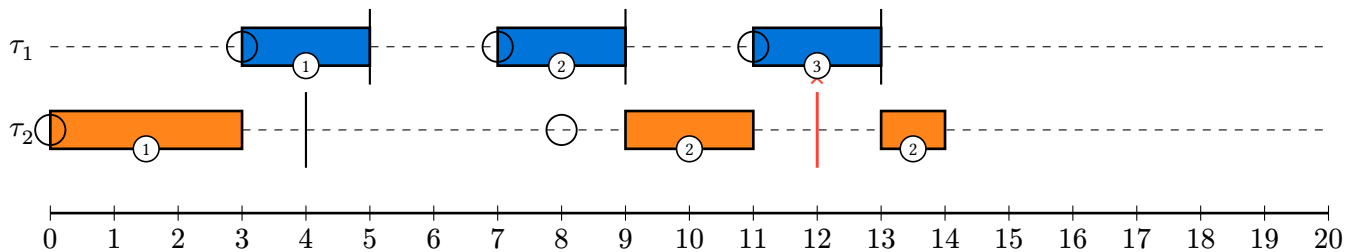
τ_1 n'est pas lowest-priority viable (avec $\tau_2 \succ \tau_1$)

1.13.4. Inversion des priorités

$$\tau_1 = (O_1 = 3, C_1 = 2, D_1 = 2, T_1 = 4)$$

$$\tau_2 = (O_2 = 0, C_2 = 3, D_2 = 4, T_2 = 8)$$

$$\tau_1 \succ \tau_2$$



1.13.5. Audsley

Pour tester lowest-priority on utilise la formule d'Audsley interactivement

$$w_i^{k+1} = C_i + \sum_{j \neq i} C_j \left\lceil \frac{w_i^k + (O_i - O_j)}{T_j} \right\rceil$$

Lowest priority Test

On accepte si $R_i \leq D_i$

$$\tau_2 \succ \tau_1 :$$

$$w_1^0 = C_1 = 2$$

Itération 1 :

$$w_1^1 = 2 + 3 \left\lceil \frac{2 + (O_1 - O_2)}{T_2} \right\rceil = 2 + 3 \left\lceil \frac{2 + (3 - 0)}{8} \right\rceil = 5$$

Itération 2 :

$$w_1^2 = 2 + 3 \left\lceil \frac{5 + 3}{8} \right\rceil$$

$$= 2 + 3 \lceil 1 \rceil = 5$$

$$R_1 = 5 > D_1 = 2$$

τ_1 n'est pas lowest priority feasible

$$\tau_1 \succ \tau_2 :$$

$$w_2^0 = C_2 = 3$$

Itération 1 :

$$w_2^1 = 3 + 2 \left\lceil \frac{3 + (O_1 - O_2)}{T_1} \right\rceil = 3 + 2 \left\lceil \frac{3 + (0 - 3)}{4} \right\rceil = 3$$

$$R_2 = 3 \leq D_2 = 4$$

τ_2 est lowest priority feasible

Ca test ne prend pas en compte les interférences.

1.14. For a synchronous implicit-deadline task system, demonstrate the strong optimality of EDF by constructing a counterexample where RM fails but EDF succeeds.

1.14.1. Résumé de la réponse

On construit un système périodique synchrone à deadlines implicites (donc $D_i = T_i$) tel que $U < 1$.

Alors EDF est faisable (car pour deadlines implicites, $U \leq 1$ est suffisant), mais RM rate une échéance : cela prouve la forte optimalité de EDF (EDF ordonnance tout ce que RM peut ordonnancer, et parfois plus).

1.14.2. Rappel formule

- **Utilisation Processeur** : $U = \sum_{i=1}^n \frac{C_i}{T_i}$
- **Théorème fondamental (Liu & Layland)** : EDF faisable $\iff U \leq 1$
 - Pour un système périodique préemptif, synchrone, à deadlines implicites
 - Condition nécessaire et suffisante
 - si $U \leq 1$, EDF trouve toujours un ordonnancement valide
 - si $U > 1$, aucun ordonnanceur ne peut réussir
- **Règle de priorité EDF** : $d_{i,k} = r_{i,k} + D_i$

1.14.3. Contre-exemple (synchrone, deadlines implicites)

On considère 3 tâches périodiques, toutes relâchées à $t = 0$:

$$\tau_1 = (C_1 = 1, T_1 = 2, D_1 = 2)$$

$$\tau_2 = (C_2 = 1, T_2 = 5, D_2 = 5)$$

$$\tau_3 = (C_3 = 2, T_3 = 7, D_3 = 7)$$

Utilisation totale :

$$U = \frac{1}{2} + \frac{1}{5} + \frac{2}{7} = 0.5 + 0.2 + 0.2857... = 0.9857... < 1$$

Donc le système est EDF-faisable (deadlines implicites).

1.14.4. Execution sous RM et EDF

Sous RM, la priorité est inversement proportionnelle à la période :

$$\tau_1 \succ \tau_2 \succ \tau_3$$

$$(2 < 5 < 7).$$

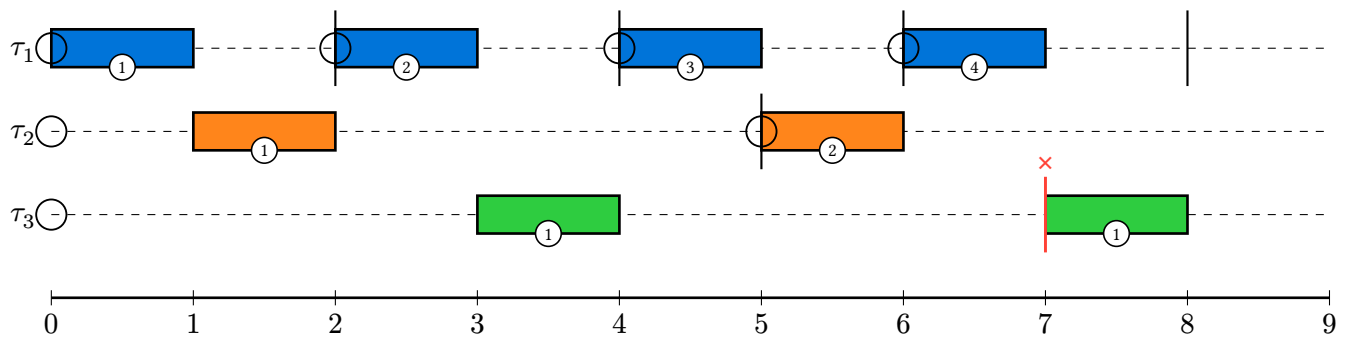
Dans le cas de EDF le job qui a la deadline absolue la plus proche va etre la plus proche.

$$\tau_1 = (C_1 = 1, T_1 = 2, D_1 = 2)$$

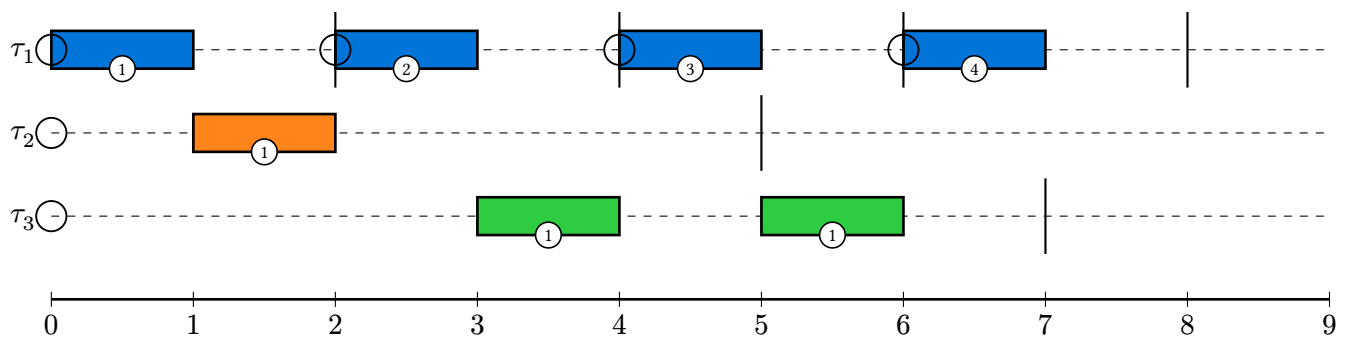
$$\tau_2 = (C_2 = 1, T_2 = 5, D_2 = 5)$$

$$\tau_3 = (C_3 = 2, T_3 = 7, D_3 = 7)$$

$$\text{RM} : \tau_1 \succ \tau_2 \succ \tau_3$$



Ordonnancement avec RM



Ordonnancement avec EDF

Comme $U < 1$ et que les deadlines sont implicites, EDF peut maintenir la faisabilité sur tout l'hyperperiode.

1.14.5. Conclusion

Pour les tâches périodiques à deadlines implicites :

$$S_{\text{RM}} \subseteq S_{\text{EDF}}$$

- S_{RM} : systeme ordonnacables par RM
- S_{EDF} : systeme ordonnacables par EDF

Tout systeme RM-faisable verifie forcément $U \leq 1$ or $U \leq 1 \Rightarrow \text{EDF-Faisable}$

$$\text{RM-Faisable} \Rightarrow \text{EDF-Faisable}$$

Mais avec le contre exemple on a que $U < 1$ et RM rate une deadline mais EDF faisable. Il existe donc :

$$\text{EDF faisable} \wedge \text{RM non faisable}$$

On trouve alors que RM est inclu dans EDF

$$S_{\text{RM}} \subset S_{\text{EDF}}$$

Pour les systèmes périodiques préemptifs à deadlines implicites, EDF est fortement optimal par rapport à RM : tout système ordonnançable par RM l'est aussi par EDF, et il existe des systèmes ordonnançables par EDF qui ne le sont pas par RM.

1.15. Prove the optimality of EDF.

1.15.1. Résumé

EDF (Earliest Deadline First) est **optimal** sur uniprocasseur préemptif : si un ensemble de jobs est faisable (il existe **au moins un** ordonnancement qui respecte toutes les échéances), alors EDF produit aussi un ordonnancement faisable.

L'idée clé est un **argument d'échange** (swap) répété : à chaque instant, on peut transformer un horaire faisable quelconque pour qu'il "imite" EDF de plus en plus longtemps, sans casser la faisabilité.

1.15.2. Rappel

- **Deadline absolue** du k -ième job de la tâche τ_i : $d_{\{i,k\}} = O_i + (k-1)T_i + D_i$.
- **Relâchement** : $a_{\{i,k\}} = O_i + (k-1)T_i$.
- **Exécution requise** : $e_{\{i,k\}} = C_i$.
- **Temps de réponse d'un job** : $R_{\{i,k\}} = f_{\{i,k\}} - a_{\{i,k\}}$ (fin moins relâchement).
- **À chaque instant t** : $\text{edf}(t) = \arg_j \text{ active à } t \ d_j$ (plus petite deadline absolue $\rightarrow$ plus haute priorité).
- **Utilisation d'une tâche** : $U(\tau_i) = \frac{C_i}{T_i}$.
- **Utilisation totale** : $U(\tau) = \sum_i \frac{C_i}{T_i}$.

1.15.3. Cadre et notations (jobs)

On raisonne au **niveau job** (ce qui rend le résultat valable au-delà du simple cas périodique).

Soit un ensemble fini de jobs J . Chaque job j est caractérisé par :

- a_j : date de relâchement (arrival/release),
- e_j : temps d'exécution (WCET / processing time),
- d_j : **deadline absolue**.

Un ordonnancement est **faisable** si chaque job reçoit exactement e_j unités CPU dans $[a_j, d_j)$ (et donc termine avant d_j).

Définition EDF : à tout instant t , EDF exécute un job actif ayant la plus petite deadline absolue.

Notons $\text{edf}(t) = \arg \min_j \text{ active à } t \ d_j$.

Théorème (Optimalité EDF). Si l'ensemble de jobs J est faisable, alors l'ordonnancement EDF de J est faisable.

1.15.4. Preuve (par induction + échanges)

On suppose qu'il existe un ordonnancement faisable (quelconque) de J ; appelons-le $\sigma(0)$.

On construit par induction une suite d'ordonnancements faisables $\sigma(i)$ tels que :

- $\sigma(i)$ est faisable,
- et pour tout $t \in (0, 1, \dots, i-1)$, on a $\sigma(i)(t) = \text{edf}(t)$

(i.e., $\sigma(i)$ "copie" EDF sur les i premiers instants).

• Base

Pour $i = 0$, la propriété est vraie : $\sigma(0)$ existe et est faisable par hypothèse.

• Induction

Supposons la propriété vraie à l'étape i et montrons-la à $i + 1$.

- Si $\sigma(i)(i) = \text{edf}(i)$, alors on prend simplement $\sigma(i + 1) = \sigma(i)$: c'est encore faisable et ça copie EDF jusqu'à i .
- Sinon, à l'instant i , $\sigma(i)$ exécute un job j_1 tandis que EDF veut exécuter un autre job j_2 :
 - $j_1 = \sigma(i)(i)$ avec deadline d_1 ,
 - $j_2 = \text{edf}(i)$ avec deadline d_2 ,
 - par définition de EDF : $d_2 \leq d_1$.

Comme $\sigma(i)$ est faisable, le job j_2 est bien exécuté suffisamment avant sa deadline : il existe au moins un créneau dans l'intervalle $[i, d_2)$ où $\sigma(i)$ exécute j_2 (sinon j_2 raterait d_2).

Or $d_2 \leq d_1$, donc ce créneau est aussi strictement avant d_1 . On définit alors $\sigma(i + 1)$ en **échangeant** (swap) :

- à l'instant i , on exécute j_2 (comme EDF),
- et on déplace une unité d'exécution de j_2 (prise plus tard avant d_2) vers j_1 à cet instant-là.

Conséquences :

- j_2 reste faisable : il "perd" une unité plus tard mais la récupère à l'instant i (avant d_2).
- j_1 reste faisable : l'unité qu'on lui retire à l'instant i est remplacée à un instant $\leq d_2 \leq d_1$, donc toujours avant sa propre deadline.

Donc $\sigma(i + 1)$ est faisable et coïncide avec EDF à l'instant i en plus des instants précédents.

1.15.5. Conclusion

Par induction, on obtient un ordonnancement faisable qui coïncide avec EDF sur tout l'horizon considéré ; donc EDF est faisable dès qu'un ordonnancement faisable existe : EDF est optimal.

1.15.6. Intuition à retenir pour l'oral

EDF "met d'abord en sécurité" les jobs les plus urgents (plus petite deadline). Si un horaire faisable n'exécute pas le job le plus urgent à un instant donné, on peut permuter localement l'ordre (swap) sans faire rater de deadline, car le job urgent a une deadline $\leq$ à celle du job exécuté à sa place.

1.15.7. Autre preuve courte

Soit un ensemble de jobs supposé faisable et ordonnancé par EDF.

Supposons par l'absurde que EDF rate une deadline, et soit d la première deadline ratée.

Soit $[t_0, d]$ le dernier intervalle continuellement occupé (busy interval) avant d . Dans cet intervalle, EDF n'exécute que des jobs dont la deadline est $\leq d$.

Notons $W(d) = \sum_{j: d_j \leq d} e_j$ la charge totale de ces jobs.

Comme EDF rate la deadline d , on a nécessairement $W(d) > d - t_0$. Or tout ordonnanceur doit exécuter cette même charge avant d . La capacité maximale du processeur sur $[t_0, d]$ est $d - t_0$. Donc aucun ordonnanceur ne peut être faisable.

Contradiction avec l'hypothèse initiale : EDF est optimal.

1.16. For EDF, implicit deadline systems: prove the fact that $U \leq 1$ is necessary and sufficient.

1.16.1. Résumé

Pour des tâches périodiques (ou sporadiques) indépendantes, préemptives, sur **un seul processeur**, avec **deadlines implicites** ($D_i = T_i$), l'algorithme **EDF** est faisable **ssi** l'utilisation totale

$$U = \sum_i \frac{C_i}{T_i} \text{ vérifie } U \leq 1$$

.

1.16.2. Rappel – Formules utilisées

- Utilisation : $U = \sum_i \frac{C_i}{T_i}$.
- Capacité CPU sur une fenêtre $I : |I|$
- Majorant : $W(I) \leq \sum_i \left(\frac{|I|}{T_i}\right) C_i = |I| * U$
- Inégalité finale : $U \leq 1 \Rightarrow |I| * U \leq |I|$

1.16.3. Modèle et notations

Chaque tâche τ_i est définie par :

- C_i : pire temps d'exécution (WCET)
- T_i : période (et aussi deadline, car $D_i = T_i$)
- Jobs préemptifs, pas de blocage (tâches indépendantes), uniprocésseur.

Définition : $U = \sum_{i=1}^n \left(\frac{C_i}{T_i}\right)$.

1.16.4. Nécessité : si le système est faisable alors $U \leq 1$

Intuition. À long terme, la charge moyenne ne peut pas dépasser 100% du processeur.

Preuve courte. Considérons une fenêtre de temps de longueur L très grande (par ex. $L = k * \text{lcm}(T_1, \dots, T_n)$, avec k grand). Sur une durée L , la tâche τ_i libère environ $\frac{L}{T_i}$ jobs, donc elle demande environ $\left(\frac{L}{T_i}\right) * C_i$ unités de CPU.

La demande totale est donc environ : $\sum_i \left(\frac{L}{T_i}\right) * C_i = L * \sum_i \left(\frac{C_i}{T_i}\right) = L * U$.

Comme le processeur ne peut exécuter au maximum que L unités de travail sur $[0, L]$, il faut $L * U \leq L$, donc $U \leq 1$. Donc **$U \leq 1$ est nécessaire.**

1.16.5. Suffisance (preuve courte "examen", 10 lignes)

On suppose $U = \sum_i \frac{C_i}{T_i} \leq 1$ et $D_i = T_i$.

Par l'absurde, supposons qu'EDF manque une deadline ; soit t^* le **premier** instant de manque.

Considérons l'intervalle $I = [t_0, t^*]$ où t_0 est le dernier instant avant t^* tel que, à t_0 , il n'existe **aucun** job avec deadline $\leq t^*$ en attente (début d'un **busy interval**).

Alors sur I , le processeur est occupé en continu à exécuter uniquement des jobs de deadline $\leq t^*$, et comme la deadline à t^* est manquée, leur travail total vérifie $W(I) > |I|$.

Or, avec $D_i = T_i$, dans toute fenêtre de longueur $|I|$ la tâche τ_i peut contribuer au plus $|I|/T_i$ jobs dont la deadline tombe dans la fenêtre, donc un travail $\leq \left(\frac{|I|}{T_i}\right) C_i$.

Ainsi $W(I) \leq \sum_i \left(\frac{|I|}{T_i}\right) C_i = |I| * \sum_i \left(\frac{C_i}{T_i}\right) = |I| * U \leq |I|$, contradiction.

Donc EDF ne manque aucune deadline : $U \leq 1$ est suffisant.

1.16.6. Conclusion

Donc, pour EDF et deadlines implicites :

$$(U \leq 1 \text{ est nécessaire et suffisant.})$$

1.17. For EDF, synchronous systems, constrained deadline: provide the feasibility interval and the procedure to determine its upper bound.

1.17.1. Résumé rapide

Pour EDF avec des tâches périodiques, synchrones, à deadlines contraintes ($D_i \leq T_i$), on peut se limiter à un intervalle fini : le premier busy period $[0, L)$, où L est le premier instant d'inactivité après 0. L'énoncé important est : avant une première deadline ratée, EDF ne produit pas d'unités idle (en synchrone, deadlines contraintes).

1.17.2. Idée simple

• Pourquoi un interval de faisabilité :

Parce que sur un système périodique synchrone, si rien ne rate pendant le premier "bloc" continu d'activité (busy period), alors le comportement "pire cas" a déjà été observé : on ne va pas découvrir une première deadline ratée "après" un retour à l'inactivité.

• Interval :

$$\text{Feasibility interval pour EDF (synchrone, } D_i \leq T_i) = [0, L)$$

où L est le premier idle après l'origine (fin du premier busy period)

• Majorant et itération

L est le plus petit point fixe positif de l'équation de charge :

$$L = \sum_{i=1}^n \left\lceil \frac{L}{T_i} \right\rceil \cdot C_i$$

Que l'on calcule par itération de point fixe :

$$\begin{cases} w_0 = \sum_{i=1}^n C_i \\ w_{k+1} = W(w_k) = \sum_{i=1}^n \left\lceil \frac{w_k}{T_i} \right\rceil C_i \end{cases}$$

- Si $w_{k+1} = w_k$, alors $L = w_k$
- Si la suite croît sans se stabiliser, L n'est pas borné

• Test de faisabilité $[0, L)$

On simule EDF sur $[0, L)$ (ou on applique un test équivalent), et si aucune deadline n'est ratée pour les jobs relâchés dans cet intervalle, on conclut faisabilité sur toute la vie du système (c'est exactement le rôle d'un "feasibility interval").

1.17.3. Conclusion

- Le résultat clé utilisé pour borner l'intervalle est : en synchrone et deadlines contraintes, EDF ne met pas le CPU idle avant qu'une deadline soit ratée. Donc la première fois que le CPU redevient idle (L) marque la fin du "pire bloc continu" à vérifier.
- Le calcul de L via $L = W(L)$ est exactement la recherche du premier idle point (fin du premier busy period).

1.18. LLF: define the notion of laxity, the priority management, and the thrashing situation using an example.

1.18.1. Résumé

LLF (Least Laxity First) est un ordonnanceur à priorité dynamique : à chaque instant, il exécute le job qui a la plus petite laxité (le moins de “marge” avant rater sa deadline). La laxité mesure combien de temps le job peut encore “attendre” tout en restant faisable. LLF est optimal (au sens : s’il existe un planning faisable, LLF en trouve un), mais il peut provoquer beaucoup de préemptions (thrashing).

1.18.2. Rappel

Pour un job $J = (a, e, d)$:

- **CPU déjà exécuté** : $\varepsilon_J(t)$
- **Reste à exécuter** : $e_{\text{rest}}(t) = e - \varepsilon_J(t)$
- **Laxité** : $l_J(t) = d - t - (e - \varepsilon_J(t)) = (d - t) - e_{\text{rest}}(t)$
 - Pour un job périodique τ_i relâché à $r_{i,k}$
- **Deadline absolue** : $d_{i,k} = r_{i,k} + D_i$
- **LLF, choix de priorité** : exécuter le job J qui minimise $l_J(t)$ (tie-break déterministe).

1.18.3. Laxité

On considère un job J caractérisé par (a, e, d) :

- a : release
- e : temps CPU total requis (WCET)
- d : deadline absolue du job

Soit $\varepsilon_J(t)$ le CPU déjà consommé par J depuis son relâche à l’instant t . Le reste à exécuter vaut

$$e_{\text{rest}}(t) = e - \varepsilon_J(t)$$

La **laxité** de J à l’instant t est :

$$l_J(t) = d - t - (e - \varepsilon_J(t))$$

Interprétation :

- $l_J(t) > 0$: le job peut encore être retardé, au plus $l_J(t)$ unités
- $l_J(t) = 0$: il faut exécuter le job immédiatement sans interruption sinon on rate la deadline
- $l_J(t) < 0$: deadline inévitablement ratée

1.18.4. Priority management

LLF = priorité dynamique au niveau job

1. A chaque instant t (souvent à chaque tick en temps discret ou chaque event), on calcule $l_J(t)$ pour tous les jobs actifs.
2. On choisit le job avec laxité minimale
3. Préemption, si un autre job est plus urgent on le préempte immédiatement
4. Les égalités sont départagées de manière déterministe (règle fixe)

1.18.5. Thrashing LLF

Le thrashing = énormément de préemptions/commutations pour très peu de progrès utile.

Pire cas exemple

- A l’instant t , deux jobs actifs J_1, J_2 ont la même laxité
- Supposons qu’on exécute J_1 sur $[t, t + 1)$

- Comme J_1 a exécuté 1 unité, son “reste a faire” diminue de 1 et le temps avance de 1. Sa Laxité reste inchangée.

$$l_{J_1}(t+1) = l_{J_1}(t)$$

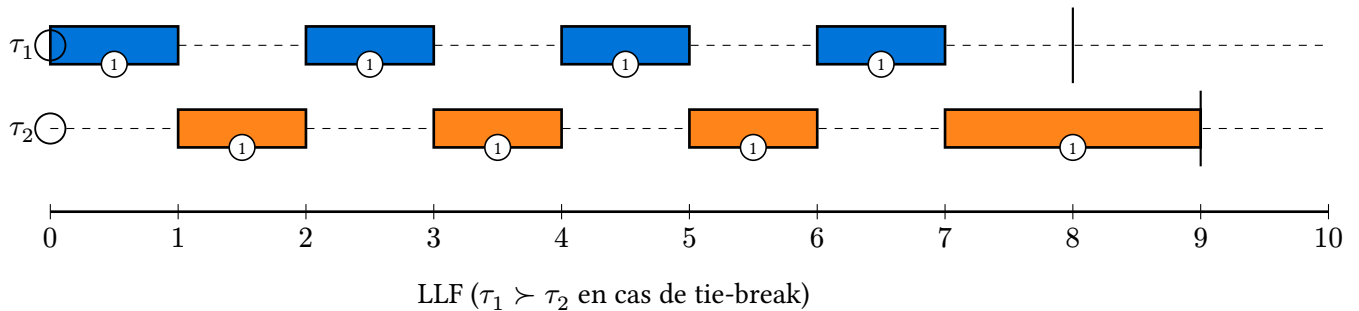
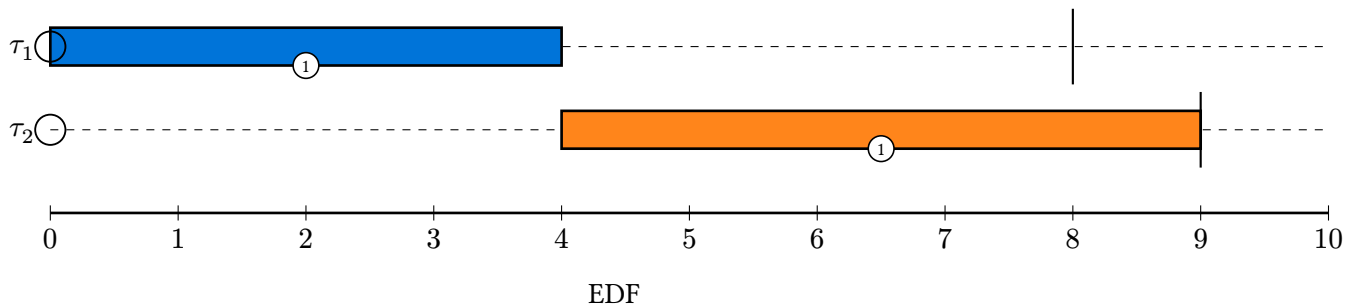
- J_2 n’a pas été exécuté mais le temps avance et donc sa laxité aussi.

$$l_{J_2}(t+1) = l_{J_2}(t) - 1$$

- Donc a $t + 1$, J_2 devient strictement plus urgent $\Rightarrow$ LLF préempte J_2
- Le scénario se répète $\Rightarrow$ alternance quasi à chaque tick

$$\tau_1 = (C_1 = 4, D_1 = 8, T_1 = 10)$$

$$\tau_2 = (C_2 = 5, D_2 = 9, T_2 = 10)$$



Explication LLF

Calcul des laxité en $t = 0$

- $\tau_1 \Rightarrow l_1(0) = 8 - 0 - 4 = 4$
- $\tau_2 \Rightarrow l_2(0) = 9 - 0 - 5 = 4$
- Tie-Break donc on choisit la tâche en priorité

Calcul des laxité en $t = 1$

- $\tau_1 \Rightarrow l_1(1) = 8 - 1 - 3 = 4$
- $\tau_2 \Rightarrow l_2(1) = 9 - 1 - 5 = 3$
- τ_2 est plus urgent $\rightarrow$ **préemption**

1.19. Build an LLF schedule for the following system (C, D, T) :

$$(2, 6, 4)(4, 5, 6).$$

1.19.1. Rappel

Pour un job $J = (a, e, d)$:

- CPU déjà exécuté : $\varepsilon_j(t)$
- Reste a exécuté : $\varepsilon_{\text{rest}}(t) = e - \varepsilon_j(t)$
- Laxité : $l_j(t) = d - t - (e - \varepsilon_j(t)) = (d - t) - \varepsilon_{\text{rest}}(t)$

- Pour un job périodique τ_i relâché à $r_{i,k}$
- **Deadline absolue** : $d_{i,k} = r_{i,k} + D_i$
- **LLF, choix de priorité** : exécuter le job J qui minimise $l_J(t)$ (tie-break déterministe).

1.19.2. LLF Scheduler

$$\tau_1 = (C_1 = 2, T_1 = 6, D_1 = 4)$$

$$\tau_2 = (C_2 = 4, T_2 = 5, D_2 = 6)$$

• $t = 0$

- $\tau_1 \rightarrow l_1(0) = 4 - 0 - 2 = 2$
- $\tau_2 \rightarrow l_2(0) = 6 - 0 - 4 = 2$
- **Tie-Break donc**
préemption au choix,
 τ_1 exécuté

• $t = 1$

- $\tau_1, e = 1 \rightarrow l_1(1) = 4 - 1 - 1 = 2$
- $\tau_2, e = 4 \rightarrow l_2(1) = 6 - 1 - 4 = 1$
- τ_2 exécuté

• $t = 2$

- $\tau_1, e = 1 \rightarrow l_1(2) = 4 - 2 - 1 = 1$
- $\tau_2, e = 4 \rightarrow l_2(2) = 6 - 2 - 3 = 1$
- **Tie-Break donc**
préemption au choix,
 τ_1 exécuté

• $t = 3$

- $\tau_1, e = 0 \rightarrow$ **Job Terminé**
- $\tau_2, e = 3 \rightarrow l_2(3) = 6 - 3 - 3 = 1$
- τ_2 exécuté

• $t = 4$

- $\tau_1, e = 0 \rightarrow$ **Job Terminé**
- $\tau_2, e = 2 \rightarrow l_2(4) = 6 - 4 - 2 = 0$
- τ_2 exécuté **Job Terminé**

• $t = 5$

- $\tau_1, e = 1 \rightarrow l_1(5) = 6 - 5 - 1 = 0$
- $\tau_2, e = 3 \rightarrow l_2(5) = 11 - 5 - 4 = 2$
- τ_1 exécuté **Job Terminé**

• $t = 6$ Nouveau relâchement

- $\tau_1, e = 2 \rightarrow l_1(6) = 10 - 6 - 2 = 2$
- $\tau_2, e = 4 \rightarrow l_2(6) = 11 - 6 - 4 = 1$
- τ_2 exécuté

• $t = 7$

- $\tau_1, e = 2 \rightarrow l_1(7) = 10 - 7 - 2 = 1$
- $\tau_2, e = 3 \rightarrow l_2(7) = 11 - 7 - 3 = 1$
- **Tie-Break donc**
préemption au choix,
 τ_1 exécuté

• $t = 8$

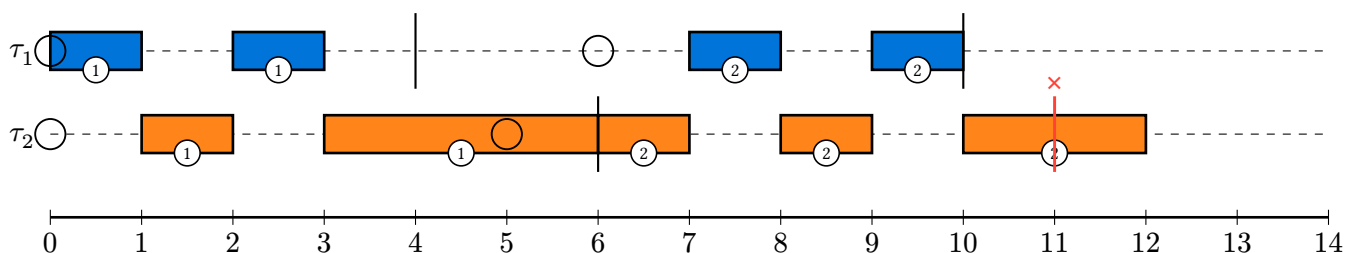
- $\tau_1, e = 1 \rightarrow l_1(8) = 10 - 8 - 1 = 1$
- $\tau_2, e = 3 \rightarrow l_2(8) = 11 - 8 - 3 = 0$
- τ_2 exécuté

• $t = 9$

- $\tau_1, e = 1 \rightarrow l_1(9) = 10 - 9 - 1 = 0$
- $\tau_2, e = 2 \rightarrow l_2(9) = 11 - 9 - 2 = 0$
- **Tie-Break donc**
préemption au choix,
 τ_1 exécuté

• $t = 10$

- $\tau_1, e = 1 \rightarrow l_1(10) = 11 - 10 - 2 = -1$
IMPOSSIBLE
- $\tau_2, e = 4 \rightarrow l_2(10) = 16 - 10 - 4 = 2$
- τ_2 exécuté



1.19.3. Question supplémentaires

- EDF aurait il fait mieux ?

Non le problème est infaisable, EDF ne pourrait pas faire mieux. $\sum C = 12 > 11$ sur $[0, 11]$

1.20. For what type(s) of tasks and which scheduling algorithm(s) is the response time of the first job of each task sufficient to conclude the schedulability of the task?

1.20.1. Résumé

Le test “je calcule le temps de réponse du premier job de chaque tâche et je vérifie $R_i \leq D_i$ ” est suffisant essentiellement dans le cadre mono-processeur préemptif avec priorités fixes (FTP) et tâches à deadline contrainte ($D_i \leq T_i$), quand le pire cas correspond au critical instant (relâchement synchrone).

Dès qu’on sort de ce cadre (deadlines arbitraires, offsets, EDF...), le pire cas peut arriver sur un job ultérieur, donc le “1er job” ne suffit plus.

1.20.2. Rappel

- **Utilisation** : $U = \sum_i \frac{C_i}{T_i}$

1.20.3. Cadre ou le premier job suffit

- Modele uni-processeur préemptif à taches indépendantes
- Taches périodiques ou sporadiques
- Ordonnanceur à priorités fixes FTP
 - Rate Monotonic T_i plus petite $\rightarrow$ plus prioritaire
 - Deadline Monotonic D_i plus petite $\rightarrow$ plus prioritaire
- Deadlines contraintes $D_i \leq T_i$
- Pire cas = critical instant (relâchement synchrone)
 - Justification : selon Liu & Layland, le pire cas pour une tâche sous FTP arrive quand toutes les tâches sont relâchées simultanément (critical instant). Donc le 1er job de chaque tâche, relâché à $t = 0$, subit le pire cas

Pourquoi le 1er job suffit-il ?

Dans un systeme synchrone $O_i = 0$:

- Sous FTP avec deadlines contraintes, le pire cas de réponse d’une tâche τ_i arrive quand toutes les tâches sont relâchées en même temps (critical instant)
- Donc le 1er job de τ_i , relâché à $t = 0$, subit le pire cas
- Si $R_i \leq D_i$ pour ce job, alors tous les jobs de τ_i respecteront leur deadline.

Il n’y aura pas d’accumulation d’erreurs car les deadlines sont contraintes.

Si pour tout $i \in [1, n]$, $R_i^1 \leq D_i$ alors le systeme est ordonnançable sous ce FTP

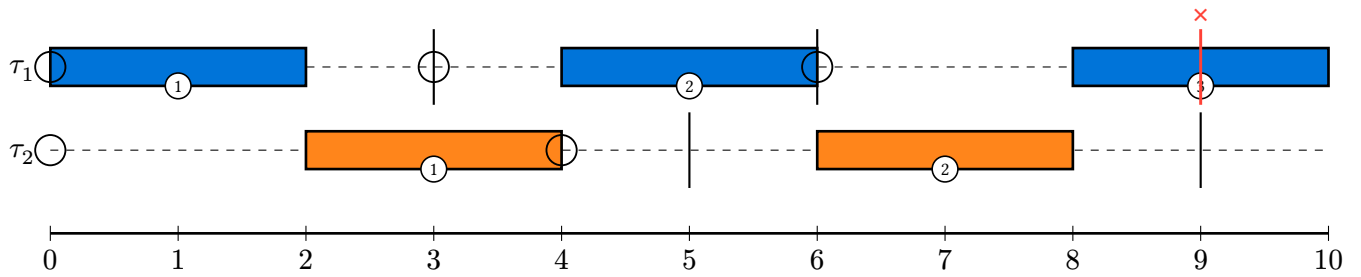
1.20.4. Cas ou le premier job ne suffit pas

- Deadlines arbitraires $D_i > T_i$
 - Un job peut rater sa deadline même si le 1er job passe
 - Exemple classique :

$$\tau_1 = (C_1 = 2, D_1 = 3, T_1 = 3)$$

$$\tau_2 = (C_2 = 2, D_2 = 5, T_2 = 4)$$

$$\tau_1 \succ \tau_2$$



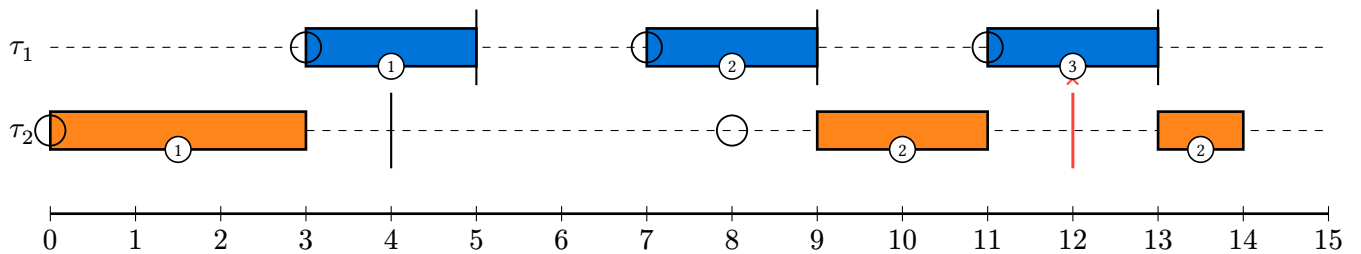
- Offsets $O_i > 0$

- Le pire cas peut arriver plus tard que le 1er job
- Exemple classique :

$$\tau_1 = (O_1 = 3, C_1 = 2, D_1 = 2, T_1 = 4)$$

$$\tau_2 = (O_2 = 0, C_2 = 3, D_2 = 4, T_2 = 8)$$

$$\tau_1 \succ \tau_2$$



- Ordonnanceur dynamique (EDF, LLF)

- Le pire cas peut arriver plus tard que le 1er job
- Exemple classique :

$$\tau_1 = (C_1 = 2, T_1 = 6, D_1 = 4)$$

$$\tau_2 = (C_2 = 4, T_2 = 5, D_2 = 6)$$

- $t = 0$

- $\tau_1 \rightarrow l_1(0) = 4 - 0 - 2 = 2$
- $\tau_2 \rightarrow l_2(0) = 6 - 0 - 4 = 2$
- **Tie-Break donc**
préemption au choix,
 τ_1 exécuté

- $t = 1$

- $\tau_1, e = 1 \rightarrow l_1(1) = 4 - 1 - 1 = 2$
- $\tau_2, e = 4 \rightarrow l_2(1) = 6 - 1 - 4 = 1$
- **τ_2 exécuté**

- $t = 2$

- $\tau_1, e = 1 \rightarrow l_1(2) = 4 - 2 - 1 = 1$
- $\tau_2, e = 4 \rightarrow l_2(2) = 6 - 2 - 3 = 1$
- **Tie-Break donc**
préemption au choix,
 τ_1 exécuté

- $t = 3$

- $\tau_1, e = 0 \rightarrow$ **Job Terminé**
- $\tau_2, e = 3 \rightarrow l_2(3) = 6 - 3 - 3 = 0$
- **τ_2 exécuté**

- $t = 4$

- $\tau_1, e = 0 \rightarrow$ **Job Terminé**
- $\tau_2, e = 2 \rightarrow l_2(4) = 6 - 4 - 2 = 0$
- **τ_2 exécuté Job Terminé**

- $t = 5$

- $\tau_1, e = 1 \rightarrow l_1(5) = 6 - 5 - 1 = 0$
- $\tau_2, e = 3 \rightarrow l_2(5) = 11 - 5 - 4 = 2$
- **τ_1 exécuté Job Terminé**

- $t = 6$ **Nouveau relâchement**

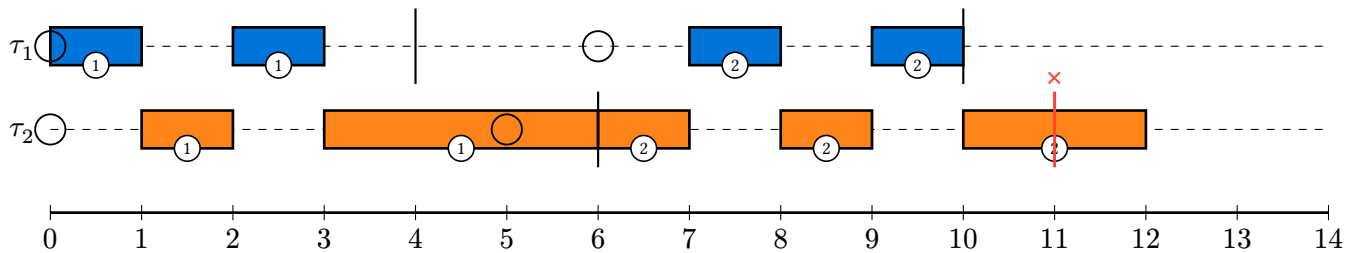
- $t = 7$

- $\tau_1, e = 2 \rightarrow l_1(7) = 10 - 7 - 2 = 1$

- $t = 8$

- $\tau_1, e = 1 \rightarrow l_1(8) = 10 - 8 - 1 = 1$

- $\tau_1, e = 2 \rightarrow l_1(6) = 10 - 6 - 2 = 2$
 - $\tau_2, e = 4 \rightarrow l_2(6) = 11 - 6 - 4 = 1$
 - τ_2 exécuté
- $t = 9$
 - $\tau_1, e = 1 \rightarrow l_1(9) = 10 - 9 - 1 = 0$
 - $\tau_2, e = 2 \rightarrow l_2(9) = 11 - 9 - 2 = 0$
 - **Tie-Break donc**
préemption au choix,
 τ_1 exécuté
- $\tau_2, e = 3 \rightarrow l_2(7) = 11 - 7 - 3 = 1$
 - **Tie-Break donc**
préemption au choix,
 τ_1 exécuté
- $\tau_2, e = 3 \rightarrow l_2(8) = 11 - 8 - 3 = 0$
 - τ_2 exécuté
- $t = 10$
 - $\tau_1, e = 1 \rightarrow l_1(10) = 11 - 10 - 2 = -1$
IMPOSSIBLE
 - $\tau_2, e = 4 \rightarrow l_2(10) = 16 - 10 - 4 = 2$
 - τ_2 exécuté



1.21. Provide at least three different feasibility intervals, specifying the type of tasks and the scheduler involved for each.

1.21.1. Résumé

Un intervalle de faisabilité est un intervalle fini tel que si aucune deadline n'est ratée pour tous les jobs relâchés dedans, alors aucune deadline ne sera jamais ratée ensuite.

1.21.2. Systeme synchrone à deadlines contraintes FTP (RM,DM,...):

Deadlines contraintes $D_i \leq T_i$

$$[0, D_{\max}) \text{ avec } D_{\max} = \max_{i=1} D_i$$

C'est un intervalle de faisabilité pour les tâches périodiques synchrones à deadlines contraintes sous FTP (donc RM/DM inclus).

En synchrone + $D_i \leq T_i$, le "pire" se voit dès le début. Si aucun job ne rate avant la plus grande deadline relative $D_{\max}$, alors rien de "nouveau" ne peut apparaître plus tard pour casser la faisabilité.

1.21.3. Systeme synchrone à deadlines arbitraires FTP (RM,DM,...):

Deadlines arbitraires $D_i > T_i$

Ici D_i peut être $> T_i$:

$$[0, \lambda)$$

ou λ est le premier idle point (fin du premier busy period) après $t = 0$.

1.21.4. Systèmes asynchrones, deadlines contraintes

Deadlines contraintes $D_i \leq T_i$

$$[0, O_{\max} + 2P)$$

avec :

$$O_{\max} = \max_{i=1}^n O_i$$

$$P = \text{lcm}(T_1, T_2, \dots, T_n)$$

Le scheduler devient périodique après $O_{\max} + P$, donc on peut se limiter à $O_{\max} + 2P$ pour vérifier la faisabilité.

1.21.5. Systèmes asynchrones, deadlines contraintes – FTP

Deadlines arbitraires $D_i > T_i$

$$[0, S_n + P)$$

ou les S_i sont définis par récurrence :

$$S_1 = O_1$$

$$S_i = \max \left(O_i, O_i + \sum_{j=1}^{i-1} \left\lceil \frac{S_i + O_i - O_j}{T_i} \right\rceil \cdot T_i \right) \text{ pour } i = 2, \dots, n$$

et $P = \text{lcm}(T_1, T_2, \dots, T_n)$

1.21.6. Réponses supplémentaires

- Pourquoi ces intervalles fonctionnent-ils ?

• Système synchrone, deadlines contraintes, FTP :

Parce que dans ce cadre, le pire cas de réponse d'une tâche τ_i arrive quand toutes les tâches sont relâchées en même temps (critical instant). Donc le 1er job de chaque tâche, relâché à $t = 0$, subit le pire cas. Si aucun job ne rate avant la plus grande deadline relative $D_{\max}$, alors rien de "nouveau" ne peut apparaître plus tard pour casser la faisabilité.

• Système synchrone, deadlines arbitraires, FTP :

En synchrone + deadlines arbitraires, le pire cas peut arriver plus tard que le 1er job. Cependant, on sait que EDF ne met pas le CPU idle avant qu'une deadline soit ratée. Donc la première fois que le CPU redevient idle (λ) marque la fin du "pire bloc continu" à vérifier.

• Système asynchrone, deadlines contraintes :

Après $O_{\max} + P$, le schéma de relâchement des tâches se répète (car les périodes sont commensurables). Donc si aucun job ne rate avant $O_{\max} + 2P$, rien de "nouveau" ne peut apparaître plus tard pour casser la faisabilité.

• Système asynchrone, deadlines arbitraires, FTP :

Après $S_n + P$, le schéma de relâchement des tâches se répète (car les périodes sont commensurables). Donc si aucun job ne rate avant $S_n + P$, rien de "nouveau" ne peut apparaître plus tard pour casser la faisabilité.

1.22. For global scheduling, prove that no online optimal scheduler exists.

1.22.1. Résumé

En ordonnancement global multiprocesseur, aucun ordonnanceur en ligne ne peut être optimal pour tous les systèmes de tâches.

L'idée clé est qu'un ordonnanceur en ligne ne connaît pas le futur : on peut construire deux scénarios indiscernables jusqu'à un instant t , mais qui exigent des décisions différentes pour rester faisables. Toute décision en ligne mènera donc à un échec dans au moins un des deux cas.

- Un ordonnanceur en ligne décide sans connaître les arrivées futures.
- Optimal signifie : s'il existe un ordonnancement faisable, l'algorithme le trouve.
- En multiprocesseur global, le futur influence fortement les décisions présentes (migrations, choix des tâches à exécuter).
- On peut donc forcer une contradiction :
 - une même décision locale est correcte dans un futur possible, mais incorrecte dans un autre futur également possible.

L'optimalité en ligne est donc impossible.

1.22.2. Rappel

- **En ligne** : décisions basées uniquement sur l'état courant, sans connaissance du futur.
- **Optimal** : si un ordonnancement faisable existe, l'algorithme le trouve.
- **Global** : les tâches peuvent migrer entre processeurs.
- **Indistinguabilité** : deux scénarios identiques jusqu'à un instant donné

1.22.3. Preuve formelle

Considérons un système avec :

- m processeurs identiques
- ordonnancement global (tâches peuvent migrer)
- des tâches périodiques ou sporadiques
- deadlines implicites ($D_i = T_i$) ou arbitraires
- algorithme en ligne (décisions basées uniquement sur l'état courant)

Hypothèse : Supposons qu'il existe un ordonnanceur en ligne optimal A. On construit deux systèmes de tâches τ et τ' tels que :

- Ils sont strictement identiques jusqu'à un instant t.
- L'ordonnanceur A prend donc exactement les mêmes décisions jusqu'à t.
- Après t, :
 - Dans τ , la décision prise par A permet de respecter toutes les deadlines.
 - Dans τ' , la même décision conduit à une violation de deadline.
- Pourtant τ' admet un ordonnancement faisable en prenant une décision différente à t.

Cela contredit l'optimalité de A.

1.22.4. Exemple

Considérons 2 processeurs ($m=2$) et les tâches suivantes :

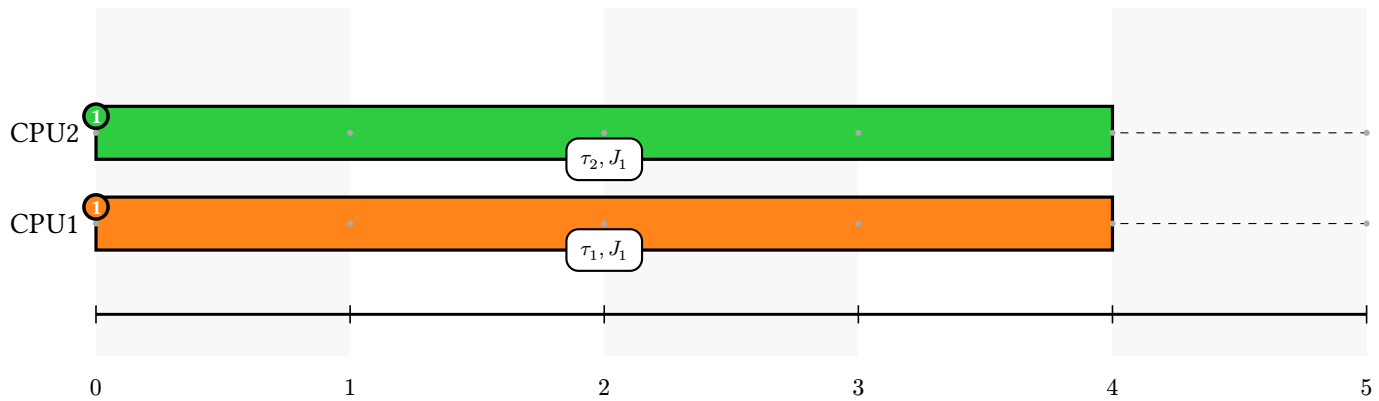
$$\tau_1 = (C_1 = 4, D_1 = 10)$$

$$\tau_2 = (C_2 = 4, D_2 = 10)$$

Utilisation totale $U = \left(\frac{C_1}{D_1} + \frac{C_2}{D_2}\right) = 0.8 < m = 2$, donc un ordonnancement faisable existe.

Scénario 1 : Les deux tâches arrivent à $t=0$.

- L'ordonnanceur A exécute τ_1 sur P1 et τ_2 sur P2.
- Les deux tâches terminent avant leurs deadlines respectives.



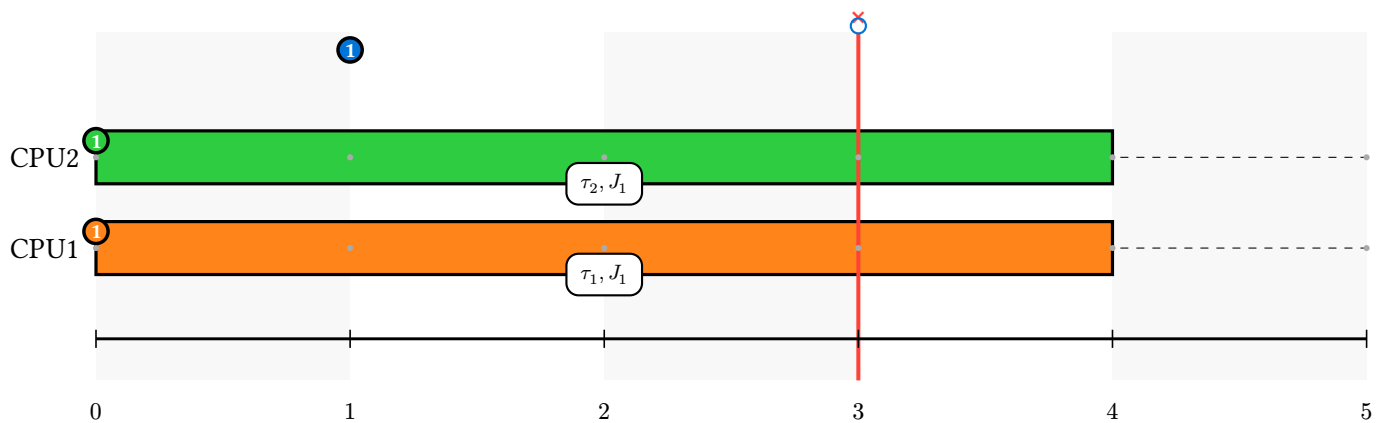
Scénario 2 : à $t = 1$ une tâche τ_3 arrive avec $(C_3 = 1, D_3 = 2)$.

→ τ_3 doit s'exécuter entre $t = 1$ et $t = 2$ **Problème**, à $t = 1$:

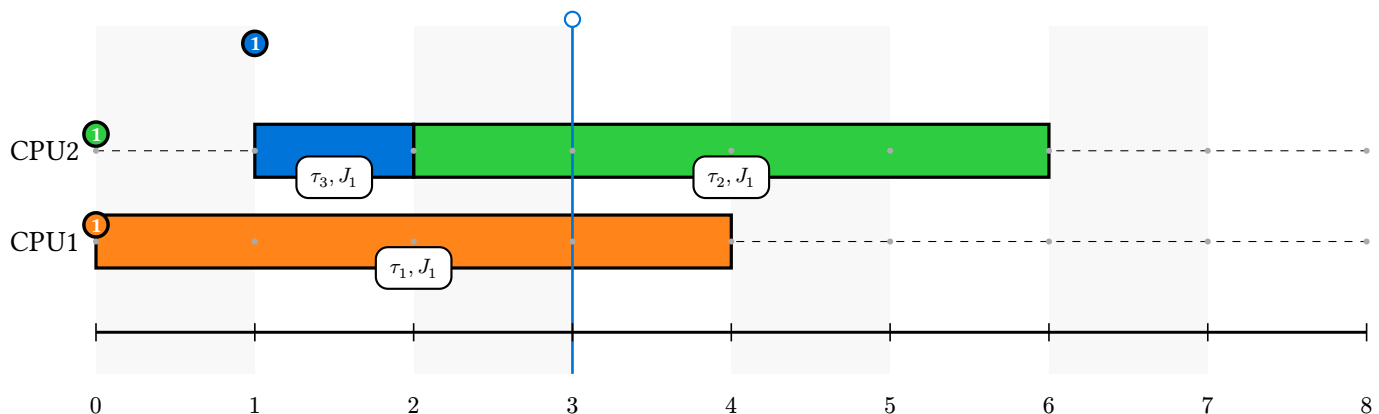
- P_1 exécute τ_1 (reste 3 unités)
- P_2 exécute τ_2 (reste 3 unités)

Mais il existait un ordonnancement faisable si à $t = 1$, A avait préempté τ_2 pour exécuter τ_3 sur P_2 .

On ne pouvait pas savoir que τ_3 arriverait à $t = 1$ lors de la décision initiale. Ainsi, l'ordonnanceur A ne peut pas être optimal pour les deux scénarios.



Ordonnancement faisable alternatif :



1.22.5. Conclusion

Nous avons montré qu'aucun ordonnanceur en ligne ne peut être optimal en ordonnancement global multiprocesseur, car des décisions prises sans connaissance du futur peuvent conduire à des

violations de deadlines dans certains scénarios, même si un ordonnancement faisable existe, il n'est pas optimal.

1.23. Explain why the task-partitioning problem for implicit-deadline periodic tasks on identical processors can be interpreted as a special case of the bin-packing problem. For $m = 2$, provide a small task set for which First-Fit Decreasing Utilisation (FFDU) and Worst-Fit Decreasing Utilisation (WFDU) produce different partitions, and discuss which of the two heuristics yields the better assignment with your example.

1.23.1. Résumé

Le partitionnement de tâches périodiques à deadline implicite sur processeurs identiques revient à “mettre des objets dans des boîtes” : chaque tâche a une taille = utilisation U_i , chaque processeur est une boîte de capacité 1 sous EDF. C'est exactement un cas particulier de bin-packing, donc NP-complet, d'où l'usage d'heuristiques gloutonnes comme FFDU et WFDU.

1.23.2. Rappel

1.23.3. Pourquoi c'est un cas de bin-packing ?

On est dans le cadre **deadline implicite** : $D_i = T_i$ pour chaque tâche τ_i .

$$U_i = U(\tau_i) = \frac{C_i}{T_i}$$
$$U(\tau) = \sum_{i=1}^n \frac{C_i}{T_i}$$

Sous EDF, si on assigne un sous-ensemble de tâches à un processeur π_k , une condition de base (EDF → **nécessaire et suffisante**):

$$\sum_{\tau_i \in \pi_k} U_i \leq 1$$

Lien entre RTOS et bin-packing :

Objets ↔ tâches avec taille U_i

Bins ↔ coeurs

Capacité des bins ↔ Borne d'utilisation admissible (sous EDF : 1, RM : une borne plus faible)

- **Objectif** : Minimiser le nombre de processeurs utilisés (bins remplis) tout en respectant la contrainte d'utilisation. Donc **NP-Complet**

1.23.4. Definition FFDU et WFDU en partitionné EDF

On trie d'abord les tâches par utilisation décroissantes. Ensuite à chaque tâches courante τ_i :

FFDU (First-Fit Decreasing Utilisation):

- Parcourir les processeurs dans un ordre fixe $\pi_1, \pi_2, \dots, \pi_m$
- Placer τ_i sur le premier processeur où elle “rentre” :

$$\text{si } U(\pi_k) + U_i \leq 1$$

Principe : Remplir les processeurs le plus tôt possible. Avantages :

- Simple et rapide

- Bonnes performances en pratique

Inconvénients :

- Peut conduire à des déséquilibres de charge entre processeurs
- Peut saturer certains processeurs rapidement

WFDU (Worst-Fit Decreasing Utilisation):

- Placer τ_i sur un processeur éligible ayant la charge la plus faible (minimum $U(\pi_k)$)

Principe : Équilibrer la charge au maximum. Avantages :

- Meilleur équilibre de charge entre processeurs
- Plus robuste face aux variations de charge

Inconvénients :

- Peut fragmenter l'espace disponible
- Peut empêcher l'ajout de nouvelles tâches

Best Fit Decreasing Utilisation (BFDU) : Placer τ_i sur le processeur éligible qui laisserait le moins d'espace libre après placement.

1.23.5. Exemple avec $m = 2$ ou FFDU et WFDU différent

Considérons les 3 tâches (deadline implicite) avec utilisation suivantes :

$$U_1 = 0.6, U_2 = 0.4, U_3 = 0.4$$

Application de FFDU :

- Trier : $\tau_1(0.6), \tau_2(0.4), \tau_3(0.4)$
- Placer τ_1 sur $\pi_1 : U(\pi_1) = 0.6$

Charges : $U(\pi_1) = 0.6, U(\pi_2) = 0$

- Placer τ_2 sur $\pi_1 : U(\pi_1) = 1.0$

Charges : $U(\pi_1) = 1.0, U(\pi_2) = 0$

- Placer τ_3 sur $\pi_2 : U(\pi_2) = 0.4$

Final : $U(\pi_1) = 1.0, U(\pi_2) = 0.4$

Application de WFDU :

- Trier : $\tau_1(0.6), \tau_2(0.4), \tau_3(0.4)$
- Placer τ_1 sur $\pi_1 : U(\pi_1) = 0.6$

(Tie-breaker arbitraire)

Charges : $U(\pi_1) = 0.6, U(\pi_2) = 0$

- Placer τ_2 sur :

$$\pi_1 : 0.6 + 0.4 = 1(\text{éligible})$$

$$\pi_2 : 0 + 0.4 = 0.4(\text{éligible})$$

Choix : π_2 (charge minimale)

Charges : $U(\pi_1) = 0.6, U(\pi_2) = 0.4$

- Placer τ_3 sur :

$$\pi_1 : 0.6 + 0.4 = 1(\text{éligible})$$

$$\pi_2 : 0.4 + 0.4 = 0.8(\text{éligible})$$

Final : $U(\pi_1) = 0.6, U(\pi_2) = 0.8$

1.23.6. Discussion : Quelle heuristique est meilleure ?

Comparaison des résultats :

- FFDU : $U(\pi_1) = 1.0, U(\pi_2) = 0.4$
- WFDU : $U(\pi_1) = 0.6, U(\pi_2) = 0.8$

Analyse :

- FFDU remplit complètement π_1 , mais laisse π_2 sous-utilisé.
- WFDU équilibre mieux la charge entre les deux processeurs.

- Aucune heuristique n'est optimale dans l'absolu, mais dans ce cas WFDU offre une meilleure répartition de la charge.

Dans le cas où l'on veut minimiser la charge maximale sur un processeur, WFDU est préférable ici. Mais si l'on veut garder de la place pour des tâches futures, FFDU pourrait être avantageux.

Par exemple, si une nouvelle tâche avec $U_4 = 0.3$ arrive :

- Avec FFDU, π_1 est plein, π_2 a 0.6 de libre, donc on peut l'ajouter.
- Avec WFDU, π_1 a 0.4 de libre, π_2 a 0.2 de libre, donc on ne peut pas l'ajouter.

Conclusion : Dans cet exemple, WFDU offre une meilleure répartition de la charge. Mais le choix entre FFDU et WFDU dépend du contexte spécifique et des objectifs et de ce que l'on veut optimiser:

- charge max FFDU
- équilibre WFDU
- capacité d'extension future
- nombre de processeurs utilisés

On voit aussi que les heuristiques sont locales:

- Décision sans retour en arrière
- Pas de garantie d'optimalité globale

1.24. Define the Next-Fit Decreasing Utilisation (NFDU) heuristic in the context of partitioned EDF scheduling. Apply NFDU to the following task set: $(C, T) = (5, 10), (2, 5), (1, 4), (1, 4), (1, 8), (1, 8), (1, 8), (1, 8)$. Show the resulting partition and comment on why NFDU may be highly sub-optimal, even when every task has utilisation at most 0.5.

1.24.1. Résumé

NFDU (Next-Fit Decreasing Utilisation) est une heuristique de partitionnement pour l'ordonnancement EDF partitionné : on trie les tâches par utilisation décroissante, puis on les place dans le processeur courant tant que ça tient, sinon on passe au suivant (sans jamais revenir en arrière).

Sur le jeu de tâches donné, NFDU produit une partition sur 3 processeurs, alors qu'une partition sur 2 processeurs existe : c'est un exemple clair de sa forte sous-optimalité, même si chaque tâche a une utilisation ≤ 0.5 .

1.24.2. Rappel

- **Implicit deadline** : $D_i = T_i$
- **Utilisation** : $U_i = \frac{C_i}{T_i}$
- **EDF Condition** : $\sum_{\tau_i \in \pi_k} U_i \leq 1 \Leftrightarrow$ nécessaire et suffisant pour schedule par EDF
- **EDF partitionné** : On veut une partition des tâches en m sous-ensembles $P_1, \dots, P_n$ tels que, pour chaque processeur k ,

$$U(P_k) = \sum_{\tau_i \in P_k} U_i \leq 1$$

chaque partition étant ordonnancée par EDF localement.

1.24.3. Définition de NFDU

Entrée : un ensemble de tâches $\tau = \{\tau_1, \tau_2, \dots, \tau_n\}$ avec $U_i = \frac{C_i}{T_i}$ et nombre de processeurs m .

Étapes :

1. Calculer l'utilisation de chaque tâche $U_i = \frac{C_i}{T_i}$.

2. Trier les tâches par utilisation décroissante : $U_1 \geq U_2 \geq \dots \geq U_n$.
3. Initialiser le processeur courant P_1 , avec la charge $U = 0$
4. Pour chaque tâche τ_i dans l'ordre trié :
 1. Si $U + U_i \leq 1$:
 1. Assigner τ_i à P_k
 2. Mettre à jour la charge : $U = U + U_i$
 - Sinon :
 1. Passer au processeur suivant P_{k+1}
 2. Assigner τ_i à P_{k+1}
 3. Initialiser la charge : $U = U_i$

Point clé : On ne revient jamais en arrière pour réévaluer les processeurs précédents.

1.24.4. Application de NFDU au jeu de tâches donné

Considérons les tâches avec (C, T) et leur utilisation déjà ordonnée par utilisation décroissante :

- $\tau_1 = (5, 10) \rightarrow U_1 = 0.5$
- $\tau_2 = (2, 5) \rightarrow U_2 = 0.4$
- $\tau_3 = (1, 4) \rightarrow U_3 = 0.25$
- $\tau_4 = (1, 4) \rightarrow U_4 = 0.25$
- $\tau_5 = (1, 8) \rightarrow U_5 = 0.125$
- $\tau_6 = (1, 8) \rightarrow U_6 = 0.125$
- $\tau_7 = (1, 8) \rightarrow U_7 = 0.125$
- $\tau_8 = (1, 8) \rightarrow U_8 = 0.125$

Application de NFDU : Capacité de chaque processeur = 1

- P_1 :
 - Place $\tau_1 (U = 0.5) : U = 0.5$
 - Place $\tau_2 (U = 0.4) : U = 0.9$
 - τ_3 ne rentre pas ($0.9 + 0.25 > 1$)
- P_2 :
 - Place $\tau_3 (U = 0.25) : U = 0.25$
 - Place $\tau_4 (U = 0.25) : U = 0.5$
 - Place $\tau_5 (U = 0.125) : U = 0.625$
 - Place $\tau_6 (U = 0.125) : U = 0.75$
 - Place $\tau_7 (U = 0.125) : U = 0.875$
 - Place $\tau_8 (U = 0.125) : U = 1.0$

Résultat final :

- $P_1 : \tau_1, \tau_2$ (Utilisation = 0.9)
- $P_2 : \tau_3, \tau_4, \tau_5, \tau_6, \tau_7, \tau_8$ (Utilisation = 1.0)

1.24.5. Pourquoi NFDU peut être très sous-optimal

Avec chaque tâches $u_i \leq 0.5$, tout item peut être païré avec un autre item dans le même processeur sans dépasser la capacité 1.

Le soucis est que NFDU ne réévalue jamais les processeurs précédents. Il peut donc dans les pires cas utiliser plus de processeurs que nécessaire. On a donc : **Next-Fit = décision irréversible → risque de trous (fragmentation). → Sous-optimalité.**

1.24.6. Questions supplémentaires

- **Complexité** : NFDU est $O(n \log n)$ à cause du tri initial car le placement est $O(n)$ et le tri $O(n \log n)$.
- **Pourquoi on trie en décroissant ?** : Pour maximiser l'utilisation de chaque processeur dès le début, réduisant ainsi le nombre total de processeurs utilisés.
- **Différences NFDU vs FFDU/WFDU** :
 - NFDU : un seul processeur actif à la fois, pas de retour en arrière.
 - FFDU : essaie de remplir le premier processeur possible.
 - WFDU : essaie d'équilibrer la charge entre processeurs en cherchant le moins chargé.

1.25. Define the concept of scheduling anomalies, illustrate the phenomenon for global FTP scheduling and the period parameter.

1.25.1. Résumé

Une scheduling anomaly (anomalie d'ordonnancement) en multiprocesseur est une situation où une modification "a priori bénéfique" des paramètres (par ex. augmenter une période $\rightarrow$ réduire l'utilisation) rend pourtant le système moins ordonnançable, voire fait rater une deadline.

Le cours illustre ça en global FTP (ex : global DM) : augmenter la période d'une tâche peut provoquer un miss alors que le système était schedulable avant.

1.25.2. Scheduling anomalies

Une anomalie survient quand un changement de paramètres qui semble intuitivement positif met en danger l'ordonnançabilité. Un changement "bénéfique" peut être :

- Augmenter les périodes des tâches (diminue l'utilisation)
- Diminuer les temps d'exécution
- Augmenter la vitesse des processeurs

En quoi c'est spécifique au multiprocesseur ? En multi-processeur global on a souvent des situations où :

- Les tâches peuvent migrer entre processeurs ou être préemptées fréquemment.
- **Un job ne pas s'exécuter en parallèle sur plusieurs processeurs.**

1.25.3. Exemple

Prenons le système de tâches suivant, ordonnancé par global DM sur 2 processeurs :

$$\tau_1 = (C_1 = 2, T_1 = 4, D_1 = 2)$$

$$\tau_2 = (C_2 = 3, T_2 = 5, D_2 = 3)$$

$$\tau_3 = (C_3 = 7, T_3 = 20, D_3 = 8)$$

$$\tau_1 \succ \tau_2 \succ \tau_3$$

On augmente ensuite la période de τ_1 : $T_1 : 4 \rightarrow 5$. Ce qui peut sembler positif mais l'exemple montre que τ_3 rate sa deadline $t = 8$.

- Donc si $T \uparrow \Rightarrow$ deadline miss

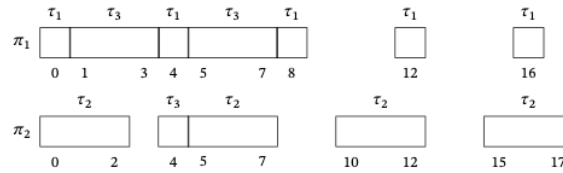


Figure 8.1: Execution of System 83

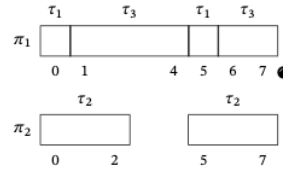


Figure 8.2: Execution System 83 with $T_1 = 5$

Quand τ_1 a une période de 4, ses activations “rythment” l’ordonnancement d’une manière qui laisse à τ_3 des créneaux suffisamment “bons” sur les 2 processeurs pour cumuler 7 unités avant $t = 8$. Quand on passe à $T_1 = 5$, les activations de τ_1 se décalent, perturbant cet équilibre et empêchant τ_3 de terminer à temps à cause de l’absence du parallélisme entre jobs (un seul job τ_3 ne peut pas se découper en 2 et tourner à la fois sur le processeur 1 et 2).

1.25.4. Questions supplémentaires

- **Est-ce que ça peut arriver en uniprocasseur ?** Ce n’est pas possible en uniprocasseur car il n’y a pas de parallélisme ni de migrations entre processeurs.

1.26. !!!!! In the context of PFair scheduling, define lag and explain the requirement $-1 < \text{lag}(\tau_i, t) < 1$. Show, on a small example with two periodic tasks, that enforcing this bound ensures that each job completes within its deadline, and explain intuitively why PFair scheduling is optimal for periodic implicit-deadline systems.

1.26.1. Résumé

PFair est une version discrète d’un ordonnancement idéal équitable où chaque tâche reçoit exactement une fraction $U_i = \frac{C_i}{T_i}$ du CPU en continu. La notion clé est le **lag**: L’écart entre le “service idéal” et le “service réel” d’une tâche.

Imposer $|\text{lag}| < 1$ force chaque tâche à rester à moins d’un quantum de son service idéal, ce qui garantit que chaque job termine avant sa deadline (implicité $D_i = T_i$). PFair est optimal car il minimise le lag pour chaque tâche.

1.26.2. Définition

1. Scheduling discret :

On modélise un ordonnancement discret par une fonction :

$$S : \tau \times \mathbb{N} \rightarrow \{0, 1\}$$

ou $S(\tau_i, l) = 1$ signifie que τ_i est exécutée sur un quantum $[l, l + 1)$

2. Définition du lag:

Pour une tâche τ_i , le lag à l'instant t est défini comme:

$$\text{lag}(\tau_i, t) = U_i \cdot t - \sum_{l=0}^{t-1} S(\tau_i, l)$$

- $U_i \cdot t$ est le service idéal que τ_i devrait avoir reçu sur $[0, t)$.
- $\sum_{l=0}^{t-1} S(\tau_i, l)$ est le service réel reçu par τ_i sur $[0, t)$.

3. Pfair scheduling:

PFair est un ordonnanceur multiprocesseur optimal pour les systèmes périodiques à deadlines implicites. Il approxime un partage de processeur idéal en garantissant que chaque tâche reste à moins d'un quantum de son service théorique, ce qui assure que toutes les deadlines sont respectées.

- Chaque tâche $\tau_i = (C_i, T_i)$ doit recevoir une fraction constante du CPU $U_i = \frac{C_i}{T_i}$
 - En continu : on ferait du processor sharing parfait
 - En discret : PFair force l'ordonnancement à rester proche de cet idéal en utilisant la notion de lag.

1.26.3. Preuve

- Si $\text{lag}(\tau_i, t)$ est grand et positif, τ_i a reçu moins de service que son idéal. τ_i est donc en retard.
- Si $\text{lag}(\tau_i, t)$ est négatif, τ_i a pris de l'avance.

Imposer

$$-1 < \text{lag}(\tau_i, t) < 1 \quad \forall i, \forall t \in \mathbb{N}$$

signifie à tout instant discret, chaque tâche est à moins d'un quantum de son allocation idéale. Donc elle reçoit forcément soit

$$\lfloor U_i t \rfloor \text{ ou } \lceil U_i t \rceil$$

sur le quantum $[0, t)$

1.26.4. Exemple

Prenons 1 processeur (même logique sur $m > 1$, juste "jusqu'à m tâches par quantum")

$$\tau_1 = (C_1 = 1, T_1 = 2) \Rightarrow U_1 = 0.5$$

$$\tau_2 = (C_2 = 1, T_2 = 4) \Rightarrow U_2 = 0.25$$

$$U(\tau) = 0.75 < 1 \rightarrow \text{faisable}$$

Construisons un Pfair schedule sur $[0, 4)$:

$$[0, 1) \rightarrow \tau_1 \Rightarrow S(\tau_1, 0) = 1, S(\tau_2, 0) = 0$$

$$[1, 2) \rightarrow \tau_2 \Rightarrow S(\tau_1, 1) = 0, S(\tau_2, 1) = 1$$

$$[2, 3) \rightarrow \tau_1 \Rightarrow S(\tau_1, 2) = 1, S(\tau_2, 2) = 0$$

$$[3, 4) \rightarrow \text{idle} \Rightarrow S(\tau_1, 3) = 0, S(\tau_2, 3) = 0$$

Lag de τ_1 :

$$\text{lag}(\tau_1, t) = 0.5t - \sum_{l=0}^{t-1} S(\tau_1, l)$$

$$t = 1 \rightarrow 0.5 - 1 = -0.5$$

$$t = 2 \rightarrow 1 - 1 = 0$$

$$t = 3 \rightarrow 1.5 - 2 = -0.5$$

$$t = 4 \rightarrow 2 - 2 = 0$$

toujours dans $(-1, 1)$

Lag de τ_2 :

$$\text{lag}(\tau_2, t) = 0.25t - \sum_{l=0}^{t-1} S(\tau_2, l)$$

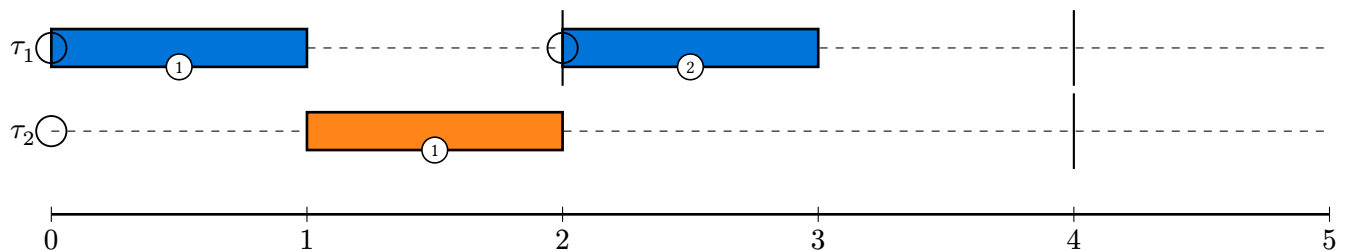
$$t = 1 \rightarrow 0.25 - 0 = 0.25$$

$$t = 2 \rightarrow 0.5 - 1 = -0.5$$

$$t = 3 \rightarrow 0.75 - 1 = -0.25$$

$$t = 4 \rightarrow 1 - 1 = 0$$

toujours dans $(-1, 1)$



On voit que le “principe “ de lag ne laisse jamais une tâche accumuler plus d’un quantum de retard, donc pas de dérive qui ferait rater une deadline

1.26.5. Optimalité de PFair

1. En temps continu, l’idéal fair schedule est parfait si $U(\tau) \leq m$ et $U_{\max} \leq 1$. On peut donner à chaque tâche une fraction constante de U_i sur chaque instant, et toutes les deadlines sont respectées.
2. En temps discret on ne peut pas donner une fraction à l’intérieur d’un quantum.
3. PFair est précisément la “meilleure approximation discrète” : elle garantit que l’écart à l’idéal reste < 1 quantum en permanence. Donc, aux instants critiques $t = lT_i$, on retombe exactement sur l’allocation idéale, ce qui force chaque job à avoir reçu ses C_i unités dans sa fenêtre.

Donc, pour les systèmes périodiques à deadlines implicites (où la fenêtre naturelle est $[lT_i, (l+1)T_i)$, PFair atteint le comportement “idéal” aux bons instants et ne peut pas faire mieux : si un système est faisable, PFair le réalisera (c’est l’idée derrière “optimal scheduler” dans ce cadre).

1.26.6. Pourquoi PFair garantit les deadlines

- A chaque frontière de période $t = lT_i$:

$$U_i t = lC_i \in \mathbb{N}$$

- Avec $|\text{lag}| < 1$, la tâche a reçu exactement lC_i quanta
- Donc chaque job reçoit ses C_i unités dans sa fenêtre $[r_i, d_i)$

Tout les jobs finissent avant leur deadline.

1.27. Given the task set $(C, T) = (1, 3), (2, 6), (5, 15), (6, 18)$, construct the global DM schedule on two processors over interval $[0, 18)$. Indicate all preemptions and migrations. State whether any deadline is missed and justify your conclusion.

1.27.1. Rappel

Global DM :

- Priorité fixe
- Plus petite deadline = plus haute priorité
- $D_i = T_i$ (deadlines implicites)

Ordonnancement global :

- Une file globale
- Les $m = 2$ tâches prêtes les plus prioritaires sont exécutées
- Prémption autorisée
- Migration autorisée

1.27.2. Tâches et priorités

$$\tau_1 = (C_1 = 1, T_1 = 3) \Rightarrow U_1 = \frac{1}{3} \approx 0.33$$

$$\tau_2 = (C_2 = 2, T_2 = 6) \Rightarrow U_2 = \frac{2}{6} \approx 0.33$$

$$\tau_3 = (C_3 = 5, T_3 = 15) \Rightarrow U_3 = \frac{5}{15} \approx 0.33$$

$$\tau_4 = (C_4 = 6, T_4 = 18) \Rightarrow U_4 = \frac{6}{18} \approx 0.33$$

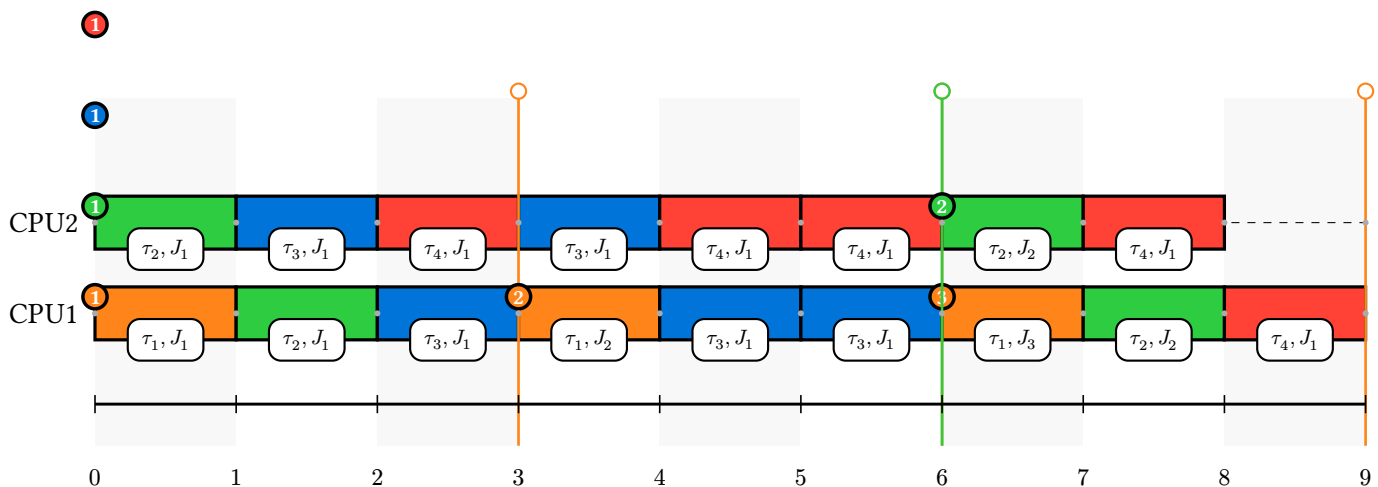
$$\tau_1 \succ \tau_2 \succ \tau_3 \succ \tau_4$$

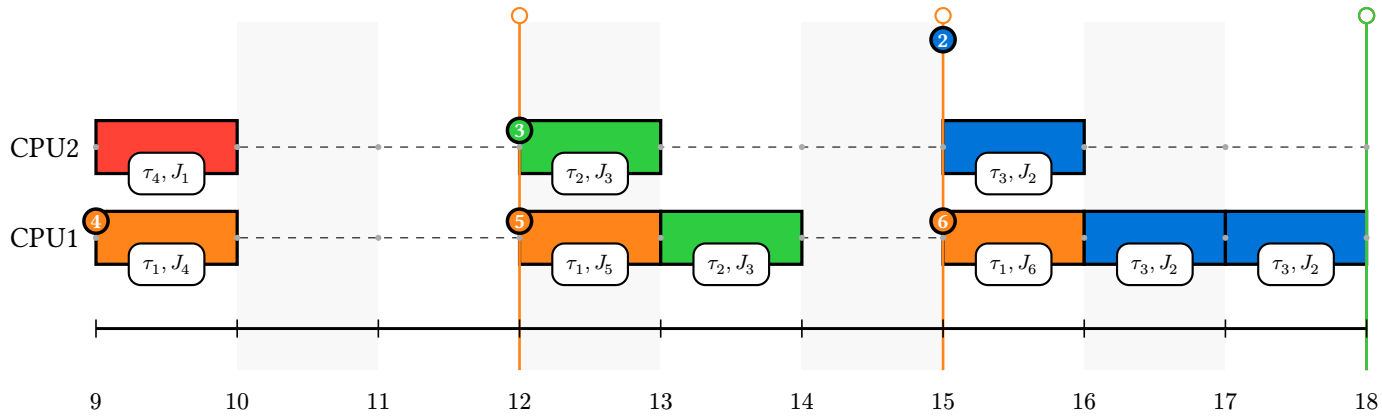
$$U(\tau) = \frac{1}{3} + \frac{2}{6} + \frac{5}{15} + \frac{6}{18} = \frac{4}{3} \leq m = 2 \rightarrow \text{faisable}$$

1.27.3. Ordonnancement

Ordonnancement G-DM sur 2 processeurs

$$\tau_1 \succ \tau_2 \succ \tau_3 \succ \tau_4$$





Un ordonnancement global DM sur deux processeurs est faisable sur $[0,18)$.

Aucune deadline n'est manquée car les tâches les plus prioritaires sont toujours exécutées en premier, et la charge totale $U(\tau) = \frac{4}{3} \leq m = 2$ garantit la faisabilité.

1.28. Provide the basic sufficient condition for EDF schedulability of implicit deadline tasks, present the EDF(k) improvement.

1.28.1. Résumé

Pour des tâches périodiques à deadlines implicites sur m processeurs identiques, une condition suffisante simple garantit la faisabilité sous EDF global. L'algorithme EDF(k) améliore cette condition en traitant différemment les k tâches les plus lourdes (forte utilisation), ce qui réduit le nombre de processeurs requis.

1.28.2. Condition suffisante EDF global

Modèle :

- Tâches périodiques, deadline implicite : $D_i = T_i$
- Utilisation : $u_i = \frac{C_i}{T_i}$
- Utilisation total = $U = \sum_i u_i$
- m processeurs identiques

Condition suffisante :

Un ensemble de tâches est ordonnançable par EDF global si :

$$\begin{aligned} \max_i u_i &\leq 1 \\ U &\leq m - (m - 1) \cdot u_{\max} \\ \text{avec } u_{\max} &= \max_i u_i \end{aligned}$$

Intuition :

- $u_{\max} \leq 1$ aucune tâche ne demande plus qu'un processeur
- La seconde inégalité compense les interférences dues au partage global (migrations, préemptions) : plus une tâche est lourde, plus elle "consomme" la marge globale.

Cette condition est suffisante mais non nécessaire : EDF peut réussir même si elle n'est pas vérifiée.

1.28.3. Principe de l'amélioration EDF(k)

Idée :

- Trier les tâches par utilisation décroissante :

$$u_1 \geq u_2 \geq \dots \geq u_n$$

- Les k premières tâches sont réservées implicitement. Chacune est supposée occuper un processeur à elle seule.
- Les tâches restantes sont planifiées par EDF sur les processeurs restants.

On réduit l'interférence causée par les grosses tâches.

1.28.4. Condition suffisante EDF(k)

Pour un $k \in \{1, \dots, n\}$, l'ensemble est faisable si :

$$\forall i \leq k : u_i \leq 1$$

$$\sum_{i=k+1}^n u_i \leq (m - k)(1 - u_{k+1})$$

- Les k tâches les plus lourdes consomment chacune ≈ 1 processeurs.
- Il reste $m - k$ processeur pour le suatres.
- La pénalité dépend maintenant de u_{k+1} (la plus lourde des plus petites)
 - $k = 0 \rightarrow$ on retrouve la condition EDF global classique
 - $k = m \rightarrow$ toutes les tâches lourdes sont isolées

1.28.5. EDF(k) meilleur que EDF

- EDF global pénalise toutes la charge a cause de $u_{\max}$
- EDF(k) isole les tâches problématiques, ce qui :
 - Diminue la pénalité
 - réduit le nombre de processeurs requis
 - donne une borne plus serrée

1.28.6. Exemple

Considérons le systeme suivant :

$$\begin{aligned}\tau_1 &= (C = 3, T = 4) \Rightarrow u_1 = 0.75 \\ \tau_2 &= (C = 1, T = 4) \Rightarrow u_2 = 0.25 \\ \tau_3 &= (C = 1, T = 4) \Rightarrow u_3 = 0.25 \\ \tau_4 &= (C = 1, T = 4) \Rightarrow u_4 = 0.25 \\ U &= 0.75 + 0.25 + 0.25 + 0.25 = 1.5 \\ u_{\max} &= 0.75\end{aligned}$$

EDF global : Condition :

$$U \leq m - (m - 1) \cdot u_{\max}$$

Avec $m=2$:

$$\begin{aligned}2 - 1 \cdot 0.75 &= 1.25 \\ U = 1.50 &\not\leq 1.25\end{aligned}$$

Conclusion : la condition EDF global échoue $\rightarrow$ EDF peut quand même marcher, mais pas garanti par cette borne.

EDF(k) avec $k=1$: On trie les tâches par utilisation décroissante :

$$0.75(\tau_1) > 0.25(\tau_2) > 0.25(\tau_3) > 0.25(\tau_4)$$

On prend $k = 1$ Condition EDF(1):

$$\sum_{i=k+1}^n u_i \leq (m - k)(1 - u_{k+1})$$

$$\sum_{i=2}^4 u_i \leq (2 - 1)(1 - 0.25)$$

$$0.75 \leq 1 \cdot (1 - 0.25) = 0.75$$

Conclusion : EDF(1) garantit la faisabilité sur 2 processeurs, alors que la borne EDF global ne le garantit pas.

Intuition :

- τ_1 au $u_1 = 0.75$
- EDF global dit :
 - “Cette tâche peut bloquer presque un CPU $\rightarrow$ je pénalise tout le monde”
- EDF(1) dit :
 - “Je suppose qu’elle monopolise un CPU, et je regarde si le reste tient sur l’autre”

1.28.7. Question supplémentaire

Est-ce que EDF(k) change l’ordonnancement ?

Non. EDF(k) ne change que l’analyse, pas le scheduler.

EDF(k) est-il optimal ? Non, c’est une borne suffisante améliorée. On peut donc rater une deadline.

1.29. Consider the sporadic implicit-deadline system $(C, T) = (7, 10), (3, 12), (3, 12), (2, 8), (1, 5)$. Sort the tasks by decreasing utilisation. For each value $k \in \{1, \dots, 5\}$, compute the required number of processors according to the EDF(k). Identify the optimal value of k and comment on the improvement compared to pure global EDF.

1.29.1. Résumé

On calcule d’abord l’utilisation $U_i = \frac{C_i}{T_i}$, on trie par U_i décroissant, puis on applique le test suffisant EDF(k):

$$m \geq (k - 1) + \left\lceil \frac{U(\tau^{k+1})}{1 - U(\tau_k)} \right\rceil$$

ou $\tau^{k+1} = \{\tau_{k+1}, \dots, \tau_n\}$ donc $U(\tau^{k+1}) = \sum_{i=k+1}^n u_i$.

1.29.2. Résolution

Système :

$$\begin{aligned} \tau_1 &= (C_1 = 7, T_1 = 10) \Rightarrow u_1 = 0.7 \\ \tau_2 &= (C_2 = 3, T_2 = 12) \Rightarrow u_2 = 0.25 \\ \tau_3 &= (C_3 = 3, T_3 = 12) \Rightarrow u_3 = 0.25 \\ \tau_4 &= (C_4 = 2, T_4 = 8) \Rightarrow u_4 = 0.25 \\ \tau_5 &= (C_5 = 1, T_5 = 5) \Rightarrow u_5 = 0.2 \\ U &= 0.7 + 0.25 + 0.25 + 0.25 + 0.2 = 1.65 \end{aligned}$$

Trier par utilisation décroissante :

$$u_1 = 0.7(\tau_1) \geq u_2 = 0.25(\tau_2) = u_3 = 0.25(\tau_3) = u_4 = 0.25(\tau_4) \geq u_5 = 0.2(\tau_5)$$

Calcul des sommes :

$$U(\tau^1) = U = 1.65$$

$$U(\tau^2) = 0.25 + 0.25 + 0.25 + 0.2 = 0.95$$

$$U(\tau^3) = 0.25 + 0.25 + 0.2 = 0.7$$

$$U(\tau^4) = 0.25 + 0.2 = 0.45$$

$$U(\tau^5) = 0.2$$

$$U(\tau^6) = 0$$

Calcul de m pour chaque k :

Formule :

$$m(k) = (k - 1) + \left\lceil \frac{U(\tau^{k+1})}{1 - U_k} \right\rceil$$

• k=1 :

$$m(1) = 0 + \left\lceil \frac{U(\tau^2)}{1 - U_1} \right\rceil = \left\lceil \frac{0.95}{1 - 0.7} \right\rceil = \left\lceil \frac{0.95}{0.3} \right\rceil = \lceil 3.166... \rceil = 4$$

EDF(1) requiert 4 processeurs.

• k=2 :

$$m(2) = 1 + \left\lceil \frac{U(\tau^3)}{1 - U_2} \right\rceil = 1 + \left\lceil \frac{0.7}{0.75} \right\rceil = 1 + \lceil 0.9333... \rceil = 2$$

EDF(2) requiert 2 processeurs.

• k=3 :

$$m(3) = 2 + \left\lceil \frac{U(\tau^4)}{1 - U_3} \right\rceil = 2 + \left\lceil \frac{0.45}{0.75} \right\rceil = 2 + \lceil 0.6 \rceil = 3$$

EDF(3) requiert 3 processeurs.

• k=4 :

$$m(4) = 3 + \left\lceil \frac{U(\tau^5)}{1 - U_4} \right\rceil = 3 + \left\lceil \frac{0.2}{0.75} \right\rceil = 3 + \lceil 0.2666... \rceil = 4$$

EDF(4) requiert 4 processeurs.

• k=5 :

$$m(5) = 4 + \left\lceil \frac{U(\tau^6)}{1 - U_5} \right\rceil = 4 + \left\lceil \frac{0}{0.8} \right\rceil = 4 + \lceil 0 \rceil = 4$$

EDF(5) requiert 4 processeurs.

Conclusion :

Les valeurs trouvées :

k	1	2	3	4	5
m(k)	4	2	3	4	4

On a alors que EDF(2) est optimal, nécessitant seulement 2 processeurs, contre 4 pour EDF global (k=1).

1.29.3. Questions supplémentaire

Pourquoi EDF(1) est “mauvais” ici ?

Le test global EDF est très pessimiste quand $U_{\max}$ est grand (ici 0.7), car le dénominateur $1 - U_{\max}$ devient petit. Une tâche très “lourde” peut créer des interférences/migrations qui rendent l’analyse EDF globale défavorable.

Pourquoi EDF(k) aide ?

En “réservant” $k - 1$ processeurs aux $k - 1$ plus grosses tâches, on réduit l’interférence qu’elles imposent au reste. Le reste des tâches voit alors un test du type “EDF global” mais avec une borne basée sur U_k (plus petit), donc un $1 - U_k$ plus grand $\rightarrow$ meilleure borne.

Suffisance du test EDF(k)

EDF(k) reste un test suffisant, pas nécessaire. Il garantit la faisabilité si la condition est satisfaite, mais un système peut être faisable même si elle ne l’est pas.

1.30. Build a PFair schedule for the following system upon two processors (C, T): (2, 3)(2, 3)(4, 6). Indicate all preemptions and migrations.

1.30.1. Rappel PFair

Utilisation : Pour une tâche $\tau_i = (C_i, T_i)$, l’utilisation est :

$$u_i = \frac{C_i}{T_i}$$

Lag :

On raisonne en temps discret (quantum = 1)

Soit $S_{i(t)}$ = service reçu par τ_i sur $[0, t)$

Service fluide idéal : $u_i \cdot t$

Lag :

$$\text{lag}(\tau_i, t) = u_i \cdot t - S_{i(t)}$$

Règle PFair qui garantit les deadlines :

$$-1 < \text{lag}(\tau_i, t) < 1 \quad \forall i, \forall t \in \mathbb{N}$$

Une tâche ne peut jamais être en retard ou en avance de plus d’un quantum par rapport à son service idéal.

Fenêtre de faisabilité :

Hyperpériode :

$$H = \text{lcm}(3, 3, 6) = 6$$

1.30.2. Résolution :

Soit le système :

$$\tau_1 = (C = 2, T = 3) \Rightarrow u_1 = \frac{2}{3}$$

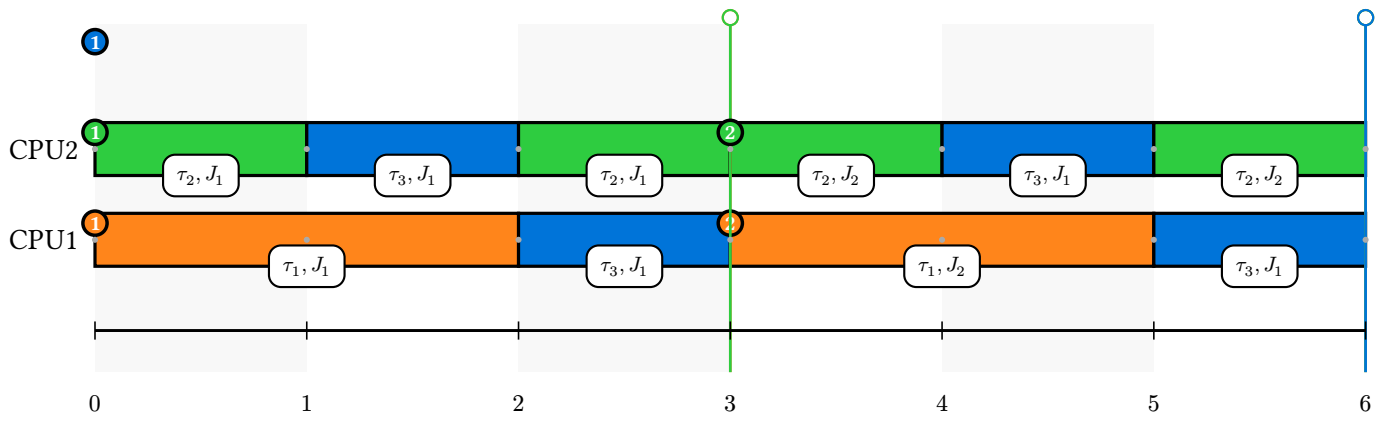
$$\tau_2 = (C = 2, T = 3) \Rightarrow u_2 = \frac{2}{3}$$

$$\tau_3 = (C = 4, T = 6) \Rightarrow u_3 = \frac{4}{6} = \frac{2}{3}$$

$$U = \sum_i u_i = 2$$

avec $m = 2$ on est pile à la limite, aucune capacité perdue donc 2 quantum exécuté à chaque temps.

On a alors une hyperpériode sur l'intervalle $[0, 6)$ car $\text{lcm}(3, 3, 6) = 6$



$t = 0 :$

$$S_1(0) = 0, S_2(0) = 0, S_3(0) = 0$$

$$\text{lag}_i(0) = 0 - 0 = 0$$

Décision pour $[0, 1) \rightarrow$ on choisit τ_1, τ_2

$t = 1 :$

$$S_1(1) = 1, S_2(1) = 1, S_3(1) = 0$$

$$\text{lag}_1(1) = \frac{2}{3} \cdot 1 - 1 = -\frac{1}{3}; \text{lag}_2(1) = \frac{2}{3} \cdot 1 - 1 = -\frac{1}{3}; \text{lag}_3(1) = \frac{2}{3} \cdot 1 - 0 = \frac{2}{3}$$

Décision pour $[1, 2) \rightarrow$ on choisit τ_3 qui a le plus grand $\text{lag}(\tau_1$ et τ_2 sont en avance donc on choisit aléatoirement, ici τ_1)

$t = 2 :$

$$S_1(2) = 2, S_2(2) = 1, S_3(2) = 1$$

$$\text{lag}_1(2) = \frac{4}{3} - 2 = -\frac{2}{3}; \text{lag}_2(2) = \frac{4}{3} - 1 = \frac{1}{3}; \text{lag}_3(2) = \frac{4}{3} - 1 = \frac{1}{3}$$

Les plus grands lag sont τ_2 et τ_3 donc on les choisit pour $[2, 3)$

$t = 3 :$

$$S_1(3) = 2, S_2(3) = 2, S_3(3) = 2$$

$$\text{lag}_1(3) = 2 - 2 = 0; \text{lag}_2(3) = 2 - 2 = 0; \text{lag}_3(3) = 2 - 2 = 0$$

Tout est égal on peut choisir n'importe quoi, on choisit τ_1 et τ_2 pour $[3, 4)$

$t = 4$:

$$S_1(4) = 3, S_2(4) = 3, S_3(4) = 2$$

$$\text{lag}_1(4) = \frac{8}{3} - 3 = -\frac{1}{3}; \text{lag}_2(4) = \frac{8}{3} - 3 = -\frac{1}{3}; \text{lag}_3(4) = \frac{8}{3} - 2 = \frac{2}{3}$$

τ_3 est plus en retard donc on le choisit avec τ_1 qui était à égalité avec τ_2 pour $[4, 5)$

$t = 5$:

$$S_1(4) = 4, S_2(4) = 3, S_3(4) = 3$$

$$\text{lag}_1(5) = \frac{10}{3} - 4 = -\frac{2}{3}; \text{lag}_2(5) = \frac{10}{3} - 3 = \frac{1}{3}; \text{lag}_3(5) = \frac{10}{3} - 3 = \frac{1}{3}$$

τ_2 et τ_3 sont en retard donc on exécute les 2

$t = 6$:

$$S_1(4) = 4, S_2(4) = 4, S_3(4) = 4$$

$$\text{lag}_i(6) = 4 - 4 = 0$$

Fin de l'hyperpériode donc retour à la normal

Préemptions :

- τ_2 est préempté à $t = 1$ pour laisser la place à τ_3
- τ_2 est préempté à $t = 4$ pour laisser la place à τ_3

Migrations :

- τ_3 migre de CPU 2 à CPU 1 à $t = 2$
- τ_3 migre de CPU 1 à CPU 2 à $t = 4$
- τ_3 migre de CPU 2 à CPU 1 à $t = 5$

1.31. Build a global EDF schedule for the following system upon two processors (C, T) : $(1, 2)(3, 6)(6, 12)$. Indicate all preemptions and migrations.

1.31.1. Rappel

Utilisation : Pour une tâche $\tau_i = (C_i, T_i)$, l'utilisation est :

$$u_i = \frac{C_i}{T_i}; U = \sum_i u_i$$

Fenêtre de faisabilité :

Hyperpériode :

$$H = \text{lcm}(2, 6, 12) = 12$$

EDF global :

Pour le k -ième quantum job de τ_i :

$$r_{i,k} = kT_i; d_{i,k} = r_{i,k} + T_i = r_{i,k} + D_i$$

À tout instant t , EDF choisit les m jobs prêts avec les plus petites deadlines absolues $d_{i,k}$.

- **Préemption** : un job s'arrête car un job plus urgent arrive.

- Migration : un job reprend plus tard sur un autre processeur que celui où il tournait avant.

1.31.2. Résolution :

Soit le système :

$$\tau_1 = (C = 1, T = 2) \Rightarrow u_1 = \frac{1}{2}$$

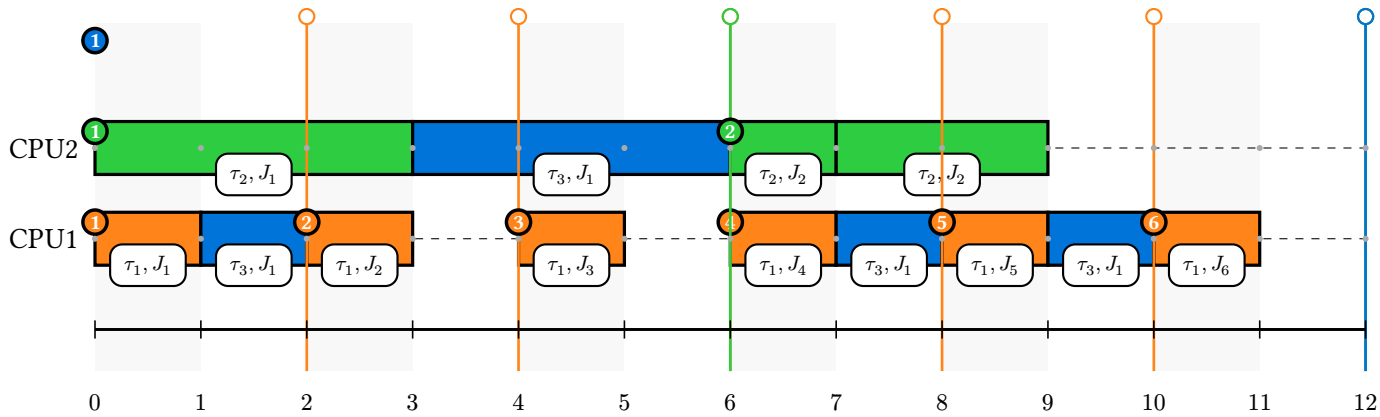
$$\tau_2 = (C = 3, T = 6) \Rightarrow u_2 = \frac{3}{6} = \frac{1}{2}$$

$$\tau_3 = (C = 6, T = 12) \Rightarrow u_3 = \frac{6}{12} = \frac{1}{2}$$

$$U = \sum_i u_i = 1.5 < 2$$

avec $m = 2$ le système est faisable.

On a alors une hyperpériode sur l'intervalle $[0, 12)$ car $\text{lcm}(2, 6, 12) = 12$



$e(J)$ = temps d'exécution restant

$t = 0$:

$$J_{1,1} : (r = 0, d = 0 + 2 = 2, e = 1)$$

$$J_{2,1} : (r = 0, d = 0 + 6 = 6, e = 3)$$

$$J_{3,1} : (r = 0, d = 0 + 12 = 12, e = 6)$$

Décision pour $[0, 1) \rightarrow$ on choisit τ_1, τ_2 (2 plus petites deadlines)

$t = 1$:

$$J_{1,1} : (r = 0, d = 2, e = 0) \text{ (terminé)}$$

$$J_{2,1} : (r = 0, d = 6, e = 2)$$

$$J_{3,1} : (r = 0, d = 12, e = 6)$$

Décision pour $[1, 2) \rightarrow$ on choisit τ_2, τ_3

$t = 2$:

$$J_{1,2} : (r = 2, d = 2 + 2 = 4, e = 1)$$

$$J_{2,1} : (r = 0, d = 6, e = 1)$$

$$J_{3,1} : (r = 0, d = 12, e = 5)$$

Décision pour $[2, 3) \rightarrow$ on choisit τ_1, τ_2

$t = 3 :$

$$J_{1,2} : (r = 2, d = 4, e = 0) \text{ (terminé)}$$

$$J_{2,1} : (r = 0, d = 6, e = 0) \text{ (terminé)}$$

$$J_{3,1} : (r = 0, d = 12, e = 5)$$

Décision pour $[3, 4) \rightarrow$ on choisit τ_3

$t = 4 :$

$$J_{1,3} : (r = 4, d = 4 + 2 = 6, e = 1)$$

$$J_{2,1} : (r = 0, d = 6, e = 0) \text{ (terminé)}$$

$$J_{3,1} : (r = 0, d = 12, e = 4)$$

Décision pour $[4, 5) \rightarrow$ on choisit τ_1, τ_3

$t = 5 :$

$$J_{1,3} : (r = 4, d = 6, e = 0) \text{ (terminé)}$$

$$J_{2,1} : (r = 0, d = 6, e = 0) \text{ (terminé)}$$

$$J_{3,1} : (r = 0, d = 12, e = 3)$$

Décision pour $[5, 6) \rightarrow$ on choisit τ_3

$t = 6 :$

$$J_{1,4} : (r = 6, d = 6 + 2 = 8, e = 1)$$

$$J_{2,2} : (r = 6, d = 6 + 6 = 12, e = 3)$$

$$J_{3,1} : (r = 0, d = 12, e = 2)$$

Décision pour $[6, 7) \rightarrow$ on choisit τ_1, τ_2

$t = 7 :$

$$J_{1,4} : (r = 6, d = 8, e = 0) \text{ (terminé)}$$

$$J_{2,2} : (r = 6, d = 12, e = 2)$$

$$J_{3,1} : (r = 0, d = 12, e = 2)$$

Décision pour $[7, 8) \rightarrow$ on choisit τ_2, τ_3

$t = 8 :$

$$J_{1,5} : (r = 8, d = 8 + 2 = 10, e = 1)$$

$$J_{2,2} : (r = 6, d = 12, e = 1)$$

$$J_{3,1} : (r = 0, d = 12, e = 1)$$

Décision pour $[8, 9) \rightarrow$ on choisit τ_1, τ_2

$t = 9 :$

$$J_{1,5} : (r = 8, d = 10, e = 0) \text{ (terminé)}$$

$$J_{2,2} : (r = 6, d = 12, e = 0) \text{ (terminé)}$$

$$J_{3,1} : (r = 0, d = 12, e = 1)$$

Décision pour $[9, 10) \rightarrow$ on choisit τ_3
 $t = 10$:

$$J_{1,6} : (r = 10, d = 10 + 2 = 12, e = 1)$$

$$J_{2,2} : (r = 6, d = 12, e = 0) \text{ (terminé)}$$

$$J_{3,1} : (r = 0, d = 12, e = 0) \text{ (terminé)}$$

Décision pour $[10, 11) \rightarrow$ on choisit τ_1
 $t = 11$:

$$J_{1,6} : (r = 10, d = 12, e = 0) \text{ (terminé)}$$

$$J_{2,2} : (r = 6, d = 12, e = 0) \text{ (terminé)}$$

$$J_{3,1} : (r = 0, d = 12, e = 0) \text{ (terminé)}$$

Décision pour $[11, 12) \rightarrow$ idle, fin de l'hyperperiode

Préemptions (EDF)

- À $t = 2$: τ_3 est préemptée (on l'exécutait sur $[1, 2)$) car $J_{1,2}$ arrive avec $d = 4 < 12$ et EDF choisit τ_1, τ_2 .
- À $t = 6$: τ_3 est préemptée (on l'exécutait sur $[5, 6)$) car $J_{1,4}$ arrive avec $d = 8$ et EDF choisit $\tau_1 +$ (départage) τ_2 parmi les jobs à $d = 12$.

Migrations

Les migrations dépendent du mapping CPU. Dans cette timeline, τ_3 s'exécute sur les segments disjoints : $[1, 2) \rightarrow$ CPU 1, $[3, 6) \rightarrow$ CPU 2, $[7, 8) \rightarrow$ CPU 1, $[9, 10) \rightarrow$ CPU 1

1.32. Provide a necessary and sufficient condition for the feasibility of implicit deadline systems upon multiprocessors.

1.32.1. Résumé

Pour des tâches périodiques/sporadiques à deadlines implicites ($D_i = T_i$) sur m processeurs identiques (préemption + migration autorisées, et un job ne s'exécute pas en parallèle sur 2 CPU), la faisabilité a un test exact :

- Aucune tâche ne demande plus qu'un processeur à elle seule.
- La somme des utilisations ne dépasse pas m

C'est à la fois nécessaire et suffisant.

1.32.2. Condition nécessaire et suffisante

Soit un système $\tau = \{\tau_1, \dots, \tau_n\}$ avec $\tau_i = (C_i, T_i)$ et $D_i = T_i$:

- **Utilisation:**

$$u_i = \frac{C_i}{T_i}; \quad U = \sum_{i=1}^n u_i$$

- **Condition de faisabilité sur m processeurs**

$$\forall i, u_i \leq 1 \quad \text{et} \quad \sum_{i=1}^n u_i \leq m$$

1. Pourquoi c'est nécessaire

- Pour τ_i , sur toute la période T_i , chaque job doit exécuter C_i avant T_i . Comme un job ne peut utiliser qu'un CPU à la fois, la vitesse max est de 1 $\Rightarrow$ il faut $C_i \leq T_i$ donc $u_i \leq 1$.

$$u_i = \frac{C_i}{T_i} > 1 \Rightarrow \text{infaisable}$$

- Sur une très grande durée L , le système demande environ $U \cdot L$ unités de calcul, alors que la plateforme fournit au plus $m \cdot L$. Si $U > m$, la demande dépasse la capacité. $\Rightarrow$ impossible

$$U > m \Rightarrow \exists L : \text{demande} > \text{capacité} \Rightarrow \text{infaisable}$$

2. Pourquoi c'est suffisant

Construisons un planning idéal et montrons qu'un ordonnanceur optimal sait l'approximer.

- **Planning fluide** : imagine que chaque tâche τ_i reçoit en continu une fraction u_i d'un processeur. Alors, sur tout intervalle $[t, t + T_i)$, elle reçoit exactement $u_i T_i = C_i \Rightarrow$ chaque job finit pile à sa deadline.
- La contrainte $\sum u_i \leq m$ garantit que la somme de ces fractions ne dépasse pas la capacité totale m
- Ensuite un ordonnanceur optimal comme PFair discrétise ce planning fluide en exécutant à chaque instant des tâches entières sur des processeurs, tout en gradant le lag borné, ce qui assure que chaque job termine avant sa deadline.

Des que $\forall i, u_i \leq 1$ et $U \leq m$, il existe un ordonnancement faisable.

1.32.3. Questions supplémentaires

1. Pourquoi $u_i \leq 1$ est obligatoire alors qu'on a m processeurs ?

Parce qu'un job ne peut pas s'exécuter sur deux processeurs simultanément $\Rightarrow$ vitesse max = 1.

1.33. PAS FINI !!!!!!!!!!!!!!! Half-proof of the fact that partitioning and global scheduling are incomparable: system such that a partition exists while all global-FTP schedulers fail.

1.33.1. Résumé

Pour montrer que partitionné et global-FTP sont incomparables, il suffit d'exhiber un système faisable par partitionnement (chaque cœur planifie "son" sous-système) mais impossible à ordonnancer par n'importe quel global-FTP. Le cours donne exactement un tel contre-exemple (appelé System 77) et l'argument "moitié-preuve" repose sur un piège de priorité : dès que tu fixes un ordre FTP entre deux tâches "longues", l'une d'elles est forcément servie trop tard et rate sa deadline.

1.33.2. Exemple

Considérons le système avec $m = 2$ à 4 tâches synchrones:

$$\begin{aligned} \tau_1 : (C_1 = 2, D_1 = 2, T_1 = 3) \quad \tau_2 : (C_2 = 3, D_2 = 3, T_2 = 4) \\ \tau_3 : (C_3 = 4, D_3 = 12, T_3 = 12) \quad \tau_4 : (C_4 = 3, D_4 = 12, T_4 = 12) \end{aligned}$$

• Montrer qu'un partitionnement existe

Essayons le partitionnement suivant :

$$\begin{aligned} \pi_1 &= \{\tau_1, \tau_3\} \\ \pi_2 &= \{\tau_2, \tau_4\} \end{aligned}$$

Sur π_1 :

$$U(\tau_1) + U(\tau_3) = \frac{2}{3} + \frac{4}{12} = \frac{2}{3} + \frac{1}{3} = 1$$

Sur π_2 :

$$U(\tau_2) + U(\tau_4) = \frac{3}{4} + \frac{3}{12} = \frac{3}{4} + \frac{1}{4} = 1$$

Chaque processeur est plein mais pas au delà de 1 donc partition plausible.

1.34. Provide Amdahl's law and illustrate its consequences.

1.34.1. Résumé

La loi d'Amdahl dit que l'accélération maximale d'un programme parallélisé est bornée par sa partie inévitablement séquentielle : même avec une infinité de processeurs, on ne dépasse pas $\frac{1}{f}$ si f est la fraction séquentielle. Conséquence clé : au-delà d'un certain nombre de cœurs, ajouter des processeurs rapporte très peu (rendements décroissants).

1.34.2. Rappel

Modèle :

- T_1 = temps d'exécution sur 1 processeur
- T_p = temps d'exécution sur p processeurs
- $f \in [0, 1]$ fraction séquentielle
- $1 - f$ fraction parallélisable

Temps sur p processeurs :

$$T_p = T_1 \left(f + \frac{1-f}{p} \right)$$

SpeedUp (accélération) :

$$S(p) = \frac{T_1}{T_p} = \frac{1}{f + \frac{1-f}{p}}$$

Borne quand $p \rightarrow \infty$:

$$\lim_{p \rightarrow \infty} S(p) = \frac{1}{f}$$

Efficacité :

$$E(p) = \frac{S(p)}{p}$$

Mesure à quel point les processeurs sont bien utilisés. $E(p)$ chute souvent quand p augmente.

1.34.3. Explication simple

Même si l'on a p processeurs :

- la partie séquentielle fT_1 ne peut pas aller au plus vite
- donc elle devient un "bottle-neck"

Plus p grandit, plus $\frac{1-f}{p}$ devient petit, et plus le temps total se rapproche de fT_1 . Donc le speedup se rapproche de $\frac{1}{f}$, quoi qu'il arrive.

1.34.4. Illustration chiffrées

1. **Exemple 1**, seulement 10% de séquentiel :

Prenons $f = 0.1$

- $p = 10$

$$S(10) = \frac{1}{0.1 + \frac{0.9}{10}} = \frac{1}{0.1 + 0.09} = \frac{1}{0.19} \approx 5.26$$

- $p = 100$

$$S(100) = \frac{1}{0.1 + \frac{0.9}{100}} = \frac{1}{0.1 + 0.009} = \frac{1}{0.109} \approx 9.17$$

- $\lim_{p \rightarrow \infty}$

$$S_{\max} = \frac{1}{0.1} = 10$$

Conclusion : même avec 100 coeurs, on n'atteins même pas $\times 10$, car 10% séquentiel impose le plafond

2. **Exemple 2**, 1% de séquentiel :

$f = 0.01 \Rightarrow S_{\max} = 100$ Cela signifie qu'il faut rendre le code extrêmement parallélisable pour espérer un gros speedup

1.34.5. Conséquences importantes

1. **Rendement décroissant**

Le gain marginal entre p et $p + 1$ diminue, on se rapproche du plafond $\frac{1}{f}$

2. **Pour scaler, il faut attaquer f , pas seulement ajouter des coeurs**

Optimiser la partie séquentielle (I/O, sections critiques, dépendances, synchronisations, algorithmes,...) a souvent un impact énorme.

3. **Strong Scaling : loi pessimiste**

Amdahl suppose une taille de problème fixe. On accélère la même tâche en mettant plus de processeurs. Dans ce cadre, le séquentiel devient dominant

4. **Overheads réels**

Dans la réalité, on a souvent :

- coûts de création/ gestion de threads,
- synchronisation, barrières,
- communication, contention mémoire/cache
- déséquilibre de charge

5. **Design**

Pour faire du bon parallélisme on peut :

- réduire les sections critiques
- éviter les dépendances globales
- découper en tâches indépendantes
- équilibrer les charges
- minimiser la communication/synchronisations
- améliorer la localité mémoire

1.34.6. Questions supplémentaires

1. Si on a $f = 0.2$, quel speedup max ?

$$S_{\max} = \frac{1}{0.2} = 5$$

2. Pourquoi on observe parfois un speedup < 1 ?

Overheads + contention + mauvais équilibrage $\Rightarrow T_p > T_1$

3. A quoi sert l'efficacité $E(p)$?

Mesurer l'utilisation des ressources. Si $E(p)$ s'écroule, ajouter des coeurs gaspille.

4. Quelle hypothèse est la plus forte dans Amdahl ?

Partie parallélisable divisée par p parfaitement + pas d'Overheads + taille fixe

1.35. Bitonic Mergesort: principes, algorithms, complexity.

1.35.1. Résumé

Le bitonic mergesort (souvent appelé bitonic sort / Batcher bitonic sorting network) est un algorithme de tri conçu pour le parallèle : il repose sur des étapes de compare-and-swap organisées en réseau.

L'idée clé est de :

1. Construire des séquences bitoniques
2. Les fusionner (merge) efficacement en parallèle

Avec une profondeur totale en $O((\log n)^2)$

1.35.2. Principe

1. Séquence bitonique

Une séquence de longueur 2^k est bitonique si elle monte puis descend (ou devient telle après une rotation cyclique)

Exemple : 3, 5, 8, 9, 7, 4, 2, 1

2. Brique élémentaire (compare and swap)

Une opération compare-and-swap prend deux éléments et les remet dans l'ordre croissant ou décroissant en 1 unité de temps

3. Lemma clef

Soit une séquence bitonique $(a_1, \dots, a_{2k})$. On compare en parallèle les paires (a_i, a_{k+i}) et on définit :

$$d_i = \min(a_i, a_{k+i}), \quad e_i = \max(a_i, a_{k+i}) \quad (i = 1 \dots k)$$

alors :

- $(d_1, \dots, d_k)$ et $(e_1, \dots, e_k)$ sont bitoniques
- $\max(d) \leq \min(e)$, donc tous les d doivent finir avant tous les e dans l'ordre final.

Intuition En une "grande" étape parallèle, tu sépares la moitié "petits" et la moitié "grands", puis tu recommences récursivement sur chaque moitié.

1.35.3. Algorithme

1. **BitonicMerge(n)** : fusionner une séquence bitonique

Pour trier une séquence bitonique de taille $n = 2^k$ en ordre croissant :

- **Etape de séparation** pour $i = 1 \dots \frac{n}{2}$, compare-and-swap $(i, i + \frac{n}{2})$ dans la direction voulue
- **Récursion en parallèle** sur les deux moitiés de taille $\frac{n}{2}$

2. **bitonicMerge(n)** : trier une séquence arbitraire

Deux niveaux :

- **Construire du bitonique** (pré-processing) à partir d'une séquence arbitraire:
 - On fait $\frac{n}{2}$ compare-and-swap sur $(a_1, a_2), (a_3, a_4), \dots$
 - Alterance : indice impairs en croissant, indices pairs décroissant
 - On obtient des bitoniques de taille 4, puis on regroupe pour faire taille 8, etc... jusqu'à bitonic de taille n .
- **Appliquer BitonicMerge** (récursivement) pour obtenir la séquence entièrement triée

Le pré-processing a une complexité du même ordre que le tri lui-même.

- **Primitive : compare-and-swap orienté**

```
COMPARE_AND_SWAP(A, i, j, direction):  
1  if direction == ASC and A[i] > A[j] then  
2      swap(A[i], A[j])  
3  if direction == DESC and A[i] < A[j] then  
4      swap(A[i], A[j])
```

- **BitonicMerge : fusionner une séquence bitonique de taille n**

```
BITONICMERGE(A, lo, n, direction):  
1  if n ≤ 1 then return  
2  k =  $\frac{n}{2}$   
3  for i = lo to lo + k - 1 do // in parallel  
4      compare_and_swap(A, i, i + k, direction)  
5  end  
6  BitonicMerge(A, lo, k, direction)  
7  BitonicMerge(A, lo + k, k, direction)
```

- **BitonicSort : trier n éléments**

```
BITONICSORT(A, lo, n, direction):  
1  if n ≤ 1 then return  
2  k =  $\frac{n}{2}$   
3  BitonicSort(A, lo, k, ASC)  
4  BitonicSort(A, lo + k, k, DESC)  
5  bitonicMerge(A, lo, n, direction)
```

1.35.4. Complexité

1. Temps parallèle (profondeur)

Dans ce modèle "beaucoup de cœurs", l'algorithme a une complexité en temps :

$$T_{\infty}(n) : O((\log n)^2)$$

C'est la profondeur du réseau de comparateurs

2. Travail total (nombre de compare-and-swap)

Le cours donne que le nombre maximal de compare-and-swap simultanés (donc le volume de comparaisons agrégé) est :

$$W(n) = O(n(\log n)^2)$$

3. Lien “Mergesort”

On l’appelle “bitonic mergesort” car le cœur est une opération de merge (BitonicMerge) appliquée de manière récursive, un peu comme mergesort, mais :

- Mergesort classique fusionne deux listes triées
- Bitonic merge fusionne une liste bitonique via un réseau fixe de comparateurs.

1.35.5. Exemple

Considérons une séquence $n = 8 = 2^3$

$$A = [7, 2, 6, 3, 8, 5, 1, 4]$$

1. Construction d'une séquence bitonic

- Compare-and-swap par paires (pré-processing)

Paire	Règle	Résultat
(7, 2)	ASC	(2, 7)
(6, 3)	DESC	(6, 3)
(8, 5)	ASC	(5, 8)
(1, 4)	DESC	(4, 1)

On a donc : $[7, 2, 6, 3, 8, 5, 1, 4] \Rightarrow [2, 7, 6, 3, 5, 8, 4, 1]$

2. Création de bitonique de taille 4

On appelle BitonicMerge(4) sur chaque bloc. Bloc 1 = $[2, 7, 6, 3]$

- Comparaison $(i, i + 2)$:

- $(2, 6) \rightarrow (2, 6)$
- $(7, 3) \rightarrow (3, 7)$

$\Rightarrow [2, 3, 6, 7]$

Bloc 2 = $[5, 8, 4, 1]$

- Comparaison $(i, i + 2)$:

- $(5, 4) \rightarrow (4, 5)$
- $(8, 1) \rightarrow (1, 8)$

$\Rightarrow [4, 1, 5, 8]$

$[2, 7, 6, 3, 5, 8, 4, 1] \Rightarrow [2, 3, 6, 7, 4, 1, 5, 8]$

3. Création de bitonique de taille 8

Comparaison centrales $(i, i + 4)$

Comparaison	Résultat
(2, 4)	(2, 4)
(3, 1)	(1, 3)
(6, 5)	(5, 6)
(7, 8)	(7, 8)

$[2, 3, 6, 7, 4, 1, 5, 8] \Rightarrow [2, 1, 5, 7, 4, 3, 6, 8]$

4. BitonicMerge sur chaque moitié

Moitié gauche $[2, 1, 5, 7]$

- Comparaison $(i, i + 2)$:

- $(2, 5) \rightarrow (2, 5)$
- $(1, 7) \rightarrow (1, 7)$

- Puis $(i, i + 1)$

- $(2, 1) \rightarrow (1, 2)$
- $(5, 7) \rightarrow (5, 7)$

$\Rightarrow [1, 2, 5, 7]$

Moitié gauche $[4, 3, 6, 8]$

- Comparaison $(i, i + 2)$:

- $(4, 6) \rightarrow (4, 6)$
- $(3, 8) \rightarrow (3, 8)$

- Puis $(i, i + 1)$

- $(4, 3) \rightarrow (3, 4)$
- $(6, 8) \rightarrow (6, 8)$

$\Rightarrow [3, 4, 6, 8]$

Résultat final

[1, 2, 3, 4, 5, 6, 7, 8]

1.35.6. Questions supplémentaires

1. Pourquoi faut-il $n = 2^k$?

Le réseau de comparateurs est défini récursivement en moitiés égales ; sinon il faut padding/traitements supplémentaires (souvent on complète à la puissance de 2 supérieure).

2. Pourquoi le merge marche ?

Grâce au lemme : après les comparaisons $(i, i + \frac{n}{2})$, tous les “petits” sont dans la première moitié et tous les “grands” dans la seconde, et chaque moitié reste bitonique $\rightarrow$ récursion.

3. Différence travail vs temps

- Temps = nombre d'étages (profondeur) $O((\log n)^2)$
- Travail = nombre total de comparaisons $O(n(\log n)^2)$

1.36. Please apply the bitonic pre-processing process in order to convert this arbitrary sequence (length $n = 2^3$): $\langle 8, 7, 9, 5, 10, 3, 12, 1 \rangle$ into a bitonic sequence of length 8.

1. Pre-processing

$A = \langle 8, 7, 9, 5, 10, 3, 12, 1 \rangle$

Paire	Règle	Résultat
(8, 7)	ASC	(7, 8)
(9, 5)	DESC	(9, 5)
(10, 3)	ASC	(3, 10)
(12, 1)	DESC	(12, 1)

$A = \langle 8, 7, 9, 5, 10, 3, 12, 1 \rangle$

$\Rightarrow \langle 7, 8, 9, 5, 3, 10, 12, 1 \rangle$

2. On obtient des blocs bitoniques de taille 4, puis on les trie en alternant $\uparrow / \downarrow$

Paire	Règle	Résultat
(7, 8, 9, 5)	ASC	(5, 7, 8, 9)
(3, 10, 12, 1)	DESC	(12, 10, 3, 1)

$A = \langle 7, 8, 9, 5, 3, 10, 12, 1 \rangle$

$\Rightarrow \langle 5, 7, 8, 9, 12, 10, 3, 1 \rangle$

1.36.1. Question supplémentaire

1. Pourquoi alterner $\uparrow / \downarrow$ sur les paires puis sur les blocs ?

Pour forcer localement une structure “montée puis descente”, afin que la concaténation des blocs triés (un $\uparrow$ puis un $\downarrow$) devienne bitonique.

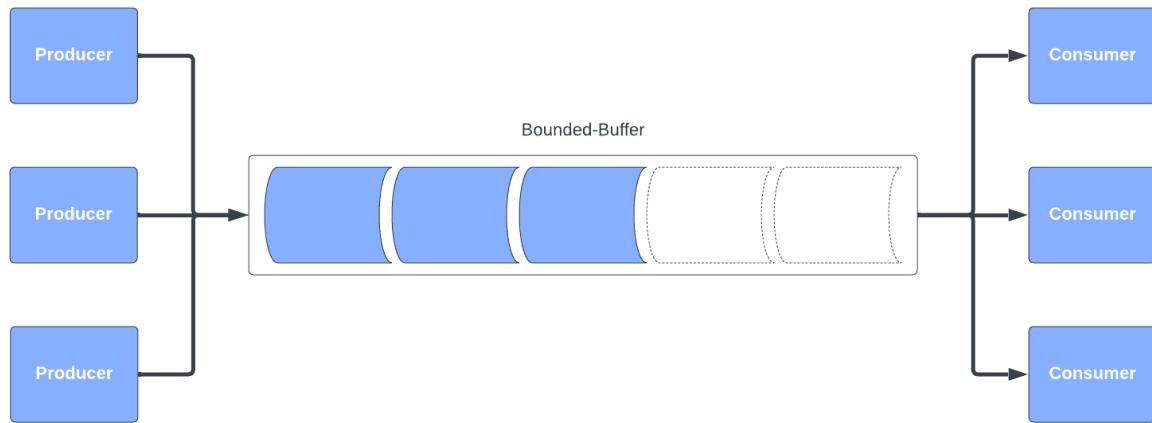
2. Est-ce que la séquence bitonique obtenue est unique ?

Non : plusieurs pré-traitements/rotations possibles peuvent produire des bitoniques différentes.
L'important est de suivre la procédure.

3. Pourquoi on veut une séquence bitonique ?

Parce que Merge_{2k} (bitonic merge) trie efficacement une bitonique en parallèle

1.37. Solution to the producer-consumer problem, bounded buffers.



1.37.1. Résumé rapide

Le problème producteur–consommateur modélise des processus qui produisent et consomment des données partagées via un tampon de capacité finie. La solution correcte impose exclusion mutuelle, synchronisation et absence de perte/blocage, typiquement avec sémaphores (ou moniteurs).

1.37.2. Idée principale

- Plusieurs producteurs déposent des éléments dans un buffer borné
- Plusieurs consommateurs retirent des éléments
- Contraintes :
 1. Pas de dépassements du buffer (si plein → producteur bloqués)
 2. Pas de lecture à vide (si vide → consumer bloqués)
 3. Accès concurrent sur la structure partagées

Ce que l'on veut garantir :

- Sécurité :
 1. Pas d'overflow, ne jamais écrire si le buffer est plein
 2. Pas d'underflow, ne jamais lire si le buffer est vide
 3. Cohérence de structure, pas de corruption des indices, pointeurs
- Vivacité :
 4. Pas de deadlock, le système ne se bloque pas tout seul
 5. Progression :
 - si le buffer n'est pas plein un producteur doit pouvoir déposer
 - si le buffer n'est pas vide un consumer doit pouvoir retirer
 6. Pas de starvation, chaque thread finit par passer

1.37.3. Solution avec sémaphores

Variables partagées :

- buffer[N] : tampon circulaire de taille N
- in, out : indices circulaires

- Sémaphores :
 - `mutex = 1` → exclusion mutuelle
 - `empty = 1` → nombre de cases libres
 - `full = 0` → nombre de cases occupées

Invariant importants

- $0 \leq \text{full} \leq N$
- $0 \leq \text{empty} \leq N$
- $\text{full} + \text{empty} = N$
- Accès à `buffer`, `in`, `out` uniquement sous `mutex`

1.37.4. Pseudo code

<pre> PRODUCER(): 1 while(true) do 2 wait(empty) // attente case libre 3 wait(mutex) // SC 4 buffer[in] = item 5 in = (in + 1) % N 6 signal(mutex) // sortie SC 7 signal(full) // un elem de plus 8 end </pre>	<pre> CONSUMER(): 1 while(true) do 2 wait(full) // attente element 3 wait(mutex) // SC 4 item = buffer[out] 5 out = (out + 1) % N 6 signal(mutex) // sortie SC 7 // une case libre de signal(empty) plus 8 consume(item) 9 end </pre>
---	---

1.37.5. Pourquoi cela fonctionne ?

- **Exclusion mutuelle**
 - `mutex` garanti qu'un seul thread modifie `buffer/in/out` à la fois. Un seul thread n'a accès à la section critique à la fois
- **Synchronisation correcte**

Pas d'overflow ni underflow

- `empty` empêche un producteur d'écrire dans un buffer plein
- `full` empêche un consumer de lire un buffer vide
- **Absence de deadlock**

Un consumer ne peut lire que si un producer a écrit

- Ordre identique des sémaphore (`empty/full` → `mutex`)
- Pas d'attente circulaire
- **Pas de starvation**
 - Si les sémaphores sont équitables (FIFO)

1.37.6. Questions supplémentaires

- **Pourquoi `mutex` est nécessaire si on a `empty/full` ?**

Parce que `empty/full` gèrent la quantité, mais pas l'accès concurrent à `in/out/buffer` (race condition possible).

- **Pourquoi l'ordre `wait(empty/full)` puis `wait(mutex)` ?**

Pour ne jamais bloquer en tenant mutex sur une condition que l'autre camp doit résoudre.

- Et si on mettait `wait(mutex)` avant `wait(empty)` ?

Deadlock : un producteur prend mutex puis bloque sur `empty==0`, empêchant un consommateur de prendre mutex pour consommer et libérer `empty`.

- FIFO garanti ?

Dépend de l'implémentation des sémaphores et de l'OS (souvent non strict), donc on formule la propriété avec une hypothèse d'équité.

1.37.7. Conclusion

Le problème producteur-consommateur avec tampon borné est résolu proprement par deux sémaphores de synchronisation (`empty`, `full`) et un sémaphore d'exclusion mutuelle (`mutex`), garantissant sûreté, progression et absence de deadlock sous hypothèses standards.

1.38. Describe the Bakery algorithm.

1.38.1. Résumé

L'algorithme Bakery (Lamport) résout le problème de section critique pour N processus en leur attribuant des tickets (comme à la boulangerie) : le plus petit ticket (puis l'ID) passe en premier. Il garantit exclusion mutuelle, absence de deadlock et absence de starvation sous des hypothèses simples de mémoire partagée.

1.38.2. Idée principale

- Chaque processus qui veut entrer en section critique prend un numéro.
- Ensuite il attend tant qu'il existe un autre processus "devant lui" dans l'ordre.
- En sortant, il rend son ticket (met son numéro à 0).

On impose un ordre total strict sur les demandes via :

- priorité au plus petit ticket,
- et en cas d'égalité, au plus petit identifiant (PID).

1.38.3. Pseudo-Code

```
BAKERYALGORITHM( $N$  processes):
1  integer array  $[1, \dots, n]$  number  $\leftarrow [0, \dots, 0]$ 
2  while(true) do
3      non-critical section
4      number $[i] \leftarrow 1 + \max(\text{number})$ 
5      for all other processes  $j$ 
6          await(number $[j] = 0$  or number $[i] \ll$  number $[j]$ )
7      critical-section
8      number $[i] \leftarrow 0$ 
9  end
```

1.38.4. Relation <<

Définition : pour 2 processus P_i, P_j :

$$(i \ll j) \iff \begin{cases} \text{number}[i] < \text{number}[j] \\ \vee \\ \text{number}[i] = \text{number}[j] \wedge i < j \end{cases}$$

On regarde le ticket ou si égalité on regarde l'identifiant

Exemple

On a $\text{number} = [1, 2, 3]$

Donc :

$$P_1 \rightarrow (1, 1), P_2 \rightarrow (2, 2), P_3 \rightarrow (3, 3)$$

Comparaison 1 par 1:

$$(1, P_1) \ll (2, P_2) \rightarrow 1 < 2 \rightarrow \text{vrai}$$

$$(2, P_2) \ll (3, P_3) \rightarrow 2 < 3 \rightarrow \text{vrai}$$

$$(1, P_1) \ll (2, P_2) \ll (3, P_3)$$

Cas avec égalité

Supposons $\text{number} = [1, 1]$

Alors $P_1 = (1, 1)$, $P_2 = (1, 2)$ donc $i = 1$ et $j = 2$

$$(1 \ll 2) \iff \begin{cases} \text{number}[i] = 1 < \text{number}[j] = 1 \rightarrow \text{FAUX} \\ \vee \\ \text{number}[i] = \text{number}[j] \wedge i < j \rightarrow \text{VRAI} \end{cases}$$

$$(2 \ll 1) \iff \begin{cases} \text{number}[i] = 1 < \text{number}[j] = 1 \rightarrow \text{FAUX} \\ \vee \\ \text{number}[i] = \text{number}[j] \wedge i < j \rightarrow \text{FAUX} \end{cases}$$

$$(1, P_1) \ll (2, P_2)$$

1.38.5. Preuve

1. Exclusion mutuelle

Supposons par contradiction que deux processus distinct i et j sont en SC en même temps:

- Pour que i entre, il a dû vérifier (pour j) : $\text{number}[j] = 0$ ou $\text{number}[i] \ll \text{number}[j]$.
- Pour que j entre, il a dû vérifier (pour i) : $\text{number}[i] = 0$ ou $\text{number}[j] \ll \text{number}[i]$.

Or si les deux sont en SC, alors $\text{number}[i] \neq 0$ et $\text{number}[j] \neq 0$. Donc on doit avoir à la fois :

- $\text{number}[i] \ll \text{number}[j]$
- $\text{number}[j] \ll \text{number}[i]$

Ce qui est impossible

2. Absence de deadlock

Si au moins un processus veut entrer, il existe un ensemble non-vide de processus $\text{number}[k] > 0$.

Comme $\ll$ est un ordre total strict, il existe un minimum (le plus petit couple $(\text{number}, \text{id})$), appelons-le p .

- Tous les autres j ont soit $\text{number}[j] = 0$, soit $(\text{number}[p], p) \ll (\text{number}[j], j)$.
- Donc les tests de p finissent par passer $\rightarrow p$ entre.

Donc pas de situation où "tout le monde attend pour toujours" (deadlock)

3. Absence de starvation (attente bornée / fairness)

Même idée : quand i prend un ticket, il se place quelque part dans un ordre total.

- Les seuls processus qui peuvent passer avant i sont ceux ayant un couple (ticket, id) strictement plus petit.
- Comme les tickets sont entiers positifs et que chaque processus remet $number=0$ en sortant, i ne peut pas être dépassé indéfiniment : ceux devant lui finissent par sortir.

Donc pas de starvation (si le système continue d'avancer).

1.38.6. Exemple

Prenons 3 processus P_1, P_2, P_3 , ils demandent tous un accès en SC presque en même temps.

Hypothèse initiale :

```
number = [0,0,0]
choosing = [false,false,false]
id = P1,P2,P3
```

- **Etape 1:** P_1 demande la section critique

1. $choosing[1] \leftarrow true$
2. $number[1] \leftarrow 1 + \max(0,0,0) = 1$
3. $choosing[1] \leftarrow false$

$number = [1, 0, 0]$

- **Etape 2:** P_2 demande la section critique

1. $choosing[2] \leftarrow true$
2. $number[2] \leftarrow 1 + \max(1,0,0) = 2$
3. $choosing[2] \leftarrow false$

$number = [1, 2, 0]$

- **Etape 3:** P_3 demande la section critique

1. $choosing[3] \leftarrow true$
2. $number[3] \leftarrow 1 + \max(1,2,0) = 3$
3. $choosing[3] \leftarrow false$

$number = [1, 2, 3]$

- **Etape 4 :** Comparaison avec la relation $\ll$

$$(number[i], i) \ll (number[j], j) \iff \begin{cases} number[i] < number[j] \\ \vee \\ number[i] = number[j] \wedge i < j \end{cases}$$

Ordre obtenu :

$$(1, P_1) \ll (2, P_2) \ll (3, P_3)$$

- **Etape 5 :** Entrée en SC

Entrée de P_1

- compare avec P_2 et P_3 :
 - $1 \ll 2$ et $1 \ll 3 \rightarrow OK$
- Entre en SC

Sortie de P_1

- $number[1] \leftarrow 0$

number = [0, 2, 3]

Entrée de P_2

- compare avec P_1 : number[1]=0 <- ignoré
- compare avec P_3 : $2 \ll 3 \rightarrow$ OK
- Entre en SC

Sortie de P_2

- number[2] <- 0

number = [0, 0, 3]

Entrée de P_3

- Tous les autres ont number=0
- Entre en SC

1.38.7. Question supplémentaires

- Pourquoi le tie-break $i < j$?

Pour éviter les ex æquo : on obtient un ordre total strict, indispensable pour prouver l'exclusion mutuelle.

- Pourquoi choosing[] ?

Parce que $\max(\text{number})$ n'est pas atomique : sans choosing[], tu peux lire un number[j] "en plein milieu" et te tromper sur l'ordre.

- number=0 signifie quoi ?

"Je ne suis pas en compétition pour la section critique" (pas en file).

- Complexité / coût ?

Chaque entrée en SC fait un scan $\max(\text{number})$ + une attente qui compare avec tous les autres $\rightarrow$ coût $O(n)$ en accès mémoire et en tests.

1.39. Two questions related to the Bakery algorithm:

1.39.1. Construct a scenario showing that the ticket numbers can be unbounded or provide an argument showing this is not possible.

1.39.1.1. Idée

Pour que les tickets soient bornés, il faudrait une constante B telle que dans toute exécution, $\text{number}[i] \leq B$ toujours. Or on peut fabriquer une exécution où un très grand ticket reste présent longtemps dans number[], forçant les nouveaux arrivants à prendre encore plus grand.

1.39.1.2. Scenario

On utilise trois processus (P_1, P_2, P_3) dans une version non atomique.

1. P_1 choisit un ticket 1, puis entre :
 - number[1]=1
2. P_2 choisit ensuite un ticket :
 - voit $\max(\text{number}) = 1 \rightarrow$ prend number[2] = 2
3. P_3 choisit ensuite un ticket :
 - voit $\max(\text{number}) = 2 \rightarrow$ prend number[3] = 3
4. Imaginon maintenant que P_3 est préempté indéfiniment ou très longtemps.

P_3 ne va donc pas faire number[3] = 0 avant un long moment. Et on garde number[3] = 3

5. P_2 et P_3 continuent à boucler à chaque fois qu'un processus veut entrer en SC il fait :

$$\text{number}[i] \leftarrow 1 + \max(\text{number})$$

Or tant que $\text{number}[3]=3$ reste, $\max(\text{number}) \geq 3$, donc on peut créer successivement des tickets 4, 5, 6, ... en retardant celui qui détient le max actuel.

1.39.1.3. Conclusion

Pour tout entier K , on peut construire une execution ou un processus obtient un ticket $> K$. Donc les tickets sont non bornés. On peut tout de meme avoir un probleme d'overflow dans la vraie vie. Donc il faut borné.

1.39.2. Construct a scenario showing that the ticket numbers can be identical or provide an argument showing this is not possible.

1.39.2.1. Cas 1 idéal, avec $\max(\text{number})$ atomique

Si on supposait que "lire max + ecrire $\text{number}[i]$ " est atomique, alors deux tickets identiques ne peuvent pas arriver :

- P_1 fait $\text{number}[1] = M+1$
- P_2 arrive apres, voit $\max = M + 1 \rightarrow \text{number}[2] = M + 2$

Donc pas d'égalité dans ce modele irréaliste.

1.39.2.2. Cas 2 réaliste,

Il est enfaite impossible de rendre $\max(\text{number})$ atomique car il faut scanner un tableau d'ou le choosing[]. **Scénario** : Supposon initialement P_1, P_2 et $\text{number} = [0, 0]$

1. P_1 : $\text{choosing}[1] = \text{true}$
2. P_2 : $\text{choosing}[2] = \text{true}$
3. P_1 calcule $\max(\text{number})=0$
4. P_2 calcule aussi $\max(\text{number})=0$ (avant que P_1 n'ecrive son numero)
5. P_1 écrit $\text{number}[1]=1$
6. P_2 écrit $\text{number}[2]=1$
7. P_1 $\text{choosing}[1]=\text{false}$
8. P_2 $\text{choosing}[2]=\text{false}$

On a bien 2 tickets identiques $\text{number}[1]=\text{number}[2]=1$ **Ca ne casse pas l'algorithme** Car la comparaison $<$ est

$$(i < j) \iff \begin{cases} \text{number}[i] < \text{number}[j] \\ \vee \\ \text{number}[i] = \text{number}[j] \wedge i < j \end{cases}$$

On obtient donc un ordre total strict sur (number, i)

Et $\text{choosing}[]$ sert à éviter de comparer $\text{number}[j]$ en plein milieu d'écriture. On attend $\text{choosing}[j]=\text{false}$ avant de tester l'ordre.

1.40. Describe the Ricart–Agrawala algorithm (distributed critical sections).

1.40.1. Résumé

L'algorithme de Ricart–Agrawala est un algorithme de section critique distribuée (exclusion mutuelle) qui n'utilise pas de coordinateur : un processus demande l'accès à la SC en diffusant une requête horodatée, et il entre en SC uniquement après avoir reçu un REPLY de tous les autres. L'ordre (et donc l'équité) est assuré par un ordre total basé sur des horloges logiques de Lamport.

1.40.2. Explication

On a N processus qui veulent accéder à une ressource partagée (SC). Pour éviter que deux entrent en même temps:

1. Quand le processus P_i veut entrer en SC il envoie à tout le monde :
 - REQUEST(ts_i , i) ou ts_i est un timestamp logique.
2. Un processus P_j qui reçoit une requête compare la priorité :
 - Si P_j n'est pas en train de vouloir la SC, il répond tout de suite REPLY
 - Si P_j veut la SC (ou est en SC) alors:
 - il répond tout de suite si la requête de P_i est plus prioritaire
 - sinon il diffère la réponse
3. P_i entre en SC quand il a reçu tous les REPLY
4. En sortant, P_i envoie les REPLY qu'il avait différés

1.40.3. Regles

• Horloges logique de Lamport

Chaque processus maintient un compteur L_i

- Avant d'envoyer un message :

$$L_i \leftarrow L_i + 1$$

Le message porte $ts = L_i$

- A la réception d'un message horodaté ts :

$$L_i \leftarrow \max(L_i, ts) + 1$$

• Ordre total entre requête (priorité)

On définit :

$$(ts_a, a) \prec (ts_b, b) \iff ts_a < ts_b \text{ ou } (ts_a = ts_b \text{ et } a < b)$$

Donc on casse les égalités avec l'identifiant du processus

• Condition d'entrée en section critique

Pour P_i :

$$\text{entrer en SC} \iff \text{REPLY reçu de tous les } P_j \neq i$$

• Complexité en message

- $N - 1$ messages REQUEST
- $N - 1$ messages REPLY
- Total :

$$M = 2(N - 1)$$

1.40.4. Explication

Variables locales (pour P_i)

- L_i : horloge logique
- requesting : booléen (vrai si P_i a demandé la SC et attend)
- $req_i = (ts_i, i)$: timestamp de sa requête courante
- deferred: ensemble des processus à qui on doit encore répondre

Pseudo-Code

DEMANDE DE LA SC(PROCESSUS I)():

```
1  requesting = true
2   $L_i \leftarrow L_i + 1$ 
3   $ts_i \leftarrow L_i$ 
4  send_REQUEST( $ts_i, i$ ) wait_REPLY() // à tout  $i \neq j$  de tout  $i \neq j$ 
5  enter_SC()
```

RECEPTION_REQUEST(ts_i, i):

```
1   $L_i \rightarrow \max(L_i, ts_k) + 1$ 
2  if ( $P_i$  in SC) or (requesting and  $req_i \prec (ts_k, k)$ ) do
3      deferred.add(k)
4  else
5      send_REPLY(k)
6  end
```

LIBÉRER_SC():

```
1  requesting = false
2  for k in deferred do
3      send_REPLY(k)
4  end
5  deferred.clear()
```

1.40.5. Pourquoi cela fonctionne :

- **Exclusion mutuelle**

Supposons par contradiction que P_i, P_k soient en SC en même temps.

- Leur requetes req_i et req_k sont comparables par l'ordre total. soit $req_i \prec req_k$, soit l'inverse
- Si $req_i \prec req_k$, alors quand P_k a envoyé REQUEST, P_i ne pouvait pas lui envoyer REPLY avant de sortir (car P_i avait priorité et diffère)

Mais P_k a besoin du REPLY de P_i pour entrer, **Contradiction**

- **Absence de deadlock**

Par l'ordre total, il existe toujours une requete minimal parmi celles en attente.

- Tous les autres processus doivent lui répondre
- Donc elle recoit tous les REPLY et entre en SC

Ca débloquent le systeme

- **Equité (starvation)**

Si :

- les canaux livrent les messages
- les processus finissent par sortir de la SC

Alors une requête ne peut pas être bloquée indéfiniment : seules des requêtes strictement plus prioritaires peuvent passer avant, et il n'y en a qu'un nombre fini "devant elle" dans l'ordre total.

1.40.6. Exemple

On a P_1, P_2, P_3 . Tous démarrent avec $L_1 = L_2 = L_3 = 0$.

On suppose une livraison rapide (pas de pertes), et on note les instants $t = 0, 1, 2, \dots$ juste pour structurer.

$t = 0 : P_1$ demande la SC

- $P_1 : L_1 := 1 ; req_1 := (1, 1)$
- Messages :
 - $P_1 \rightarrow P_2 : REQUEST(1, 1)$
 - $P_1 \rightarrow P_3 : REQUEST(1, 1)$

$t = 1 : P_2$ demande aussi la SC (quasi en même temps)

- $P_2 : L_2 := 1 ; req_2 := (1, 2)$
- Messages :
 - $P_2 \rightarrow P_1 : REQUEST(1, 2)$
 - $P_2 \rightarrow P_3 : REQUEST(1, 2)$

$t = 2 : Réceptions + décisions (comparaisons explicites)$

Chez P_2 recevant $REQUEST(1, 1)$

- Mise à jour Lamport : $L_2 := \max(L_2, 1) + 1 = \max(1, 1) + 1 = 2$
- P_2 est **requesting** avec $req_2 = (1, 2)$.
- Comparaison : $(1, 1) \prec (1, 2)$ car timestamps égaux et $1 < 2$.
- Donc la requête de P_1 est prioritaire $\Rightarrow P_2$ répond tout de suite : $P_2 \rightarrow P_1 : REPLY$

Chez P_1 recevant $REQUEST(1, 2)$

- Mise à jour Lamport : $L_1 := \max(L_1, 1) + 1 = \max(1, 1) + 1 = 2$
- P_1 est **requesting** avec $req_1 = (1, 1)$.
- Comparaison : $(1, 1) \prec (1, 2)$ (même raison).
- Ici, la requête (P_1) est prioritaire $\Rightarrow P_1$ **diffère** la réponse à P_2 :
 $deferred_1 := \{2\}$ (pas de $REPLY$ envoyé maintenant)

Chez P_3 recevant $REQUEST(1, 1)$ puis $REQUEST(1, 2)$

- Réception $REQUEST(1, 1)$:
 $L_3 := \max(0, 1) + 1 = 2$
 P_3 ne demande pas la SC $\Rightarrow$ répond : $P_3 \rightarrow P_1 : REPLY$
- Réception $REQUEST(1, 2)$:
 $L_3 := \max(2, 1) + 1 = 3$
 Toujours non-requesting $\Rightarrow$ répond : $P_3 \rightarrow P_2 : REPLY$

$t = 3 : Qui peut entrer en SC ?$

- P_1 a reçu :
 - $REPLY$ de P_2
 - $REPLY$ de P_3
 Donc P_1 a tous les $REPLY \Rightarrow P_1$ entre en SC.
- P_2 a reçu :
 - $REPLY$ de P_3
 - mais pas de $REPLY$ de P_1 (différé)

Donc P_2 attend

$t = 4 : P_1$ sort de SC (release implicite)

- P_1 sort $\Rightarrow$ il envoie les réponses différées :
 $P_1 \rightarrow P_2 : \text{REPLY}$
- Puis : $\text{deferred}_1 := \text{empty}$

$t = 5 : P_2$ entre en SC

- P_2 reçoit enfin le REPLY manquant de P_1 .
- Il a maintenant tous les REPLY $\Rightarrow P_2$ entre en SC.

Conclusion

- L'ordre total via (ts, i) force un "plus petit" gagnant.
- Ici : $(1, 1) \prec (1, 2) \Rightarrow P_1$ passe avant P_2 .
- Le REPLY différé est exactement le mécanisme qui empêche que P_2 entre tant que P_1 n'a pas fini.

1.41. Starvation definition. Illustrate the phenomenon using one of Dekker's attempts.

1.41.1. Résumé

La starvation (famine) est une violation de vivacité : un processus peut attendre indéfiniment l'accès à la section critique alors que le système continue d'évoluer. On peut l'illustrer très simplement avec la 1^{re} tentative de Dekker (strict alternation) : si un processus "ne joue jamais son tour", l'autre peut rester bloqué pour toujours.

1.41.2. Formules

- **Starvation** :

$\forall p$, si p demande la SC, alors p finit par entrer

- **Attente bornée (bounded waiting)** :

$\exists B$ tel que si p demande, alors après au plus B entrées d'autres processus, p entre

- **Progress** :

Si la SC est libre et qu'il y a des demandeurs, un demandeur doit être choisi dans un délai fini.

1.41.3. Starvation

Il existe une exécution où un processus p qui demande à entrer en section critique n'y entre jamais. Malgré le fait que les autres continuent.

En termes de propriété classique de concurrence :

- **Liveness / Progress** : "si quelqu'un veut entrer, il finit par entrer"
- **Bounded waiting** : "si p veut entrer, alors p entrera après un nombre borné d'entrées des autres"

La starvation c'est : **progress** peut encore être vrai mais **bounded waiting** est faux.

1.41.4. Illustration avec une tentative de Dekker

On considère la 1^{er} tentative de Dekker :

- Variables partagées :

$\text{turn} \in \{P, Q\}$

- Processus P :

```
await turn = P
//section critique
turn <- Q
```

- Processus Q :

```
await turn = Q
//section critique
turn <- P
```

1.41.4.1. Timeline initiale

- **Etat initial**

$$t = 0, \text{ turn} = P$$

- **Etape 1 :** Q veut entrer en SC $t = 1$

Processus	Etat
P	section non-critique
Q	bloqué sur await $\text{turn} = Q$
turn	P

- **Etape 2 :** P n'entre pas en SC $t = 2, 3, 4, \dots$

Processus	Etat
P	section non-critique (indéfiniment)
Q	bloqué sur await $\text{turn} = Q$
turn	P

- **Etape 3 :** Bloquage asymétrique

Ici Q attend un événement précis :

$$\text{turn} \leftarrow Q$$

Mais seul P peut produire cet événement mais P n'a aucune obligation de la faire.
On a donc :

$\exists$ une exécution infinie telle que Q n'entre jamais en SC

Qui est exactement la définition de la starvation.

1.41.4.2. Pourquoi ce n'est pas un deadlock

- P n'attend rien
- Q attend
- Le système continue à exécuter P

Donc:

- Pas de cycle d'attente
- Pas de blocage global
- un processus affamé pendant que l'autre vit normalement

Starvation, pas deadlock

1.41.4.3. Propriété formelle violée

Attente bornée (bounded waiting)

Il existe une borne B telle que, après la demande de Q
au plus B entrées d'autres processus peuvent précéder l'entrée de Q

Ici :

- P peut entrer **0 fois**
- Q peut attendre ∞

$\Rightarrow$ bounded waiting violée

1.42. Live-lock definition. Illustrate the phenomenon using one of Dekker's attempts.

1.42.1. Résumé rapide

Un **live-lock** est une situation où des processus restent actifs (ils exécutent des instructions) mais n'arrivent jamais à progresser vers leur objectif. Contrairement à un **deadlock**, le système n'est jamais bloqué, et contrairement à la **starvation**, aucun processus n'est ignoré par l'ordonnanceur. Dans certaines tentatives de Dekker, les processus se cèdent mutuellement la priorité indéfiniment, ce qui illustre un live-lock.

1.42.2. Rappel : objectif des algorithmes de section critique

Un algorithme correct doit garantir :

- **exclusion mutuelle**,
- **progrès** (si un processus veut entrer et que la SC est libre, il doit pouvoir entrer),
- **absence de famine** (starvation).

Le live-lock viole la propriété de **progrès**.

1.42.3. Définition formelle (live-lock)

On parle de **live-lock** lorsque :

- deux (ou plusieurs) processus continuent à s'exécuter,
- leurs états changent indéfiniment,
- mais aucun n'entre jamais dans la section critique.

Formellement :

- le système est **actif** (pas de blocage),
- mais il n'y a **aucune progression utile**.

1.42.4. Intuition

Chaque processus est « trop poli » :

- il détecte que l'autre veut entrer,
- il recule volontairement,
- mais comme l'autre fait exactement la même chose, personne n'avance.

1.42.5. Illustration avec une tentative de Dekker

Considérons deux processus P_0 et P_1 .

Variables partagées :

- $want[i] \in \{false, true\}$: intention d'entrer en SC
- (dans certaines tentatives) pas encore de variable $turn$ correcte

Étapes :

1. P_0 exécute $want[0] = true$
2. P_1 exécute $want[1] = true$
3. P_0 voit $want[1] == true \rightarrow$ recule :
 - $want[0] = false$
 - $want[0] = true$
4. P_1 voit $want[0] == true \rightarrow$ recule aussi :
 - $want[1] = false$
 - $want[1] = true$
5. Retour à la situation initiale

Ce cycle peut se répéter **indéfiniment**.

1.42.6. Pourquoi c'est bien un live-lock

- Les processus **ne sont jamais bloqués**
- Ils exécutent des instructions en boucle
- Aucun n'entre en section critique
- Le système consomme du CPU mais ne progresse pas

Donc :

- pas un deadlock (pas d'attente passive),
- pas une starvation (aucun processus n'est favorisé),
- mais bien un **live-lock**.

1.42.7. Différence avec la starvation

Propriété	Starvation	Live-Lock
Processus actif	parfois	oui
Progrès global	oui	non
Cause principale	priorité / ordonnanceur	reaction symétriques

Dans la starvation :

- un processus peut progresser,
- un autre est ignoré.

Dans le live-lock :

- **tout le monde agit**, mais **personne ne progresse**.

1.42.8. Rôle de la variable turn dans Dekker

La version correcte de Dekker introduit :

- une variable turn,
- qui brise la symétrie.

Cela empêche :

- la cession mutuelle infinie,
- donc le live-lock.

1.42.9. Ce que le prof peut demander en plus

- Comparer live-lock et deadlock sur un diagramme d'exécution
- Expliquer pourquoi turn est indispensable
- Dire si un ordonnanceur peut corriger un live-lock (non)
- Donner un autre exemple de live-lock (réseaux, backoff exponentiel mal réglé)

1.42.10. Conclusion

Le live-lock est une situation subtile où :

- le système semble actif,
- mais ne fait aucun progrès réel.

Les premières tentatives de Dekker illustrent parfaitement ce phénomène, et montrent pourquoi une brisure de symétrie est essentielle dans les algorithmes de section critique.

1.43. Describe the Byzantine Generals algorithm.

1.43.1. Résumé

Le problème des généraux byzantins modélise le consensus en présence de fautes arbitraires (dites byzantines). Il montre que pour tolérer f processus byzantins, il faut au moins $n > 3f + 1$ processus.

L'algorithme classique (OM(f)) garantit accord et validité sous hypothèse de communication fiable et synchrone.

1.43.2. Problème des généraux byzantins

On considère n processus (généraux) devant se mettre d'accord sur une décision commune (attaquer ou battre en retraite). Certains processus peuvent être **byzantins** :

- ils peuvent envoyer des messages incorrects,
- envoyer des messages différents à différents processus,
- ou ne pas envoyer de message du tout.

Objectif : obtenir un **consensus correct** malgré ces comportements arbitraires.

1.43.3. Hypothèses du modèle

- Système synchrone (temps borné pour les messages)
- Canaux de communication fiables (pas de perte, pas de corruption)
- Identité des processus connue
- Au plus f processus byzantins parmi n

1.43.4. Propriétés de correction (conditions du consensus)

On exige deux propriétés fondamentales :

Accord (Agreement) Tous les processus corrects décident la même valeur.

Validité (Validity) Si tous les processus corrects proposent la même valeur v , alors la décision est v .

(On peut aussi ajouter la **terminaison** : chaque processus correct décide en temps fini.)

1.43.5. Résultat fondamental (borne de faisabilité)

Un consensus tolérant f fautes byzantines est possible **si et seulement si** :

$$n > 3f + 1$$

Intuition :

- avec $n < 3f$, les processus byzantins peuvent créer deux visions contradictoires du système,
- les processus corrects ne peuvent pas distinguer qui ment.

1.43.6. Principe général

- Le général commandant envoie sa valeur à tous les autres.
- Chaque général relaie récursivement les messages reçus.
- Après $f + 1$ niveaux de communication, chaque général décide par majorité.

1.43.7. Définition récursive

OM(0) :

- le commandant envoie sa valeur
- les généraux adoptent la valeur reçue (ou une valeur par défaut)

OM(f), $f > 0$:

1. Le commandant envoie sa valeur à tous
2. Chaque général i exécute OM($f - 1$) en se comportant comme commandant

3. Chaque général collecte les valeurs reçues
4. Décision par majorité

1.43.8. Pourquoi $3f + 1$ est nécessaire

Pour chaque décision, un processus correct doit recevoir :

- au moins $2f + 1$ messages corrects
- pour que la majorité ne puisse pas être falsifiée par f byzantins

Ainsi :

$$n - f > 2f + 1 \rightarrow n > 3f + 1$$

1.43.9. Complexité

- Nombre de rounds : $f + 1$
- Nombre de messages : $O(n^f + 1)$

→ Algorithme théorique, peu scalable

1.43.10. Limites et remarques importantes

- Impossible en asynchrone pur (résultat FLP)
- Très coûteux en messages
- A inspiré des variantes pratiques :
 - Byzantine Fault Tolerance (PBFT)
 - Consensus blockchain (avec hypothèses supplémentaires)

1.43.11. Conclusion

Le problème des généraux byzantins formalise le consensus face à des fautes arbitraires. Il établit une borne fondamentale ($n > 3f + 1$) toujours valable aujourd'hui. C'est une pierre angulaire de la tolérance aux fautes et des systèmes distribués modernes.